2026 MANUAL 1:1 COVERAGE • FINAL AUDIT

NM6008 Modules 3 / 4 / 5

“手册原文一页不缺 + 考场可操作 SOP + 历史实战增补”。对应源文件：Module 3 22页、Module 4 9页、Module 5 5页。

> **最重要的真实性说明：**本最终 PDF 后部嵌入了三份原始 Manual 的全部 36 页，原页码、图片、表格与原文不改动。**因而 Manual 的“原文完整性”可机械验证。** 操作说明是额外解释，不替换原文；来自过去实验对话的命令以现有对话记录、截图和存档 SOP 为审计范围，不能声称遍历了已不可读取的每一条历史聊天消息。

阅读优先级	看哪里	用途
01 考试最常用	PDF 第 19–41 页：图文 SOP	按 T1→T2→T3→T4 实操；保留老师关键配图
02 发现特定报错	本增补“故障救援/命令索引”	只找报错关键词，不需要从头读
03 与老师逐句核对	M3: 第 42–63 页；M4: 第 64–72 页；M5: 第 73–77 页	共 36 页原样附入；是唯一规范文本
04 使用自动化脚本	PDF 第 78–106 页：历史完整脚本源文	未在学校所有情形经过实测，不得视为考场保底

手册来源与边界

本资料仅处理 NM6008 的 GF22 标准单元 RTL/综合/门级/功耗 Lab。另行上传的 **22FDX-PLUS PDK/DRC 保密资料** 不纳入考试指导：它们属于不同资料与使用范围，不能代替手册 GF22 cell library。

硬性约定：RTL 文件以 .v 为主；你自己的自检 TB 以 .sv 为主。老师的 Module 3 原版 TB 是 .v，本 PDF 保留原样，再额外讲 SV 适配。

AUDIT MATRIX • ALL 36 MANUAL PAGES

Manual 页码一一对应总表

下面每一个表格行对应老师手册的一整页。操作说明侧可使用本增补与后续图文 SOP；**任何遗漏的文字/图片可直接翻到末尾同页号的完整 Manual。**

M3 • 22 页

老师手册原页	必须覆盖的内容	对应 SOP / 增补
M3 P1	封面/课程名/讲师	T1 + 原版
M3 P2	学习目标与三种 Synopsys 工具	T1 + 原版
M3 P3	真值表与 Full-Adder 算式	T1 + 原版
M3 P4	§2.1 环境、RTL 准备	T1 + 原版
M3 P5	Fig.1 源码与 TB 创建	T1 + 原版
M3 P6	Fig.2(a,b) 端口映射/向量读入	T1 + 原版
M3 P7	Fig.2(c) 激励循环、写 Check.txt	T1 + 原版
M3 P8	csh + VCS 编译 + simv -gui	T1 + 原版
M3 P9	DVE Add To Waves / Start-Continue	T1 + 原版
M3 P10	Fig.5 波形、进入综合阶段	T1→T2 + 原版
M3 P11	Synthesis 目录/脚本清单	T2 + 原版
M3 P12	Fig.6 GF22 setup / Fig.7 defaults	T2 + 原版
M3 P13	Fig.8 顶层约束、命名一致	T2 + 原版
M3 P14	ReadVerilogFiles + run_topdown + DV 启动	T2 + 原版
M3 P15	Execute Script + Schematic / Fig.10	T2 + 原版
M3 P16	综合网表/SDF/SDC/日志与 GLS 复制清单	T2→T3 + 原版
M3 P17	TB 三处修改：timescale/SDF/vcdplus	T3 + 原版
M3 P18	VCS + library.f +neg_tchk + DVE	T3 + 原版
M3 P19	Fig.11 门延迟波形 / Power 目录	T3→T4 + 原版
M3 P20	修改 GF22 search path / VPD→VCD→SAIF / DC / 4 Reports	T4 + 原版
M3 P21	Basic Assignment Q1–Q4	Assignment/Optional + 原版
M3 P22	Optional 56 transitions + Critical Delay Q1–Q2	Assignment/Optional + 原版

M4 • 9 页

老师手册原页	必须覆盖的内容	对应 SOP / 增补
M4 P1	封面	Intro + 原版
M4 P2	§1 目标：序列逻辑/同步 FSM	Intro + 原版
M4 P3	Latch vs Flip-Flop 原理图 Fig.1	Intro + 原版
M4 P4	7 Inputs / 2 Outputs / FSM 概述	规格 + 原版
M4 P5	Fig.3 State transition + S/COUT 状态使能	规格 + 原版
M4 P6	state / next_state / NRST / 组合输出模板	流程 + 原版

老师手册原页	必须覆盖的内容	对应 SOP / 增补
M4 P7	FSM case + 100 ns clock TB / 同 M3 流程	流程 + 原版
M4 P8	100 ns 约束 / 合成 / GLS / Power 按 M3	流程 + 原版
M4 P9	Required gates/registers/slack + Optional FF/glitch	Assignment + 原版

M5 · 5 页

老师手册原页	必须覆盖的内容	对应 SOP / 增补
M5 P1	封面	规格 + 原版
M5 P2	§1 目标与 1×2 · 2×1 运算	规格 + 原版
M5 P3	Fig.2 五输入、17bit 输出、START 连续、OUT_STROBE	规格 + 原版
M5 P4	Fig.3 Multiplier-FF-Accumulator-FSM / signed tips	规格 + 原版
M5 P5	§2 六项必做含 inputvector.txt、短报告	Hands-on + 原版

PRE-FLIGHT · 先确认环境，别直接运行

进实验室 60 秒自检

Terminal 环境：只加载本阶段的 Synopsys 版本

T1/T3 在对应目录内执行；不要直接照抄错误路径

```
pwd
ls -la
find . -maxdepth 3 -name '*cshrc' -o -name 'library.f'
# 可在任意 Linux shell 检查
find ~/NM6008/Module3 -name '*.db' -o -name '*.sdb' | head
# 进入 C Shell，再 source 对应的环境文件
csh
source synopsys_linux_vP_2019_cshrc
which vcs
```

看到什么	怎么办
出现 shell 提示符，如 [user@host]$	Linux/csh 命令：cd, ls, cp, grep, source *_cshrc
出现 design_vision> 或 dc_shell>	DC Tcl：read_verilog, current_design, link, source *.tcl, report_power
`which vcs: Command not found`	去 T1 或 T3 目录，csh + source VCS vP-2019 环境
`which dc_shell-xg-t: Command not found`	去 T2/T4，source vR-2020 综合环境
`puts: Command not found`/`set: Variable name...`	错误地在 C Shell source 了 Tcl；要进入 dc_shell/design_vision 后再执行

> **重要：** `source scripts/run_topdown.tcl` 只能在 `design_vision>/dc_shell>` 中执行。你过去的 `script_path: Undefined variable` 就是 Shell/Tcl 混用或脚本变量未初始化造成的。

T1 • Behavioral 全套命令

课本原始方案（Module 3）

可复制命令 / 代码

```
cd ~/NM6008/Module3/Behavioral
csh
source synopsys_linux_vP_2019_cshrc
vcs -debug +v2k ./FullAdder_tb.v ./FullAdder.v
./simv -gui
```

考试推荐的分离方案（RTL .v，TB .sv）

可复制命令 / 代码

```
cd ~/NM6008/Module3/Behavioral
which vcs
vcs -full64 -sverilog -debug_access+all   ./FullAdder_tb.sv ./FullAdder.v -o simv -l compile.log
./simv -l sim.log
grep -niE 'PASS|FAIL|Error|Warning' sim.log
# 存在 VPD 时再打开
dve -vpd wave.vpd &
```

动作	正确定义
`./simv -gui`	启动带 GUI 的仿真，不一定自动生成 VPD
`./simv -l sim.log`	记录运行日志；也不自动保证 VPD
`$vcdplusfile` + `$vcdpluson`	TB 负责启动 VPD 数据记录
`$dumpfile` + `$dumpvars`	在工具支持时可以直接输出 VCD
`dve -vpd foo.vpd &`	打开已存在波形，DVE 不是独立生成信号的仿真器

VPD 生成写法

```
// 在 testbench initial 中加入（SV/Verilog）
$vcdplusfile("inter.vpd");
$vcdpluson(0, tb_FullAdder);  // 改成真实 TB scope
// stimulus + checker
$vcdplusoff;
```

> 不要混用 `FullAdder_tb` 与 `tb_FullAdder`；VCD/SAIF、SDF、Power Compiler 的实例路径由真实层次决定。

ADDED FROM PRACTICE • CHECKER / REF MODEL

T1 • 陌生题 TB 核心模板

适合你偏好的 SV Testbench（独立示例）

```
`timescale 1ns/1ps
module tb_FullAdder;
  logic A,B,CIN;
  wire S,COUT;
  int checks=0, errors=0;
  FullAdder dut(.A(A),.B(B),.CIN(CIN),.S(S),.COUT(COUT));

  initial begin
    $vcdplusfile("inter.vpd");
    $vcdpluson(0,tb_FullAdder);
    for (int n=0; n<8; n++) begin
      {A,B,CIN}=n[2:0];
      #10; // 观察并等待 DUT 稳定；不是硬件延迟
      checks++;
      if ({S,COUT} !==
          {(A^B^CIN),((A&B)|(A&CIN)|(B&CIN))}) begin
        errors++;
        $display("FAIL vector=%0d",n);
      end
    end
    if (errors==0) $display("TEST PASS: %0d checks",checks);
    else $fatal(1,"TEST FAIL: %0d errors",errors);
    $vcdplusoff;
    $finish;
  end
endmodule
```

老师原版的向量式方法同样要掌握：把 `0..7` 八行保存为 `InputVector.txt`，`$readmemh("InputVector.txt",REG_DATA)`；查看 `Check.txt`。如果目录里叫 `inputvector.txt`，Linux 大小写不一致就会找不到。

> 上面 TB 是可综合之外的**验证**代码；`#10`、`$fatal` 等都不属于 RTL 功能电路。对时序题应使用 `negedge` 驱动、`posedge` 采样后延时检查，避免竞态。

Module 3 Optional：56 种转换

共有 8 个 3-bit 输入状态。任意两个不同状态之间的一次有方向转换，共 **8 × 7 = 56** 条。手册 P22 明确将其列为 Optional；它不是 56 个静态输入组合。

56 条转换的刺激生成，先后顺序不可合并为单个状态

```
// 放在 FullAdder TB 内；可在原有 8 组合测试之后单独执行
int from_v, to_v;
for (from_v=0; from_v<8; from_v++) begin
  for (to_v=0; to_v<8; to_v++) begin
    if (from_v != to_v) begin
      {A,B,CIN} = from_v[2:0];
      #20; // 等初态稳定
      {A,B,CIN} = to_v[2:0];
      #20; // 观测最终稳态，详细延迟到 DVE 测量
      // 自检期望结果、记录两个端点/时间戳
    end
  end
end
```

你必须保存的证据	不要误判
向量 from→to，56 条的命令或日志	不能把 8 个全排列误称 56 种转换
S 和 COUT 从输入变化到输出稳定的实测时间	只选实际引发输出变化的路径
各输出的最大延迟，以及发生该延迟的输入转换	这才对应 M3 P21 Q3 / P22 Q2
使用 SDF 仿真真实门延迟（非 0 delay）	静态真值表正确不代表 Worst Case 已覆盖

T2 • Synthesis 逐文件修改总清单

文件	必须改/检查	例子
Synthesis/sources/*.v	只放本题全部 RTL，不放 TB	Adder4.v + FullAdder.v
common_setup_gf22.tcl	GF22 .db/.sdb 路径和库文件名	search_path、target_library、link_library
.synopsys_dc.setup	启动时加载哪些 Tcl、变量	通常由老师模板给出
scripts/MyDefaults.con	CLK_PERIOD、CLK_PORT、load/drive	组合题不捏造时钟
scripts/ReadVerilogFiles.tcl	每个 RTL 文件都 read_verilog	如有多个子模块别漏
scripts/TOP.tcl	set design、current_design、compile、write*	FullAdder→StateMachine→MatrixMultiplier
scripts/run_topdown.tcl	source 读取脚本与 TOP.tcl	先 read，再 compile
scripts/report.tcl	面积、参考单元、时序、功耗报告	report.rpt

展示改名和层次输出的骨架（不能直接替换老师整份 Tcl）

```
# scripts/ReadVerilogFiles.tcl: 多模块例子
read_verilog ./sources/Adder4.v
read_verilog ./sources/FullAdder.v

# scripts/run_topdown.tcl
source "${script_path}/ReadVerilogFiles.tcl"
source "${script_path}/Adder4.tcl"

# 在 Adder4.tcl 中: 按实际设计修改
set design "Adder4"
current_design Adder4
# 时序设计才创建时钟:
# create_clock -name CLK -period 100 [get_ports CLK]
compile
write -format verilog -hierarchy -output ${netlist_path}/Syn_${design}.v
write_sdf ${netlist_path}/Syn_${design}.sdf
source "${script_path}/report.tcl"
```

> 若 `script_path/netlist_path` 没有设置，必须优先确认 `.synopsys_dc.setup`、`MyDefaults.con` 与手册原脚本。不要靠猜路径。

T2 • 运行与最容易误判的报告

可复制命令 / 代码

```
cd ~/NM6008/Module3/Synthesis
csh
source synopsys_linux_vR_2020_cshrc
design_vision &
# Design Vision GUI: File -> Execute Script -> scripts/run_topdown.tcl
# 或在 design_vision> 控制台:
source ./scripts/run_topdown.tcl
report_area
report_reference
report_cell
report_timing -delay_type max -max_paths 5
report_clock
# Linux Shell 再检查产物日期和大小
ls -lh --full-time netlist/ logs/
```

M4 的 -9.35 (VIOLATED) 到底是什么?

用户实测问题的真实处理

```
# 原老师脚本中确实包含:
set_max_area 0
# 你后来查看报告明确看到:
# max_area 0.00 | Current Area 9.35 | Slack -9.35 (VIOLATED)
# 属于面积约束违例; 不是 Setup Slack 违例。
# 未为了"消红"而更改 set_max_area 0。
```

判读位置	判断标准
Area Report	leaf cells、面积、FF 数；区分 Hierarchical Block 与标准单元实例
Setup Timing Report	看 Startpoint, Endpoint, Required, Arrival, Setup Slack
Hold Timing Report	min-delay/hold 不等于 setup；0 slack 也要看约束条件
Unconstrained I/O	没有 input/output delay 的路径不能宣称完整 STA
SDF / Verilog 导出	确认输出文件时间戳是本轮；层次化子模块要导出完整
GF22 cell library	lib/db 是综合时序功耗数据；library.f 引用的 .v 才是 VCS 仿真模型

> M5 实测参考：20 ns clock、1 ns setup/hold uncertainty，曾得 Setup +14.70 ns、Hold 0 ns。只是该次实验的报告数据，**不能作为新题默认约束。**

MANUAL M3 P16-18 • NEVER SKIP THIS

T3 • 必须复制的文件、TB 三处修改

手册复制清单：RTL 大小写要以本机为准

```
cd ~/NM6008/Module3/rtl   # 注意老师文字写 RTL，但真实目录可能为 rtl
ls -la
cp ../Synthesis/netlist/Syn_FullAdder.v .
cp ../Synthesis/netlist/Syn_FullAdder.sdf .
# 手册要求 .sdc 也复制；若实际输出存在再复制：
ls ../Synthesis/netlist/Syn_FullAdder.sdc
cp ../Synthesis/netlist/Syn_FullAdder.sdc .
cp ../Behavioral/FullAdder_tb.v .
cp ../Behavioral/InputVector.txt .
cat library.f
```

手册 P17 三处必改：timescale / annotate / activity

```
`timescale 1ns/1ps
// TB 中插入（确保 DUT 确实叫 U1）：
initial $sdf_annotate("Syn_FullAdder.sdf", U1);

// 在实际 initial 测试序列开始后：
$vcdplusfile("inter.vpd");
$vcdpluson(U1);
// 在测试序列结束/finish 前：
$vcdplusoff(U1);
```

手册 P18 编译框架 + 保存日志的实用扩展

```
csh
source synopsys_linux_vP_2019_cshrc
vcs -debug +v2k +neg_tchk   ./FullAdder_tb.v ./Syn_FullAdder.v -f ./library.f   -o simv_gate -l
compile_gate.log
./simv_gate -l sim_gate.log
grep -niE 'SDF|warning|error|PASS|FAIL' sim_gate.log
ls -lh inter.vpd
```

> 你先前遇到的 `Illegal timescale for module` 应检查 TB 的 timescale 指令是否写在 module 之前，并统一网表/标准单元模型的编译时间尺度；不要去修改工艺库源文件。

T3 · SDF 生效与 DVE 波形

错误/现象	优先排查
Unknown type / unknown module/UDP	确保 -f ./library.f，路径内是提供的 GF22 simulation Verilog model
VCS_SDB_ERROR ... generated by different version	换新构建目录或备份旧 simv.daidir；用当前版本重编
SDF annotation warning	检查顶层、TB 实例、SDF 内实例路径与 netlist 层次；统计未匹配数
`+neg_tchk` warning	用于课程手册模拟兼容，不等于可以忽略真实 timing violations
波形初始 X	检查 NRST 初始化、首次采样、仿真起点
DVE 看起来“震荡”	先放大到 ns 级，再区分测试激励快速切换、真正的门级 hazard
DVE 文件不存在	VPD 由 TB 的 $vcdplus... 生成，检查当前目录和文件名
state 输出未接 DUT port	DVE/测试中可层次引用 dut.state；综合网表优化后内部名称可能消失

先保证本轮文件存在，再看 DVE

```
# 检查 VPD、日志、SDF
pwd
ls -lh *.vpd *.sdf sim_gate.log
grep -niE 'SDF|annotation|error|warning|finish' sim_gate.log
dve -vpd inter.vpd &
```

门级信号改变后立即看到窄脉冲不一定是设计错误：可能是 gate delay/glitch；评价是否可接受取决于输入变化时刻、下游采样与规格。输出注册只可声称“阻隔组合输入对端口的实时传播”；真正捕获并对比 glitch 需要相同窗口下两种门级波形。

T4 • Power 完整且不假成功

在 csh/Linux terminal 中执行

```
cd ~/NM6008/Module3/rtl/PowerAnalysis
# 在文本编辑器改 common_setup_gf22.tcl 中 search path
# 例如当前目录相对 GF22 库:
# set ADDITIONAL_SEARCH_PATH1 "../../Library/db"
# set ADDITIONAL_SEARCH_PATH2 "../../Library/sdb"
# set ADDITIONAL_SEARCH_PATH3 "../../Library ..."
# 实际位置以 ls ../../Library/db/*.db 检查为准
csh
source ../../Synthesis/synopsys_linux_vR_2020_cshrc
vpd2vcd ../inter.vpd inter.vcd
vcd2saif -i ./inter.vcd -o inter.saif
ls -lh inter.vcd inter.saif
dc_shell-xg-t
```

DC Tcl：老师四报告 + 我们额外的 SAIF 完整度校验

```
# 以下必须在 dc_shell>（不能直接在 Linux Shell）
read_verilog ../Syn_FullAdder.v
current_design FullAdder
link
# 有 CLK 的设计按约束创建时钟；组合逻辑不要捏造时钟
# create_clock -name CLK -period 100 [get_ports CLK]
read_saif -input ./inter.saif -instance FullAdder_tb/U1
report_saif -hierarchy
report_saif -missing
report_power > rpt_power.txt
report_power -hier > rpt_power_hier.txt
report_power -hier -nosplit > rpt_power_hier_nosplit.txt
report_power -cell > rpt_power_cell.txt
exit
```

> 老师手册使用 >> 追加到报告；重复跑会把新旧数据混在一起。这里改成 > 覆盖输出，考试要确保保存了原版后再覆盖。**不要把 Tcl 配置文件拿到 csh 用 source。**

PWR-201/0% 注释：即使有数字也不能当有效功耗

我们的 Module 4 Registered 一度找不到 `registered.vpd` → `registered.vcd` 转换失败 → `registered.saif` 缺失，工具仍能输出默认活动估算。修复后核实 `report_saif -hierarchy` Nets/Ports/Pins 100%，`-missing` 全为 0，才用于对比。

Module 4 完整替换表与 Optional

位置	Module 4 要替换/检查
T1 RTL Top	StateMachine：CLK, NRST, start, rst, CIN, A, B → S, COUT
FSM	S0 等 start；S1→S2→S3→S1；active 状态 rst=1 下一上升沿回 S0；NRST=0 立即回 S0
Combinational Outputs	S1/S3 使能 Sum；S2/S3 使能 Carry；其余输出 0
T1 TB	100 ns Clock Period：CLK 每 50 ns 翻转；测试异步 NRST 与同步 rst
T2 Top/clock	set design/current_design StateMachine；create_clock -period 100
T3	Syn_StateMachine.v、Syn_StateMachine.sdf、TB、library.f，反标到 dut
T4	read_verilog netlist；read_saif -instance TB_MODULE/dut；100% 覆盖目标
M4 P9 Required	Gates/Registers、Timing Slack 解释
M4 P9 Optional	Raw vs Registered output，+2 FF 的 area/power 代价，时序采样延迟

M4 RTL 核心：与手册 P6 的 #1 delayed demo 不同；这里为无仿真延迟的综合写法

```
// FSM: 时序与组合逻辑分开
always @(posedge CLK or negedge NRST)
  if(!NRST) state <= S0;
  else      state <= next_state;
always @(*) begin
  next_state = S0;
  case (state)
    S0: next_state = start ? S1 : S0;
    S1: next_state = rst ? S0 : S2;
    S2: next_state = rst ? S0 : S3;
    S3: next_state = rst ? S0 : S1;
    default: next_state = S0;
  endcase
end
```

Module 4 真实测得原版 **2 FF → 4 FF**，面积 **6.514560 → 9.354240（+43.59%）**；匹配 SAIF 的功耗 **862.8274 → 1241.8026 nW（+43.92%）**。这些是已做实验的示例，不能填入陌生题。

Module 5 独有协议与验收

规格字段	应该写进 RTL/TB 的具体检查
`MatrixMultiplier`	Top 名必须按 M5 P3；A/B 都为 8-bit signed
计算	A1×B1 + A0×B0；输入 A1/A0、B1/B0 在同一总线上顺序到达
输出	OUT[16:0]、OUT_STROBE；只在 Strobe=1 检查结果
START	启动后连续执行，只有 Reset 可中断（按手册）
架构	共享 Multiplier + Accumulator + FSM，控制 EN1/EN2/EN3
中间值	8×8 signed → 16-bit product；sum 使用 17-bit sign extension
T1 手册要求	inputvector.txt，至少正常、负数、极值、连续、复位中断
T2/T3/T4	沿用 M3，变更 Top、模块/文件名、时钟、SDF/SAIF 层次
M5 P5	6项 hands-on：RTL/TB、波形、GF22 综合、SDF 后仿、动态+漏电功耗、简报

纠正 M5 手册 P4 的 C[16] 笔误：16-bit C[15:0] 应 sign-extend C[15]

```
wire signed [15:0] p = $signed(A) * $signed(B);
wire signed [16:0] p17 = $signed({p[15], p});
wire signed [16:0] sum17 = $signed(acc17) + p17;
// 这里仅是算术片段，不能替代 FSM 与寄存器时序设计。
```

> 真实 Module 5 的一次结果：Behavioral/Gate 均 18/18 PASS；面积 1965.81、7287 cells（其中缓冲/反相器很多）；Setup +14.70 ns、Hold 0；Total Power 623.4345 µW，SAIF 目标设计 100%。**这是一次报告数据而非评分标准。**

Module 5 额外排错

你遇到过较多 SDF timing check/negative-limit warning，以及 `read_saif` 报大量未匹配对象。不能用“有功耗数字”掩盖这些情况，应核对是否真正反标到目标寄存器、activity 对应本轮实例层次、是否仍有缺失注释。

历史操作命令补齐表

命令	用途/你之前如何使用
`pwd` / `ls -la` / `ls -lh`	确认目录、隐藏文件/权限、文件大小
`cat library.f` / `less logs/report.rpt`	查看 FileList 与长报告；less 中 `/keyword` 搜索，q 退出
`find ~/NM6008 -name library.f -print`	找标准单元 FileList，注意目录大小写
`grep -RniE ... scripts/`	找 Tcl 里的 Clock/Compile/Top 定义和所在行
`grep -nE "^module |^endmodule" Syn_*.v`	判断 hierarchy netlist 是否完整
`ls -lh --full-time netlist/* logs/*`	确认重新综合真的刷新文件
`cp -a source dest`	保留权限/时间/目录层次；`cp -rf` 强制递归覆盖，不适合盲覆盖教学目录
`csh` + `source ...cshrc`	加载 VCS/DC 工具环境；必须与当前阶段对应
`which vcs`/`which dc_shell-xg-t`	判断是 PATH/环境问题还是编译问题
`vcs -full64 -sverilog -debug_access+all`	SV TB、自检可见性；以本机版本为准
`vcs ... -f ./library.f +neg_tchk`	门级模型及 timing check 兼容；不能泛化为禁用 Timing
`./simv -l file.log`	跑仿真、存日志；要 VPD 得启用 $vcdplus*
`dve -vpd file.vpd &`	图形打开已有活动数据
`vpd2vcd` / `vcd2vpd` / `vcd2saif`	波形格式转换；M3 T4 必需 vpd2vcd 与 vcd2saif
`report_saif -hierarchy` / `-missing`	活动映射覆盖/缺失核查（课程手册没展开）
`report_power -cell` / `-hier`	分单元/层次的功耗报告
`create_clock -period 100`	M4 clock setup；实际值因新题而异
`report_timing -delay_type max`	Setup/corner critical path；需加 input/output constraints
`report_area`/`report_reference`/`report_cell`	单元数量与面积，区分层次单元
`$sdf_annotate` / `$vcdpluson`	分别添加门延迟与记录活动，不是同一功能
`exit` / `Ctrl+C` / `jobs`	退出子 shell、停止前台进程、检查后台任务；quit 不是 Linux shell 内建
`git pull --ff-only`	用户项目更新的通用协作命令，不是 M3–5 Manual 必做

故障救援：代码、环境和波形

现象	第一修复动作	不要做
VCS command not found	csh; source Behavioral 或 rtl 的 VCS vP_2019_cshrc	不要误以为只是 License 丢了
dc_shell command not found	csh; source Synthesis vR_2020_cshrc	不要 source common_setup_gf22.tcl 到 csh
Unknown type `wire`	检查打错的关键字/非 ASCII 字符；确认 input wire 格式	不要盲目改成 SV interface
Syntax error at `always`	确认是否拼为 always（非 alwys）、前一行 ;/begin/end	不要只改报错这一行
Syntax error at `;`	检查 `assign S = ...;`、括号与逻辑运算符	不要漏 `endmodule`
`Illegal timescale for module`	将 `timescale` 放 module 前，统一 -timescale 编译	不要编辑 GF22 模型
VFS_SDB_ERROR	使用独立 -o/-Mdir 或备份旧 simv.daidir 后当前版本重编	不要反复覆盖旧调试数据库
DVE 所有波形 flat/X	看 Reset 初始化/Run 是否真的执行/时间轴缩放	不能凭一帧说 RTL 错
`Error: Current design is not defined UID-4`	先 read_verilog、current_design、link，再执行 cell/clock 命令	不要把启动时配置脚本报错当设计失效
`script_path: Undefined variable`	不要在 Linux csh source 设计 Tcl；检查 .synopsys_dc.setup	不要把 Tcl 当 shell 脚本
`No clocks in design`	确认组合逻辑无时钟；时序题 create_clock 且 get_ports 匹配	不要所有设计都加时钟
`max_area Slack -9.35 VIOLATED`	确认是面积约束 0 vs 实际面积，不是 Setup	不要随意修改老师优化目标

故障救援：SDF、SAIF、功耗

现象	第一修复动作	如何验收
`library.f No such file`	find 找到实际 RTL/rtl/Library 中正确文件并核对路径	cat library.f 每行存在
`Unknown module HDBSLT20_...`	检查 -f library.f 与 GF22 .v 模型	VCS elaborate 无 unresolved
`SDF instance not found`	核对 TB dut、综合网表层次、sdf 名	日志 Doing SDF annotation ... Done，检查 warnings
`inter.vpd doesn’ t exist`	核对 $vcdplusfile、$vcdpluson、运行目录与最新时间	ls -lh *.vpd
`vpd2vcd file open error`	源 VPD 是否存在/非空，文件路径、权限	输出 .vcd 非空
`vcd2saif cannot open`	先解决上一步 VCD 缺失	SAIF 文件非空
`read_saif PWR-201`	路径不对/SAIF 不存在；先 ls	read_saif 正常，report_saif 有注释
`report_saif` 0%	TB/DUT hierarchy mismatch 或错误的活动记录范围	目标 nets/pins/ports 覆盖正常
`report_saif` 100% 但 result 奇怪	检查激励时长、reset/start、相同角落、时钟频率/工具版本	同条件两份报告可比较
report_power 输出数字却没有 SAIF	不采用估计数字当本轮实测 activity-based 结果	先处理 missing activity
report_power 未保存	在 dc_shell 用 > rpt_power.txt；Shell 检查路径	grep 能看到完整报告
quit 不退出 / ctrl-c 卡住	Linux 用 exit; 前台用 Ctrl+C, 看 jobs；必要时停止实际进程	返回 shell 提示符

FINAL GATES • MUST NOT SKIP

考场交卷与反复运行签核

关卡	只需要确认这几件事
拿到题后	Top/Ports/Clock/Reset/Word Width/Signedness/Latency/Vector 输入格式写清楚
T1 PASS	从当前 RTL 编译得到新 simv；TB 与 RefModel 对照；日志 PASS；波形符合 Spec
T2 PASS	工艺库 GF22；网表、SDF、SDC（如脚本生成）、report.rpt 时间戳刷新；Timing/Area 可解释
T3 PASS	GF22 模型存在、SDF 生效、门级输出/时序正确、实际 VPD 存在
T4 PASS	VPD→VCD→SAIF 完成、SAIF instance 正确、目标注释有效、四份 Power Report
M3 P21	8组合表 + Gates + S/COUT Delay + Worst Case + 两类功耗比较
M3 P22 Optional	56 个非相同输入状态转换 + Critical Delay 及发生转换
M4 P9	Gates / Registers / Slack 解释；Optional FF 成本仅报告实际验证
M5 P5	逐拍 signed 输入 + output strobe / 6个 Hands-on 交付
重跑后	旧日志不要当新结果；检查 `ls -lh --full-time`、grep 错误、输入向量
最后存档	RTL、TB、Tcl、向量、sim/compile 日志、netlist/SDF、report、wave、SAIF、报告截图

考试用四个 `csh run`：它是我们写的参考自动化，不是 2026 Manual 承诺的功能。只有在机房以已跑通工程把 T1–T4、日志和时间戳全验证后才能依赖。无法跑通时严格回退本 PDF 逐步手动流程。

前端实验 • 考场可执行 SOP

修订版：Module 3 原图 + Module 4/5 替换卡 + 完整 TB + 排错与签核

四阶段，照着做。

以老师的 **2026 Module 3 Full-Adder Manual** 为唯一流程基准；针对 Module 4、5 只在相应步骤替换 Top / TB / 时钟 / 运算逻辑。

阶段	手册	工作目录	完成后必须看到
T1 行为仿真	§ 2.1 P4–9	Behavioral/	VCS 正常；DVE 波形
T2 综合	§ 2.2 P10–16	Synthesis/	Syn_FullAdder.v/.sdf/.sdc；report.rpt
T3 门级仿真	§ 2.3 P16–18	rtl/（按实际大小写）	SDF 注释；门级波形；VPD
T4 功耗	§ 2.4 P19–20	rtl/PowerAnalysis/	inter.saif；四份功耗报告
交卷	§ 3 P21	报告	真值表 / Cells / Delay / Power

图文原流程：按 T1→T4 依次做。**本 PDF 索引：**T1 第2–8页；T2 第9–13页；T3 第14–16页；T4 第17–19页；总表第20页；排错第21–22页；签核第23页。**四个 csh run 自动化模板尚未在实验室环境验证，考试优先按本 PDF 的手动流程。**

开始前：只核对这五项

在 Linux Terminal（不是 dc_shell）

```
cd ~/NM6008/Module3
pwd
ls -la
find . -name '*cshrc' -o -name 'library.f'
ls Library
```

记住两种提示符

[user@pc]$：Linux / C Shell；**design_vision>**、**dc_shell>**：Synopsys Tcl。**千万不要**把 Tcl 用 Linux 的 source 直接运行。

三个 Module 的 Top

M3：FullAdder • 组合逻辑
M4：StateMachine • FSM / 100 ns
M5：MatrixMultiplier • 时序数据通路

使用规则　原手册目录和脚本优先；每次执行前用 pwd、ls 核查。此 PDF 中的命令针对 Module3 实例。

T1 • RTL 设计

§2.1.A–B • 手册 P4–5

① 进入 Behavioral，编写 RTL

```
cd ~/NM6008/Module3/Behavioral
ls -la
# 打开 FullAdder.v 修改、保存
```

FullAdder.v

```
1  // ------------------------------------------------------------------------------------------
2  // Version | Programmer                                    | Date       | Remark
3  // ------------------------------------------------------------------------------------------
4  // V1      | Dr Kwen-Siong Chong (kschong@ntu.edu.sg)      | 31/07/2013 | Initial version
5  // ------------------------------------------------------------------------------------------
6  //
7  // The code is a behavioural code for a full adder
8  // The code is for teaching purpose in the NTU-TUM class, NM6008.
9
10 module FullAdder (//inputs
11                   CIN,
12                   A,
13                   B,
14                   //outputs
15                   S, COUT);
16
17 input CIN;
18 input A;
19 input B;
20
21 output S;
22 output COUT;
23
24 assign S    = A ^ B ^ CIN;
25 assign COUT = (A & B) | (CIN & (A ^ B));
26
27 endmodule
28
```

原手册 Fig. 1 • FullAdder.v 代码截图（P5）

② 完成后必须检查的内容

可复制示例 • 等价实现；以手册 Fig. 1 为对照

```
module FullAdder (
    input wire A, B, CIN,
    output wire S, COUT
);
assign S    = A ^ B ^ CIN;
assign COUT = (A & B) | (A & CIN) | (B & CIN);
endmodule
```

验收　端口 A/B/CIN、S/COUT 与手册真值表一致；RTL 文件已保存。

陌生题：先改模块名、端口位宽、组合公式或状态机；不要只更名 FullAdder 就当完成新题。

T1 · Testbench（上）

§2.1.C · 手册 P5–6

① 建立 Testbench：端口和 DUT 对齐

```
10
11  module FullAdder_tb;
12
13  parameter N = 8;
14  parameter DELAY = 50;
15
16  reg CIN;
17  reg A;
18  reg B;
19
20  wire S;
21  wire COUT;
22
23  //Input Vector
24  reg [2:0] REG_DATA [N-1: 0];
25
26  integer file1;
27  integer i;
28
29  FullAdder U1 (//inputs
30                .CIN(CIN),
31                .A(A),
32                .B(B),
33                //outputs
34                .S(S),
35                .COUT(COUT));
36
```

原手册 Fig. 2(a) · FullAdder_tb.v：IO 声明和 U1 例化（P6）

核心：FullAdder_tb 是 TB Module；U1 是 DUT Instance

```
module FullAdder_tb;
reg CIN, A, B;
wire S, COUT;
FullAdder U1 (.CIN(CIN),.A(A),.B(B),.S(S),.COUT(COUT));
// 下方继续写向量读取、initial、激励循环；最后才写 endmodule
```

② 在 Testbench 加入测试向量读取

```
37  initial
38    begin
39      file1 = $fopen("./Check.txt");
40      if (!file1) $finish;
41
42      $readmemh("./InputVector.txt", REG_DATA);
43    end
44
```

原手册 Fig. 2(b) · $readmemh + Check.txt（P6）

```
// 与手册原 TB 的 REG_DATA 声明配套
$readmemh("./InputVector.txt", REG_DATA);
```

> 手册 TB 同时打开 Check.txt，把每组输入/输出写入文件。正式验收还要对照真值表，不要将“有 Check.txt”当成“自动 PASS”。

T1 · Testbench（下）

§2.1.C · 手册 P6–7

③ 按向量循环驱动输入

```
45  initial
46  begin
47    CIN = 0;
48    A  = 0;
49    B = 0;
50
51    $fdisplay(file1,"No\tB\tA\tCIN\tS\tCOUT");
52
53    for (i = 0; i < N; i = i + 1)
54      begin
55        {B, A, CIN}  = REG_DATA[i];
56
57
58        $fdisplay(file1,"%d\t%d\t%d\t%d\t%d\t%d", i, CIN, A, B, S, COUT);
59        #(DELAY);
60
61      end
62
63    $fclose(file1);
64    $finish;
65  end
66
67  initial
68   begin
69     #(100*DELAY);
70     $finish;
71   end
72
73  endmodule
```

原手册 Fig. 2(c) · for 循环、$fdisplay、$finish（P7）

框架示意：请按你的 Check.txt 格式和 DUT 端口调整

```
// P7 片段：放在 initial begin...end 中
for (i=0; i<N; i=i+1) begin
    {B,A,CIN} = REG_DATA[i];
    #(DELAY);
    $fdisplay(file1, "%0d %b %b %b %b %b",
              i, CIN, A, B, S, COUT);
end
$finish;
```

Module 3

覆盖三输入的 **8 组**组合。若测试输入转换的 Worst Case Delay，P22 Optional 要考虑 56 种不同前后向量转换。

Module 4 / 5

必须改为按时钟、复位、握手/有效信号提供激励；MatrixMultiplier 要注意 Signed 运算和 OUT_STROBE。

验收 InputVector.txt 实际存在；TB 用 $readmemh 读它；循环数量和输入组数一致。

T1 · 可直接敲的完整 TB

补充：原手册 P5–7 的片段必须拼成完整 Testbench 才能运行

> 本页是**额外提供的可运行自检示例**，不是老师截图中的原始 TB。复制为 `Behavioral/FullAdder_tb.v`；门级阶段复制同一份 TB。

完整 FullAdder_tb.v (Verilog-2001)

```
`timescale 1ns/1ps
module FullAdder_tb;
    reg A, B, CIN;
    wire S, COUT;
    reg [2:0] REG_DATA [0:7];
    reg exp_s, exp_c;
    integer i, errors, fd;
    FullAdder U1 (.A(A), .B(B), .CIN(CIN), .S(S), .COUT(COUT));
`ifdef GATE_SIM
    initial $sdf_annotate("Syn_FullAdder.sdf", U1);
`endif
    initial begin
        $vcdplusfile("inter.vpd");
        $vcdpluson(0, FullAdder_tb);
        $readmemh("InputVector.txt", REG_DATA);
        fd = $fopen("Check.txt", "w");
        A=0; B=0; CIN=0; errors=0;
        for (i=0; i<8; i=i+1) begin
            {A,B,CIN} = REG_DATA[i];
            #20; // 比较前等待组合逻辑稳定（含SDF）
            exp_s = A ^ B ^ CIN;
            exp_c = (A&B) | (A&CIN) | (B&CIN);
            $fdisplay(fd,"%0d %b%b%b %b %b",i,A,B,CIN,S,COUT);
            if ({S,COUT} !== {exp_s,exp_c}) begin
                errors = errors+1;
                $display("FAIL case=%0d",i);
            end
        end
        $fclose(fd);
        if (errors==0) $display("TEST PASS: 8 checks");
        else $display("TEST FAIL: %0d errors",errors);
        $vcdplusoff;
        $finish;
    end
endmodule
```

InputVector.txt：8 行十六进制数

```
0
1
2
3
4
5
6
7
```

通过标准　T1 日志出现 TEST PASS: 8 checks；T3 启用 GATE_SIM 后必须仍然 PASS。

重要　这里的 8 次检查只验证静态真值表，不能测 Worst-Case。**向量位序要一致：**本示例 {A,B,CIN}，原手册采用 {B,A,CIN}。不要混用。

T1 · 编译 / DVE 查看

§2.1.D · 手册 P8–10

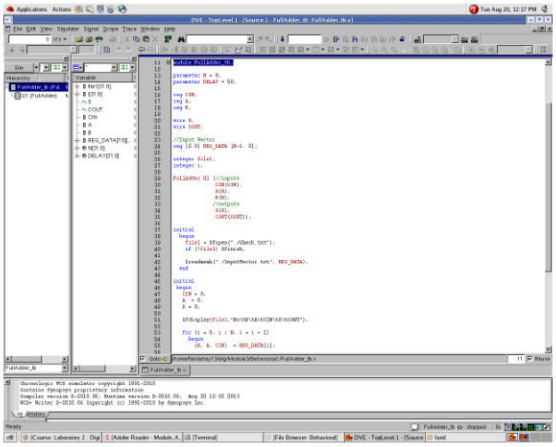
① 启动 VCS（原手册 P8）

环境 → 编译 → DVE GUI

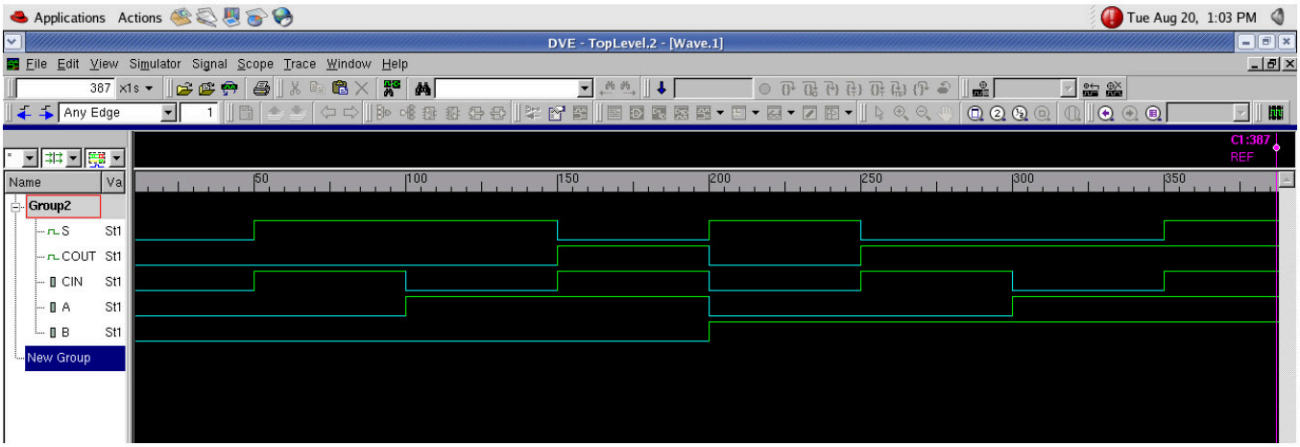
```
cd ~/NM6008/Module3/Behavioral
csh
source synopsys_linux_vP_2019_cshrc
which vcs
vcs -debug +v2k ./FullAdder_tb.v ./FullAdder.v
./simv -gui
```

② DVE 中实际点击

1. 选择 **FullAdder_tb** 层次 → 选 **A / B / CIN / S / COUT**。
2. 右键 **Add To Waves → New Wave View**。
3. 点击 **Simulator → Start/Continue**，查看响应。



原手册 Fig. 4 · DVE 层次与信号窗口（P9）



原手册 Fig. 5 · 行为仿真波形（P10）

两种运行方式：./simv -gui 用于交互调试；./simv -l sim.log 保存文本日志。**均不保证产生 VPD**；VPD 是否写出取决于 TB 波形记录设置。

验收　DVE 波形可读，8 组输入与输出符合真值表，保留截图。

Module 4 • FSM 替换卡

Module 4 手册 P4–9：在 T1/T2/T3/T4 的同一位置替换

手册状态转换

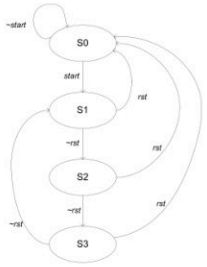


Fig. 3 The state transition diagram

Consider now the 2 outputs, namely *S* and *COUT*. In this exercise, the outputs are required to be realized as combinational logic. In both S1 and S3 stage, the *S* output will function as a Sum signal according to the three input signals, *A*, *B*, and *CIN*. In both S2 and S3 stage, the *COUT* signal will function as a Carry signal according to the three input signals, *A*, *B*, and *CIN*. Otherwise, both the *S* and *COUT* signals are '0'. Refer to the Module 3 on how the *S* and *COUT* signals can be generated.

实现重点

NRST=0	异步状态清零 S0
S0 + start	下一上升沿进 S1
S1→S2→S3→S1	rst=0 自动循环
S1/S2/S3 + rst	下一上升沿回 S0
S 输出	S1/S3 开 Sum
COUT 输出	S2/S3 开 Carry

替换 Behavioral/StateMachine.v 的参考实现

```
module StateMachine (
    input wire CLK, NRST, start, rst, CIN, A, B,
    output wire S, COUT
);
localparam [1:0] S0=2'b00,S1=2'b01,S2=2'b10,S3=2'b11;
reg [1:0] state, next_state;
always @(posedge CLK or negedge NRST)
    if (!NRST) state <= S0;
    else       state <= next_state;
always @(*) begin
    next_state = S0;
    case(state)
        S0: next_state = start ? S1 : S0;
        S1: next_state = rst ? S0 : S2;
        S2: next_state = rst ? S0 : S3;
        S3: next_state = rst ? S0 : S1;
        default: next_state = S0;
    endcase
end
wire sum=A^B^CIN;
wire carry=(A&B)|(A&CIN)|(B&CIN);
assign S    = ((state==S1)||(state==S3)) && sum;
assign COUT = ((state==S2)||(state==S3)) && carry;
endmodule
```

手册 vs 工程 M4 P6 使用 `state <= #1 ...` 来表示仿真延迟；此处给出可综合写法，不加 #1。手册 P8 写 “100ns frequency”，实际是 **Clock Period = 100 ns**，TB 可用 `forever #50 CLK=~CLK;` TB 在时钟非采样沿驱动信号以避免竞态。

Module 5 • 乘累加替换卡

Module 5 手册 P3–5：输入 A/B 逐拍传输，结构由老师 Fig. 3 决定

架构图（原手册 P4）

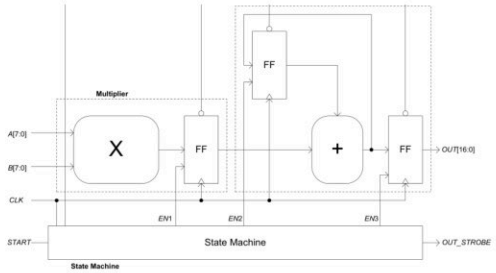


Fig. 3 A possible architecture of the Matrix Multiplier.

Students are expected to follow the architecture shown in Fig. 3 to realize the Matrix Multiplier.

1.2.3 Tips

(i) The signed multiplication can be modelled using the following code (where the *P* is output, and *A* and *B* are inputs)

```
assign P = $signed(A) * $signed(B);
```

(ii) For simplicity, all interfaced signals between the three building blocks can be registered (i.e. generated as sequential logic as opposed to combinational logic).

(iii) Signed extension is required for the Accumulator. For example, if the signal *D*

先读懂规范

Top： MatrixMultiplier

Input： CLK, NRST, START, signed A[7:0], B[7:0]

Output： OUT[16:0], OUT_STROBE

输出为 **A1×B1 + A0×B0**；A1/A0、B1/B0 共用输入总线，按拍输入。

> **START** 拉高后连续运算，手册说仅 Reset 可中断。
> OUT_STROBE=1 才认为 OUT 有效。

Signed 运算正确位宽：仅供你改 RTL 的片段

```
// Module 5 文档重点：signed 8-bit 输入，经两拍输入两对 A/B
wire signed [7:0] A_s = $signed(A);
wire signed [7:0] B_s = $signed(B);
wire signed [15:0] product = A_s * B_s;
// 将 16-bit 有符号积扩展成 17-bit 再做加法：
wire signed [16:0] product_ext = {product[15], product};
// 这只是运算片段，不是完整的 MatrixMultiplier
// 真正 RTL 必须另写 START/FSM/EN1/EN2/EN3/FF/OUT_STROBE
```

陌生题 TB 必测

数据与符号	(127×127)+(-128×127)，负数与最大/最小输入
协议	复位→START→第一/第二对数据→OUT_STROBE
连续性	保持 START，检查连续结果和 Strobe 周期
边界	中途 NRST、输入在非采样边沿切换、OUT 无效时不做数值比较

> **手册 P4 的注意点：** 示例说 C 为 16-bit，却写 `C[16]`；对 16-bit 的 `C[15:0]` 应使用 `{C[15],C}`。这是按位宽修正，不是照抄手册代码。

T2 · 综合环境 / 工艺库

§2.2.A–B · 手册 P10–12

① 将 RTL 复制到综合目录（P11）

```
cd ~/NM6008/Module3/Synthesis
cp ../Behavioral/FullAdder.v sources/
ls -la
ls scripts/ sources/ netlist/ logs/
```

② 检查 GF22 配置与默认约束（P11–12）

```
Info: Running script [info script]\n"

##########################################################################
 Setup Variables
##########################################################################

 following variables, use a blank space to separate multiple entries.
: set TARGET_LIBRARY_FILES "lib1.db lib2.db lib3.db"

IONAL_SEARCH_PATH1        " ../Library/db";#  Additional search path
IONAL_SEARCH_PATH2        "../Library/sdb"       ;#  GF library cells
IONAL_SEARCH_PATH3        "../Library $SYNDIR/libraries/syn ./scripts ./sources ./ddc" ;
onal search path

T_LIBRARY_FILES          "gf22nsdvlogl20hdl116a_TT_0P90V_0P00V_0P00V_0P00V_25C.db" ;#  Target technology logical lib
T_LIBRARY_SYMBOL         "gf22nsdvlogl20hdl116a.sdb"                               ;#  Target technology logical librarie

IONAL_LINK_LIB_FILES      ""  ;#  Extra link logical libraries not included in TARGET_LIBRARY_FILES

IBRARY_FILES              ""  ;#  List of max min library pairs "max1 min1 max2 min2 max3 min3"...

FERENCE_LIB_DIRS          ""  ;#  Milkyway reference libraries (include IC Compiler ILMs here)

FERENCE_CONTROL_FILE      ""  ;#  Reference Control file to define the Milkyway reference libs

FILE                      ""  ;#  Milkyway technology file
ILE                       ""  ;#  Mapping file for TLUplus
JS_MAX_FILE               ""  ;#  Max TLUplus file
JS_MIN_FILE               ""  ;#  Min TLUplus file

OUTING_LAYER             ""   ;# Min routing layer
OUTING_LAYER             ""   ;# Max routing layer
```

原手册 Fig. 6 · common_setup_gf22.tcl：GF22 库和路径配置（P12）

```
#### ------------------------------------------------------
#### | Define clock or virtual clock for the modules
#### ------------------------------------------------------

#### Define clock variables
set CLK_PERIOD 20
set FIX_DELAY 10
set CLK_SKEW [expr ($CLK_PERIOD * 0.05)]
set INPUT_DELAY [expr ($CLK_PERIOD * 0.15)]
set OUTPUT_DELAY [expr ($CLK_PERIOD * 0.15)]

## Create real clock if clock port is found
if {[sizeof_collection [get_ports CLK]] > 0} {
   set CLK_PORT CLK
   create_clock -period $CLK_PERIOD CLK
}

## Create virtual clock if clock port is not found
if {[sizeof_collection [get_ports CLK]] == 0} {
   set CLK_PORT VCLK
   create_clock -period $CLK_PERIOD VCLK
}

## Apply default timing constraints for modules
set_input_delay $INPUT_DELAY -max -clock $CLK_PORT [remove_from_collection [all_inputs] $CLK_PORT]
set_output_delay $OUTPUT_DELAY -max -clock $CLK_PORT [all_outputs]
set_clock_uncertainty -setup $CLK_SKEW $CLK_PORT
set_clock_uncertainty -hold $CLK_SKEW $CLK_PORT

#### ------------------------------------------------------
#### | Define Design Environment
#### ------------------------------------------------------

## Define variables
set DRIVE_CELL H565_GS_IVX2
set DRIVE_PIN {Z}
set OUTPUT_LOAD [load_of CORE65GPSVT/H565_GS_IVX2/A]
set MAX_OUTPUT_LOAD [expr ($OUTPUT_LOAD * 4)]
set WIRELOAD_MODEL 10k

set_load $MAX_OUTPUT_LOAD [all_outputs]
set_drive 1.5 [all_inputs]

## If real clock, set infinite drive strength
if {[sizeof_collection [get_ports CLK]] > 0} {
   set_drive 0 CLK
}
```

原手册 Fig. 7 · MyDefaults.con：默认时钟、负载、驱动条件（P12）

> common_setup_gf22.tcl 必须指到可找到 .db/.sdb 的目录；MyDefaults.con 中约束按当前实验 Spec 核对。别把时序电路的 Clock Period 机械照搬到组合加法器。

T2 • 综合 Tcl 怎么改

§2.2.B • 手册 P13–14

① 修改三个关键 Tcl 文件

```
# scripts/ReadVerilogFiles.tcl
read_verilog ./sources/FullAdder.v

# scripts/run_topdown.tcl
source "${script_path}/ReadVerilogFiles.tcl"
source "${script_path}/FullAdder.tcl"
```

② 检查 FullAdder.tcl 的 Top 与综合输出

```
# The script is designed for teaching purpose
# ------------------------------------------------------------
# Version | Programmer                | Date       | Remark
# ------------------------------------------------------------
# V1      | Dr Kwen-Siong Chong       | 31/07/2013 | Initial version
# ------------------------------------------------------------

# Script file for constraining the design
set rpt_file "report.rpt"
set design "FullAdder"

current_design FullAdder
source "${script_path}/MyDefaults.con"

# Define design environment
set_load $MAX_OUTPUT_LOAD [all_outputs]
set_driving_cell -lib_cell $DRIVE_CELL [all_inputs]

# Define design constraints
#set_input_delay  $INPUT_DELAY -clock $CLK_PORT [all_inputs]
#set_output_delay $OUTPUT_DELAY -clock $CLK_PORT [all_outputs]
set_max_area 0

# Ungroup DesignWare parts
set designware_cells [get_cells -filter "@is_oper==true" *]
if {[sizeof_collection $designware_cells] > 0} {
   set_ungroup $designware_cells true
}

#set_fix_hold [all_clocks]
compile
write -format verilog -output ${netlist_path}/Syn_$design.v -hier
write -format ddc -hierarchy -output "${ddc_path}/${design}.ddc"
write_sdf ${netlist_path}/Syn_$design.sdf
write_sdc ${netlist_path}/Syn_$design.sdc
source "${script_path}/report.tcl"
```

原手册 Fig. 8 • FullAdder.tcl 设计约束、compile 和 netlist 输出（P13）

```
# scripts/FullAdder.tcl 中要核对的核心语句
set design "FullAdder"
current_design FullAdder
source "${script_path}/MyDefaults.con"
compile
# 原脚本的 write -format verilog、write_sdf 等保留
```

验收 三个文件中的 FullAdder 名称必须对应 RTL 顶层；MyDefaults.con 与工艺库路径没有引用旧工程。

T2 · 综合脚本逐文件改

补充：陌生题最容易漏掉 Top、Clock、报告路径和重新编译

文件都在 Synthesis/ 下；每改一个立刻保存

1 · sources/	放本题的全部 RTL .v（不要放 TB）；文件名在下一步引用
2 · common_setup_gf22.tcl	保留 GF22 target/link library 名称；路径指向实际 Library/db、sdb
3 · scripts/ReadVerilogFiles.tcl	每个待综合 RTL 一条 read_verilog；或按原脚本格式
4 · scripts/run_topdown.tcl	第二条 source 指向本题 Top.tcl
5 · scripts/Top.tcl	set design、current_design、Clock/Load、compile、write/netlist/sdf
6 · scripts/MyDefaults.con	CLK_PORT、CLK_PERIOD、MAX_OUTPUT_LOAD 等按题目；不要把模板约束当真实 Spec
7 · scripts/report.tcl	至少 report_area / report_timing / report_reference / report_power

例：陌生顶层 Adder4 的三份 Tcl

```
# ReadVerilogFiles.tcl
read_verilog ./sources/Adder4.v
read_verilog ./sources/FullAdder.v

# run_topdown.tcl
source "${script_path}/ReadVerilogFiles.tcl"
source "${script_path}/Adder4.tcl"

# Adder4.tcl（保留老师脚本的其他变量和 write 命令）
set design "Adder4"
current_design Adder4
source "${script_path}/MyDefaults.con"
# 时序题才创建时钟；组合逻辑不凭空添加 CLK
create_clock -name CLK -period 100 [get_ports CLK]
compile
write -format verilog -output ${netlist_path}/Syn_${design}.v
write_sdf ${netlist_path}/Syn_${design}.sdf
source "${script_path}/report.tcl"
```

不要直接照抄示例到组合逻辑题：没有 CLK 端口时 `create_clock` 必须删除/调整。上述 Tcl 依赖老师现成的 `script_path/netlist_path`，不是从空目录运行的完整配置。

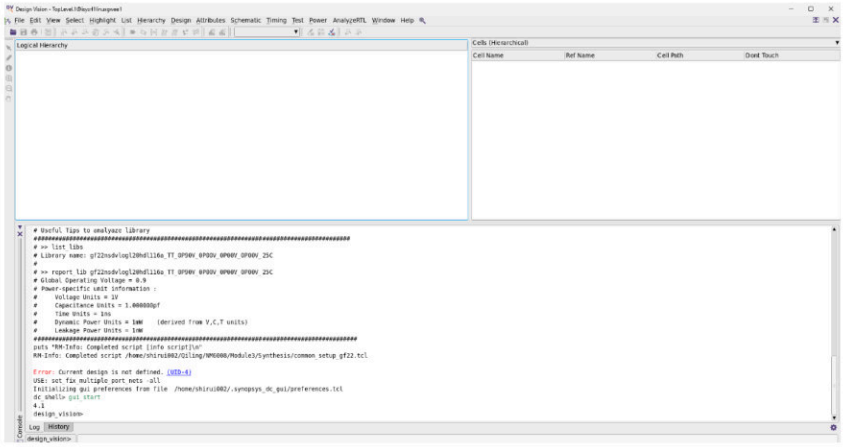
T2 • 执行综合 / 验收

§2.2.C • 手册 P14–16

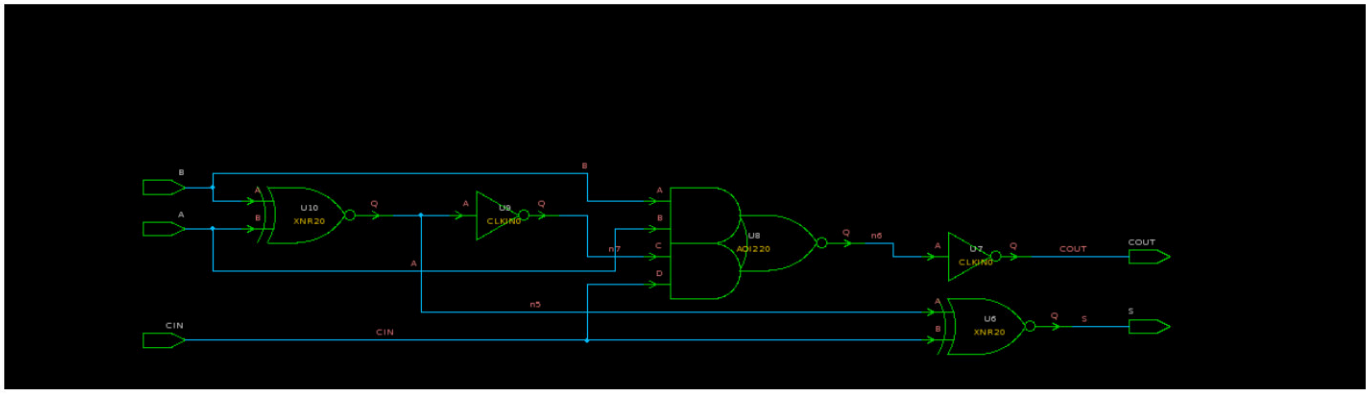
① 启动 Design Vision

```
cd ~/NM6008/Module3/Synthesis
csh
source synopsys_linux_vR_2020_cshrc
design_vision &
```

在 GUI 里： File → Execute Script → scripts/run_topdown.tcl。
综合后选 Logical Hierarchy 的 **FullAdder** → 右键 **Schematic View**。



原手册 Fig. 9 • Design Vision GUI（P15）



原手册 Fig. 10 • 综合后的标准单元 Schematic（P15）

② 在 Linux Terminal 核对输出（P16）

```
ls -lh netlist/Syn_FullAdder.v
ls -lh netlist/Syn_FullAdder.sdf
ls -lh netlist/Syn_FullAdder.sdc
less logs/report.rpt
```

验收 能打开综合网表、SDF、SDC 和 report.rpt；报告应能识别 Cells、Area 和 Timing。

T2 • 报表判读与事故预防

依据 M3 P15–16 与 M4 P8–9；先分清 Area 还是 Timing 的 VIOLATED

综合后一条一条检查（在 design_vision>）

交互式 TCL：用于补查，不替代原 run_topdown

```
current_design
report_reference
report_area
report_clock
report_timing -delay_type max -max_paths 5
report_constraint -all_violators
```

常见三种现象

No clocks in this design	M3 组合 FullAdder 没时钟可能正常；M4/M5 需核对 create_clock
Slack (MET) +99.94 ns	只代表报告列出的受约束路径；检查其他 I/O、hold、其他路径
Slack -9.35 (VIOLATED)	可能是 max_area 0 产生的 **面积违例**：看上下文 max_area/Current Area，不是直接认定 setup 失败

快速核对是否生成了本轮的新结果

Linux Terminal

```
cd ~/NM6008/Module3/Synthesis
ls -lh --full-time netlist/Syn_*.v netlist/Syn_*.sdf
ls -lh --full-time logs/report.rpt
grep -inE 'Slack|Area|cell|Error|Warning' logs/report.rpt | head -35
```

过关 Top 匹配；GF22 库映射成功；能辨认 Gates、FF、Area 和受约束 Timing；输出是刚刚重新生成的。

Module 4 历史处理 保留老师脚本 `set_max_area 0`，并在报告解释“面积目标为 0 的违例”。只有题目明确要求不同面积目标，才修改此约束；不要为刷绿结果而随便设大数。

T3 · 门级 TB 必改三处

§2.3.A–B · 手册 P16–17

① 准备门级工作目录（手册 P16）

```
cd ~/NM6008/Module3/rtl  # 手册有 RTL/rtl 混用，实际用 ls 确认
cp ../Synthesis/netlist/Syn_FullAdder.v .
cp ../Synthesis/netlist/Syn_FullAdder.sdf .
cp ../Synthesis/netlist/Syn_FullAdder.sdc .
cp ../Behavioral/FullAdder_tb.v .
cp ../Behavioral/InputVector.txt .
ls -lh library.f
```

② 在 FullAdder_tb.v 插入 P17 三处修改

```
8
9   `timescale 1ns / 1ps
10
11  module FullAdder_tb;
12
```

原手册 P17 修改 1：timescale 1ns/1ps

```
`timescale 1ns/1ps
```

```
27  integer i;
28
29  //Backannotation //Added
30    initial $sdf_annotate("Syn_FullAdder.sdf", U1);
31
32  FullAdder U1 (//inputs
```

原手册 P17 修改 2：SDF back-annotation

```
initial $sdf_annotate("Syn_FullAdder.sdf", U1);
```

```
52    B = 0;
53    $vcdpluson(U1);
54
55    $fdisplay(file1,"No\tB\tA\tCIN\tS\tCOUT");
56
57    for (i = 0; i < N; i = i + 1)
58      begin
59        {B, A, CIN}  = REG_DATA[i];
60
61        #(DELAY);
62        $fdisplay(file1,"%d\t%d\t%d\t%d\t%d\t%d", i, CIN, A, B, S, COUT);
63
64      end
65
66    $fclose(file1);
67    $vcdplusoff(U1);
68    $finish;
```

原手册 P17 修改 3：在激励前后开启/关闭切换记录

```
// 放在 TB 的 initial begin ... end 中
$vcdplusfile("inter.vpd");
$vcdpluson(U1);
// 在此执行输入激励与输出检查
$vcdplusoff(U1);
```

> 三行 VPD 命令应放进原 TB 的 initial begin...end，不要写在模块顶层。U1 是 DUT 实例名。手册只写 on/off；这里补充文件名，保证 P20 能找到 ../inter.vpd。

T3 · 完整门级命令与反标检查

补充：考试中最容易在 SDF / 模型 / 文件名上耗时间

先检查文件，后敲 VCS

Linux Terminal 复制版

```
cd ~/NM6008/Module3/rtl
pwd
ls -lh Syn_FullAdder.v Syn_FullAdder.sdf library.f
cat library.f
# 逐条确认 -v 后面的 gf22 模型路径真的存在
# TB 已复制、InputVector.txt 存在：
ls -lh FullAdder_tb.v InputVector.txt
csh
source synopsys_linux_vP_2019_cshrc
which vcs
# -f 在前，尽量避免模型与 TB 的 timescale 顺序冲突
vcs -debug +v2k +neg_tchk -f ./library.f \
    +define+GATE_SIM ./Syn_FullAdder.v ./FullAdder_tb.v \
    -o simv_gate -l compile_gate.log
./simv_gate -l sim_gate.log
ls -lh inter.vpd
# 已有 VPD：dve -vpd inter.vpd &
```

检查 SDF 不是“有文件但没生效”

Linux Terminal

```
grep -niE 'SDF|annotat|Error|Warning' sim_gate.log | head -40
# 查看传播延迟：DVE 放大输入改变沿至输出稳定的区间
```

测 Worst Case 必须改输入转换！ M3 的 8 个静态向量只验证真值表，不能直接证明遍历到最坏路径。P22 Optional：8 个向量中选不同的前后组合，共 **8×7=56** 个有序转换。测 S、COUT 的不同上升/下降延迟，记录最大值。

常见问题就按这顺序排查

Unknown cell/module	library.f 标准单元 Verilog 位置和 -f
Illegal timescale	GF22 模型编译顺序；TB timescale 在模块前；必要时检查原库文件编译设置
SDF annotation 0/failed	SDF 文件、实例 U1、网表版本、log 层次
VPD missing	TB 是否真的执行 $vcdpluson；执行目录是否一致

该编译写法是通用参考；M3 手册 P18 原始 VCS 指令仍然有效。M4 门级网表可能不再保留 dut.state 名称，门级 TB 应优先对照端口输出检查。+neg_tchk 不等于关闭所有 Timing Checks；警告需要分类，不应为让仿真 PASS 而全部屏蔽。

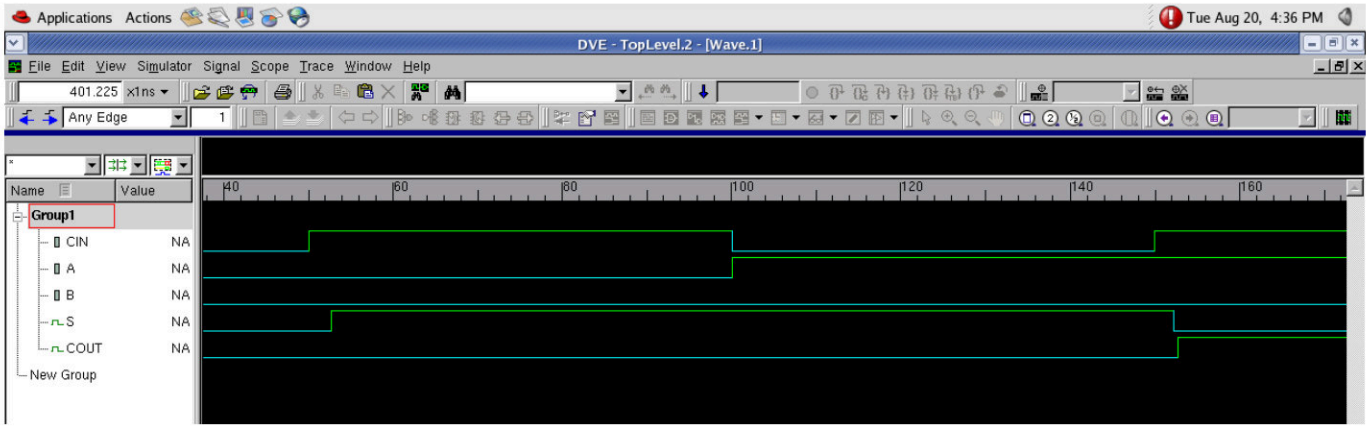
T3 · 门级 VCS / SDF 波形

§2.3.C · 手册 P18–19

① 加载 VCS，编译带标准单元的网表（P18）

```
cd ~/NM6008/Module3/rtl
csh
source synopsys_linux_vP_2019_cshrc
vcs -debug +v2k +neg_tchk ./FullAdder_tb.v ./Syn_FullAdder.v -f ./library.f
./simv -gui
```

② DVE 放大查看门延迟



原手册 Fig. 11 · Post-Synthesis Timing Waveform（P19）

选 **A/B/CIN/S/COUT** → Add To Waves → Start/Continue → 放大输入变化沿，读出 S/COUT 的传播延迟。

③ 三个验收关口

① SDF　日志应显示 SDF 注释成功；留意 Instance/Timing Arc 失败。

② 功能　同一组激励下，输出应与设计功能匹配（允许 SDF 延迟）。

③ 文件　确认 inter.vpd 或实际生成的 VPD 存在、不是空文件。

如果是 SV 文件，可改用 vcs -full64 -sverilog -debug_access+all；不要删除 -f library.f。

T4 • 活动数据与 Power Compiler

§2.4.A–B • 手册 P19–20（第一部分）

① 修改 PowerAnalysis 的 GF22 搜索路径（P19–20）

```
cd ~/NM6008/Module3/rtl/PowerAnalysis
# 打开 common_setup_gf22.tcl
# 查找 ADDITIONAL_SEARCH_PATH1，改成你自己的 Module3/Library 所在路径
ls ../../Library
ls ../../Library/db/*.db
```

> 原手册 P20 用的是另一实验账号的 /home/msc_tum_grp1/.../Module3/Library。必须换成当前账号。若实际 .db 在 Library/db，设置搜索路径时应包含 db 子目录。

② VPD → VCD → SAIF（P20）

```
csh
source ../../Synthesis/synopsys_linux_vR_2020_cshrc
vpd2vcd ../inter.vpd inter.vcd
vcd2saif -i ./inter.vcd -o inter.saif
ls -lh inter.vcd inter.saif
dc_shell-xg-t
```

③ 在 dc_shell> 中读门级网表和 SAIF

```
read_verilog ../Syn_FullAdder.v
current_design FullAdder
link
read_saif -input ./inter.saif -instance FullAdder_tb/U1
report_saif -hierarchy
report_saif -missing
```

> **验收**　库加载成功、link 无关键未解析单元、SAIF 注释覆盖有效，不得出现 0% 活动注释却当成功。

T4 · Power 脚本必改项

Module 3 P19–20：最容易混淆 C Shell、DC Tcl 和 GF22 库相对路径

1 · common_setup_gf22.tcl（Tcl，不可在 csh 中 source）

在脚本中核对：示例为 PowerAnalysis 相对路径

```
set ADDITIONAL_SEARCH_PATH1 "../../Library/db"
set ADDITIONAL_SEARCH_PATH2 "../../Library/sdb"
# ADDITIONAL_SEARCH_PATH3 按老师原有配置保留
# 原工艺库 TARGET_LIBRARY_FILES 不改名
```

2 · 转换数据 → 进入 DC

这些命令在 Linux C Shell 运行

```
cd ~/NM6008/Module3/rtl/PowerAnalysis
csh
source ../../Synthesis/synopsys_linux_vR_2020_cshrc
which vpd2vcd
which vcd2saif
which dc_shell-xg-t
ls -lh ../inter.vpd
vpd2vcd ../inter.vpd inter.vcd
vcd2saif -i inter.vcd -o inter.saif
dc_shell-xg-t
```

3 · 读 netlist + SAIF + 生成四份报告

以下逐条在 dc_shell> 运行

```
read_verilog ../Syn_FullAdder.v
current_design FullAdder
link
# M4 时序电路: create_clock -name CLK -period 100 [get_ports CLK]
# M5 时序电路: 使用本题指定的 clock period
read_saif -input ./inter.saif -instance FullAdder_tb/U1
report_saif -hierarchy
report_saif -missing
report_power > rpt_power.txt
report_power -hier > rpt_power_hier.txt
report_power -hier -nosplit > rpt_power_hier_nosplit.txt
report_power -cell > rpt_power_cell.txt
exit
```

两个易错点：若启动时出现 UID-4（Current design not defined），可能是 .synopsys_dc.setup 过早执行设计相关命令；后续读网表、current_design、link 成功后再判断。SAIF 0% 或 missing objects 不为 0 时，先查实例层次/活动记录，不可直接相信功耗值。

T4 • 报告 / 交卷

§ 2.4.B P20 + § 3 P21

① 生成老师要求的四份 Power Report

上述在 dc_shell> 内运行；原手册用 >> 追加，建议单次重跑用 > 覆盖

```
report_power > rpt_power.txt
report_power -hier > rpt_power_hier.txt
report_power -hier -nosplit > rpt_power_hier_nosplit.txt
report_power -cell > rpt_power_cell.txt
exit
```

② Linux Terminal 检查输出

```
cd ~/NM6008/Module3/rtl/PowerAnalysis
ls -lh rpt_power*.txt
grep -Ei 'Internal Power|Switching Power|Dynamic Power|Leakage Power' rpt_power.txt
```

③ 交卷按 Module 3 P21 四问整理

手册 § 3	直接取证
A.1 真值表	T1 DVE + Check.txt + 输入向量
A.2 Gates 与 S/COUT Delay	T2 report.rpt / Standard Cell Schematic
A.3 Worst-case S/COUT Delay	T3 带 SDF 的 DVE 波形
A.4 Power 比较	T2 report.rpt vs T4 rpt_power.txt，注意活动假设不同

验收 四份报告真实存在；数值和单位明确；记录 GF22 Corner、分析工具及 SAIF 映射。

Module 4 / 5 如何套用

迁移速查与故障定位 • 不是替代各模块 Spec

陌生题只需要在原流程里替换这些位置

需要改的地方	M3 FullAdder	M4 StateMachine	M5 MatrixMultiplier
RTL / Top	FullAdder	StateMachine	MatrixMultiplier
TB 激励	8 输入组合	CLK、NRST、start、rst、状态转换	CLK、复位、Signed 输入序列、OUT_STROBE
综合 Tcl	FullAdder.tcl	StateMachine.tcl	实际 Top.tcl
时钟约束	组合逻辑，不要求内部 CLK	100 ns 周期	按 M5 Spec 和 Tcl
SDF 目标	U1	实际 DUT 实例	实际 DUT 实例
SAIF -instance	FullAdder_tb/U1	实际 TB / DUT	实际 TB / DUT

常见报错：只看修复动作

看到的现象	优先排查
vcs / dc_shell Command not found	进入 csh，并 source 对应版本的 *_cshrc；which vcs
script_path: Undefined variable	误用 Linux source 跑 Tcl；改到 design_vision> 执行
Unknown module / UDP / cell	library.f 的 GF22 标准单元仿真模型路径
Illegal timescale	检查 TB 与网表的 timescale / 编译顺序
SDF annotation failed	文件名 + TB DUT 实例层次 + SDF 对应网表
SAIF 0% annotation	VPD 是否存在、转换成功、-instance 层次是否匹配
Slack -9.35 (VIOLATED)	M4 实例中是 set_max_area 0 的面积违例，非 Setup Slack

完成定义　T1 功能可信 → T2 综合网表和报告 → T3 SDF 时序波形 → T4 有效 SAIF 功耗。四个阶段都检查后再交。

紧急排错 • 第一页

看到报错先复制关键词 → 表中对应一个排查动作 → 重跑本阶段

现象 / 报错	先执行或检查
vcs: Command not found	csh → source 本阶段 vP_2019_cshrc → which vcs
dc_shell: Command not found	csh → source vR_2020_cshrc → which dc_shell-xg-t
source tcl → puts: Command not found	你在 C Shell 执行 Tcl。进入 dc_shell> 才 source
script_path: Undefined variable	同上；run_topdown.tcl 在 Design Vision 执行
Unknown type for port wire	检查关键字是否误拼成 wore、wire 是否标准输入声明
syntax error at token always	检查上一层 begin/end、endmodule、上一行分号和 always 拼写
syntax error at token ;	检查 assign 左值、运算括号、误用另一输出名称
Netlist same old timestamp	检查脚本实际执行；设计是否发生变化；净表输出目录是否正确
Top / current_design not found	read_verilog 文件是否含本题 Top、大小写、模块名
No clocks in this design	若时序题：检查 Top/CLK 端口/create_clock；组合题则可能正常
Slack (VIOLATED)	先确认报告小节是 area 还是 timing，不能只看单行 VIOLATED

可复制的救急命令

Linux Terminal

```
pwd
ls -la
find ~/NM6008 -name 'library.f' -print
find ~/NM6008 -name '*cshrc' -print
grep -RniE 'create_clock|CLOCK_PERIOD|compile' scripts/ 2>/dev/null
ls -lh --full-time netlist/ logs/
```

以上 `grep scripts/` 在 **Synthesis/** 下运行；别在 `dc_shell>` 输入 Linux 的 `grep/cp/ls`。

紧急排错 · 第二页

定位 VCS SDF / VPD / Power Compiler 的后半段问题

现象 / 报错	先检查
gf22 cell module is undefined	library.f 里的 -v 文件存在且包含对应单元
VFS_SDB_ERROR / different VCS version	换新仿真工作目录、旧版 simv.daidir 不要混用；按实验手册重新编译
Illegal timescale for module	检查模型和 TB 时间尺度声明及编译顺序
SDF annotation warning	SDF 文件、实例路径、网表版本及日志未注释 arc 数量
inter.vpd does not exist	TB 加 $vcdplusfile 和 $vcdpluson；当前运行目录；重新执行 simv
vpd2vcd file open failed	ls -lh ../inter.vpd，确定 VPD 的真实存放位置
read_saif returns 1 but 0%	-instance 映射和 VPD 记录范围不对；report_saif -missing
Power report missing library	common_setup_gf22.tcl 的 search_path 指到 db；link 成功
Power 0 / 异常	不要直接写进报告；查 SAIF 覆盖、波形窗口、时钟/输入激励
DVE only X / no toggles	是否实际点击 Start/Continue；波形缩放区间；TB 是否驱动并复位
Shell exit / quit confusing	工具 Tcl 输入 exit；Shell 用 exit 或 Ctrl+D；运行中的 DVE 关 GUI

不想误删：重试优先使用隔离目录

Linux Terminal

```
cd ~/NM6008/Module3
mkdir -p Verification_Evidence
# 原有结果不删；按次保存 log 到单独的文件名
# 不要用 rm -rf 清理未知路径；不要覆盖老师提供库文件
```

判断是否继续： T1 检查通过才做 T2；T2 Netlist/SDF/报告确认新生成才做 T3；T3 SDF 注释与 VPD 通过才做 T4。缺一项不要拿上次旧文件假装完成。

考前最后 5 分钟 · 关卡表

仅用于自查；对应 M3 Manual §2.1 / §2.2 / §2.3 / §2.4

0 · 新题先填写：不填清楚后面会反复出错

项目	写这里（考试在纸上或笔记里填）
TOP / RTL_FILES	_____
TB_MODULE / DUT_INSTANCE	_____
输入向量文件 / 位宽 / Signed	_____
Clock period / Reset polarity	_____
设计目录 / Library 路径	_____

1 · 四阶段签核

☐ T1 · Behavioral (P4–10)	RTL 完成；TB 已自动比较 Expected；8 组合/M4 状态/M5 输出有效；VCS 编译 & 仿真结果正确
☐ T2 · Synthesis (P10–16)	GF22 目标库；正确 Top；Netlist+SDF(+SDC)；report.rpt；Area / FF / slack / timing 分清
☐ T3 · Post-Synthesis (P16–19)	library.f 路径正确；TB timescale；$sdf_annotate 对准实例；Gate PASS；SDF Log 与 VPD
☐ T4 · Power (P19–20)	common_setup_gf22.tcl 路径；VPD→VCD→SAIF；read_saif 实例正确；四份报告、非空 activity

2 · 最终上交证据（Manual P21；M4 P9；M5 P5）

M3 必做	真值表 / 报告中门与延迟 / SDF Worst-Case / 综合与活动功耗比较
M4 必做	门电路 / FF 个数 / Slack 解释；Optional 若做要有独立 area/power & 波形证据
M5 必做	OUT_STROBE 与结果一致 / 结构/综合 / SDF / dynamic+leakage / 简短报告
每个项目都留	脚本文件、原始 log、功耗单位、工艺角、波形截图、执行日期

适用范围 这是 NM6008 手册的前端/综合/门级/功耗 SOP。你上传的 GF22FDX DRC/Pegasus/Calibre 资料属于物理 DRC 领域，不是 Modules 3–5 必做步骤；不要将其加进考试流程，也不要把受许可限制的工艺资料提交到公开仓库。

Nanyang Technological University
School of Electrical and Electronic Engineering

NTU-TUM MSc (IC Design) Programme
Digital IC Design Course: NM6008

Laboratory Manual

Module 3

Verilog Simulations and Synthesis of a Full-Adder

Course Instructors: A/Prof Lim Meng Hiot
A/Prof Gwee Bah Hwee
Dr Cheng Deruo
Dr Chen Juncheng
Sheng Shirui
Yu Guanhua
Lee Yujin
Liu Shukun

12nd Oct – 29st Oct 2026

Section 1: Introduction

This tutorial style laboratory manual document is meant for NTU-TUM Master of Science (MSc) course – Integrated Circuit (IC) Design, NM6008. It is a step-by-step guide for students who are new to the concept of Hardware Description Language (HDL), ultimately to provide students with the necessary understanding to begin designing circuits through HDL modelling. The preparation of the contents of this manual benefits from the contributions of the following individuals:

Assoc Prof Lim Meng Hiot	Assoc Prof Gwee Bah Hwee
Dr Chong Kwen Siong	Mr Farid Ahamed
Mr Ne Kyaw Zwa Lwin	Miss Sheng Shirui

The following sub-sections state the objectives, the design tools/process used and the experimental design information of this module – *Verilog Simulations and Synthesis of a Full-Adder*.

1.1 Objectives:

The main objective of this module is to provide a quick introduction to Verilog as a Hardware Description Language (HDL) language for modelling and simulating digital ICs. The design example used is a combinational 1-bit full-adder. It will serve to familiarize students with the Synopsys IC design tools (and the associated design methodologies). The specific objectives include:

(i) To construct a behavioural (functional) model of the full-adder using Verilog;
(ii) To construct a test-bench for simulating/validating the full-adder;
(iii) To synthesize the model of the full-adder into a netlist using the GF 22nm CMOS cell library;
(iv) To understand the synthesis design flow;
(v) To re-simulate/re-validate the netlist of the full-adder (with timing annotation) using the same test-bench in (ii);
(vi) To perform power analysis on the full-adder.

1.2 Integrated Circuit (IC) Design Tools and Process Technology

Students will be trained to use the following IC design tools:

- Verilog Compiler Simulator (Synopsys)
- Design Vision (Synopsys)
- Power Compiler (Synopsys)

For more details on Synopsys IC design tools, the following documents are suggested for further reading:

__, HDL Compiler™ for Verilog User Guide, Synopsys.
__, VCS™ User Guide, Synopsys.
__, Design Compiler™ User Guide. Synopsys.
__, Design Vision™ for Verilog User Guide, Synopsys.
__, Power Compiler™ User Guide, Synopsys

The process technology used is the GF 22nm CMOS fabrication process.

1.3 Design Information

An adder is one of the most rudimentary circuit components used for arithmetic functions, including addition, subtraction, multiplication, division and etc. A 1-bit full-adder accepts 3 input signals (A, B and C_{in}) and generates two output signals (S and C_{out}), and the truth-table of the full-adder is tabulated in Table I.

Table I: Truth-table for a Full-Adder

Input			Output	
A	B	C_{in}	S	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

The logic equations for the outputs can be formulated as follows:

$$S = A \oplus B \oplus C_{in}$$
$$C_{out} = (A \cdot B) \oplus (C_{in} \cdot (A \oplus B))$$

Section 2: Procedures

2.1 Behaviour Modelling and Simulations

A. Working Environment:

1. **Go** to your home directory on the desktop, and **choose** the folder "**Module3**".

2. In the folder "**Module3**", **choose** the folder "**Behavioral**".

3. In the folder "**Behavioral**", you should be able to see the following files.

   ./FullAdder.v
   ./FullAdder_tb.v
   ./synopsys_linux_vP_2019_cshrc
   ./InputVector.txt
   ./run

   Now, you are ready to start your hands-on.
   [Note: Save all your files in the current folder, i.e. "Behavioural"]

B. Creating a Behavioural Verilog Code

1. **Open** "FullAdder.v" using a Text Editor, i.e. **right-click** the file and **choose** "Text Editor".

2. **Write** the behavioral Verilog code, shown in Fig. 1, for the full-adder circuit.

FullAdder.v

```
1  // ------------------------------------------------------------------------------------------
2  // Version | Programmer                                    | Date       | Remark
3  // ------------------------------------------------------------------------------------------
4  // V1      | Dr Kwen-Siong Chong (kschong@ntu.edu.sg)      | 31/07/2013 | Initial version
5  // ------------------------------------------------------------------------------------------
6  //
7  // The code is a behavioural code for a full adder
8  // The code is for teaching purpose in the NTU-TUM class, NM6008.
9
10 module FullAdder (//inputs
11                   CIN,
12                   A,
13                   B,
14                   //outputs
15                   S, COUT);
16
17 input CIN;
18 input A;
19 input B;
20
21 output S;
22 output COUT;
23
24 assign S    = A ^ B ^ CIN;
25 assign COUT = (A & B) | (CIN & (A ^ B));
26
27 endmodule
28
```

Fig. 1 The Verilog code for the full-adder

3. **Save** the created code.

C. Creating a Test-bench

1. **Open** "FullAdder_tb.v" using Text Editor.

2. **Fill in** Parts 1 to 3, as shown in Fig.2 (a) to (c) respectively, in the behavioralVerilog code for realizing a full-adder circuit:

```
10
11  module FullAdder_tb;
12
13  parameter N = 8;
14  parameter DELAY = 50;
15
16  reg CIN;
17  reg A;
18  reg B;
19
20  wire S;
21  wire COUT;
22
23  //Input Vector
24  reg [2:0] REG_DATA [N-1: 0];
25
26  integer file1;
27  integer i;
28
29  FullAdder U1 (//inputs
30                .CIN(CIN),
31                .A(A),
32                .B(B),
33                //outputs
34                .S(S),
35                .COUT(COUT));
36
```

Fig. 2 (a) Part 1: Basic IO/signal declaration and mapping

```
37  initial
38    begin
39      file1 = $fopen("./Check.txt");
40      if (!file1) $finish;
41
42      $readmemh("./InputVector.txt", REG_DATA);
43    end
44
```

Fig. 2 (b) Part 2: Insert the data vector

```
45  initial
46  begin
47    CIN = 0;
48    A  = 0;
49    B = 0;
50
51    $fdisplay(file1,"No\tB\tA\tCIN\tS\tCOUT");
52
53    for (i = 0; i < N; i = i + 1)
54      begin
55        {B, A, CIN}  = REG_DATA[i];
56
57
58        $fdisplay(file1,"%d\t%d\t%d\t%d\t%d\t%d", i, CIN, A, B, S, COUT);
59        #(DELAY);
60
61      end
62
63    $fclose(file1);
64    $finish;
65  end
66
67  initial
68   begin
69     #(100*DELAY);
70     $finish;
71   end
72
73  endmodule
```

Fig. 2 (c) Part 3: Generate the test vector pattern

3. **Save** the created code.

D. Running Verilog Simulations

1. **Right-click** on the Desktop and **open** a Terminal. The Terminal will probably look like what is shown in Fig. 3.

```
[shirui002@isysrl1linuxgwee1 Behavioral]$ csh
[shirui002@isysrl1linuxgwee1 Behavioral]$ source synopsys_linux_vP_2019_cshrc
[shirui002@isysrl1linuxgwee1 Behavioral]$ vcs -debug +v2k \
?         ./FullAdder_tb.v \
?         ./FullAdder.v \
?
```

Fig. 3 A Terminal opened in Linux

2. In the Terminal, **type in** the following commands.

   >> cd /home/msc_tum_grp1/NM6008/Module3/Behavioral
   >> csh
   >> source synopsys_linux_vP_2019_cshrc
   (Noted: the file path should be changed according to your own.)
   [Note: The second command is to source the license to invoke the Synopsis tools]

3. In the Terminal, **type in** the following command.

   >> vcs -debug +v2k ./FullAdder_tb.v ./FullAdder.v

   [Tip: you can copy and save the command in step 3 into the "run" file so that you do not need to re-type the command in the future]

4. **Check** the Terminal if any error message appears. If yes, PLEASE **re-check** your Verilog codes again and repeat step 3. You must **fix all** error messages! If no error message appears, **type in** the following command to invoke the GUI.

   >> simv -gui

   [Tip: you can copy and save the command in the step 4 into the "run" file so that you do not need to re-type the command in the future]

   The Discovery Visualization Environment (DVE) GUI will be shown; see Fig. 4. You may explore the functions/tabs in the DVE GUI on your own.

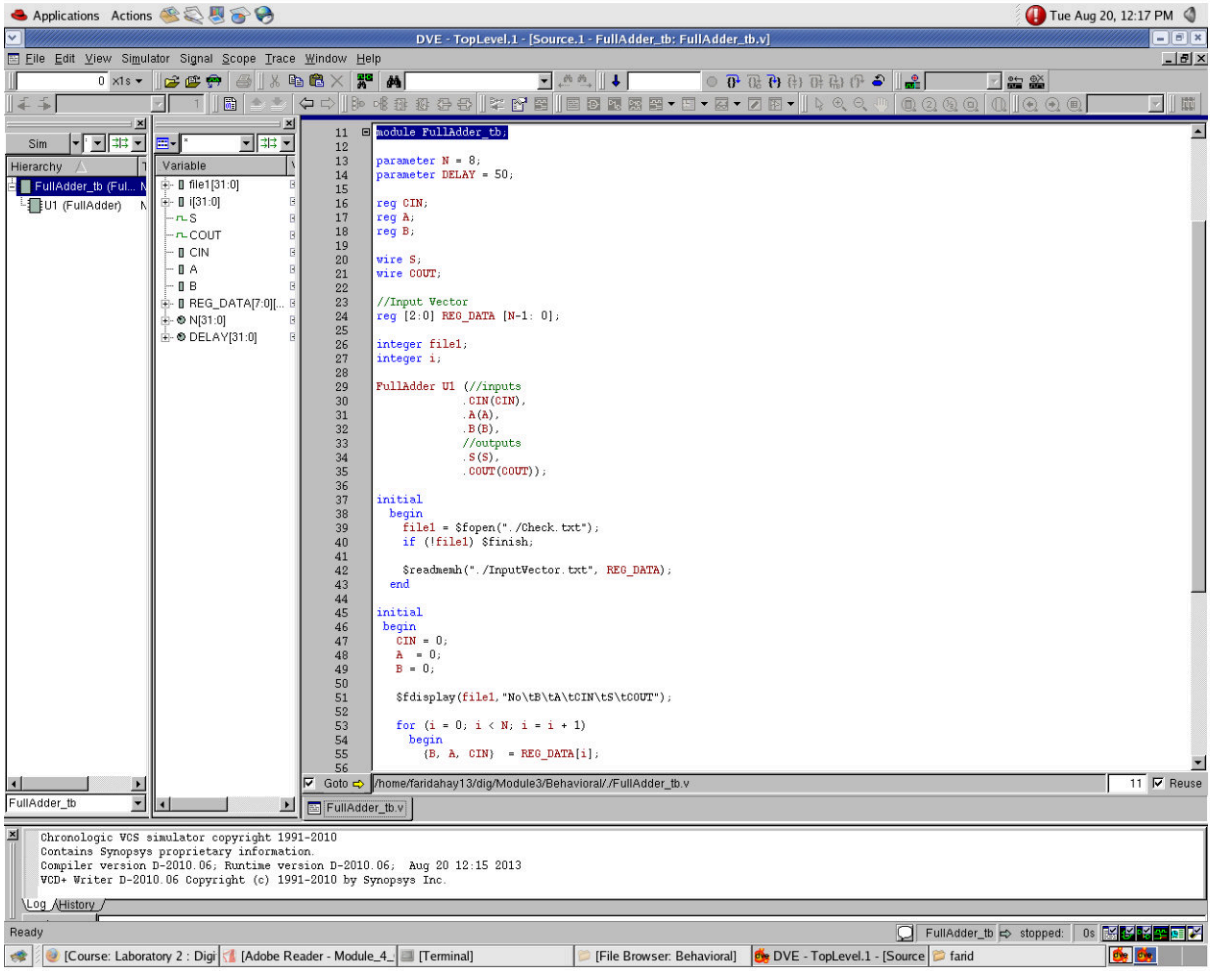


Fig. 4 The Discovery Visualization Environment (DVE) GUI

5. The next step is to **simulate** the full-adder and to **visualize** the input and output waveforms. Before doing so, the file **"inputvector.txt"** has to be written to insert various input test patterns you wish to apply to the full-adder; see Section 2.1.C Part 2.

6. In the generated DVE GUI, **select** all the variables present under the hierarchy of FullAdder_tb; **refer to** the middle column in the DVE GUI in Fig. 4.

7. **Right-click** on the selected variables, and **choose "Add To Waves"** and then **select "New Wave View".** A new DVE waveform GUI will pop up.

8. In the new popped-up DVE waveform window, **choose "simulator"** icon on the toolbar and then **select "Start/Continue".**

9. You can now visualize the input and output waveforms. **Keep** only input and output variables (i.e. A, B, CIN, S, COUT) to get a clearer view of the circuit functionality; **delete** all other variables if any. An example is shown in Fig. 5.

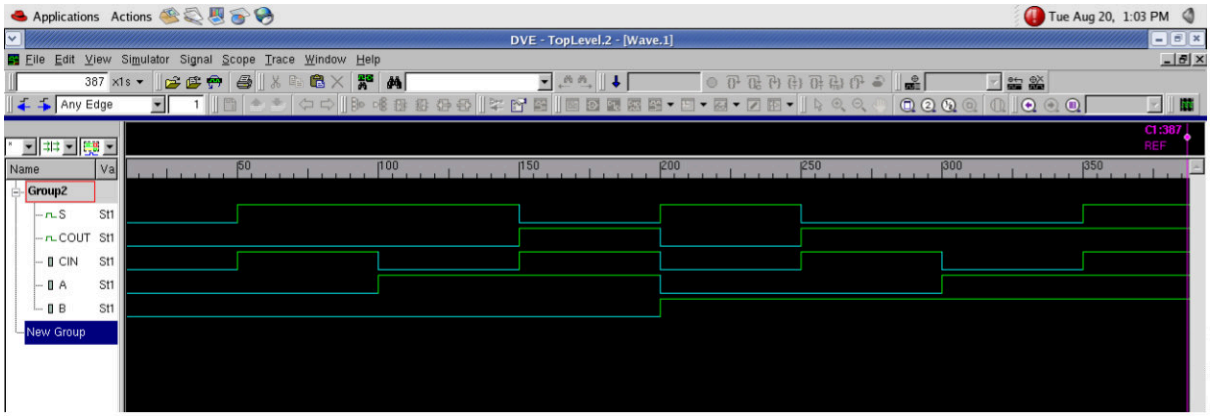


Fig. 5 The DVE waveform window showing the input and output signals

10. **Answer** the question in Section 3.A.1 in page 21.

At this stage, if you have successfully gone through all the steps specified, you have completed the **Behavioral** simulation. Congratulations!

2.2 Synthesis

The synthesis design flow will be carried out based on the Synopsys tool named "**Design Vision"** and on the GF 22nm CMOS process cell library.

A. Working Environment:

1. **Go** to your home directory on the Desktop, and **choose** the folder "**Module3**".

2. In the folder "**Module3**", **choose** the folder "**Synthesis**".

3. In the folder "Synthesis", you should be able to see the following files:

   ./.synopsys_dc.setup [Synopsys configuration file]
   ./ synopsys_linux_vR_2020_cshrc [setup file for the Synopsys tools]
   ./ common_setup_gf22.tcl [setup file for the gf22 process]

4. In the folder "Synthesis", you should be able to see the following folders.

   ./ddc
   ./logs
   ./netlist
   ./scripts
   ./sources

[Note: The arrangement of these folders is the Instructor's preference, and you can choose not to follow this kind of folder/file management if you know the synthesis design flow well]

5. **Copy** the "FullAdder.v" file (that you have designed in Section 2.1) into the folder "sources".

   [Note: In this example, you have only one Verilog file. For multiple Verilog files, you should store all your Verilog files here if you wish to synthesize them]

6. In the folder "scripts", you should be able to see the following files:

   ./FullAdder.tcl
   ./MyDefaults.con
   ./ReadVerilogFiles.tcl
   ./report.tcl
   ./run_topdown.tcl

   Now, you are ready to start your hands-on.
   [Note: Save all your files in the current folder, i.e. "Synthesis"]

B. Understand the Basic Setups/Files

Attention: Refer to the Synopsys Design Vision Manual or DC Manual if you would like to know more. Sometimes, you may need to explore the chosen Process Design Kit (PDK) in order to know/obtain more information. If possible and applicable, please approach a software application engineer and/or a PDK application engineer for help.

1. **Open** "common_setup_gf22.tcl" using Text Editor and **view** it as shown in Fig. 6. The file contains the setups on how the Instructor arranges the folder, and links various GF 22nm CMOS cell libraries. You should be able to obtain the file from any chosen PDK, and if necessary, you could modify the setups/configurations according to your preferred working style. **Close** this file.

```
Info: Running script [info script]\n"

##########################################################################
 Setup Variables
##########################################################################

 following variables, use a blank space to separate multiple entries.
: set TARGET_LIBRARY_FILES "lib1.db lib2.db lib3.db"

IONAL_SEARCH_PATH1        " ../Library/db";#  Additional search path
IONAL_SEARCH_PATH2        "../Library/sdb"        ;#  GF library cells
IONAL_SEARCH_PATH3        "../Library $SYNDIR/libraries/syn ./scripts ./sources ./ddc" ;
onal search path

T_LIBRARY_FILES          "gf22nsdvlogl20hdl116a_TT_0P90V_0P00V_0P00V_0P00V_25C.db" ;#  Target technology logical lib
T_LIBRARY_SYMBOL         "gf22nsdvlogl20hdl116a.sdb"                          ;#  Target technology logical librarie

IONAL_LINK_LIB_FILES     ""  ;#  Extra link logical libraries not included in TARGET_LIBRARY_FILES

IBRARY_FILES             ""  ;#  List of max min library pairs "max1 min1 max2 min2 max3 min3"...

FERENCE_LIB_DIRS         ""  ;#  Milkyway reference libraries (include IC Compiler ILMs here)

FERENCE_CONTROL_FILE     ""  ;#  Reference Control file to define the Milkyway reference libs

FILE                     ""  ;#  Milkyway technology file
ILE                      ""  ;#  Mapping file for TLUplus
JS_MAX_FILE              ""  ;#  Max TLUplus file
JS_MIN_FILE              ""  ;#  Min TLUplus file

OUTING_LAYER            ""   ;# Min routing layer
OUTING_LAYER            ""   ;# Max routing layer
```

Fig. 6 An example of the Synopsys configuration file "common_setup_gf22.tcl"

2. **Open** "./scripts/MyDefaults.con" and view it as shown in Fig. 7. The file contains the basic constraints/specifications – these are deemed as default values set by the instructor. Please refer to the Synopsys DC manual if you would like to understand more. You can always create your constraints/specifications here. **Close** this file.

```
#### ------------------------------------------------------
#### | Define clock or virtual clock for the modules
#### ------------------------------------------------------

#### Define clock variables
set CLK_PERIOD 20
set FIX_DELAY 10
set CLK_SKEW [expr ($CLK_PERIOD * 0.05)]
set INPUT_DELAY [expr ($CLK_PERIOD * 0.15)]
set OUTPUT_DELAY [expr ($CLK_PERIOD * 0.15)]

## Create real clock if clock port is found
if {[sizeof_collection [get_ports CLK]] > 0} {
   set CLK_PORT CLK
   create_clock -period $CLK_PERIOD CLK
}

## Create virtual clock if clock port is not found
if {[sizeof_collection [get_ports CLK]] == 0} {
   set CLK_PORT VCLK
   create_clock -period $CLK_PERIOD VCLK
}

## Apply default timing constraints for modules
set_input_delay $INPUT_DELAY -max -clock $CLK_PORT [remove_from_collection [all_inputs] $CLK_PORT]
set_output_delay $OUTPUT_DELAY -max -clock $CLK_PORT [all_outputs]
set_clock_uncertainty -setup $CLK_SKEW $CLK_PORT
set_clock_uncertainty -hold $CLK_SKEW $CLK_PORT

#### ------------------------------------------------------------
#### | Define Design Environment
#### ------------------------------------------------------------

## Define variables
set DRIVE_CELL HS65_GS_IVX2
set DRIVE_PIN {Z}
set OUTPUT_LOAD [load_of CORE65GPSVT/HS65_GS_IVX2/A]
set MAX_OUTPUT_LOAD [expr ($OUTPUT_LOAD * 4)]
set WIRELOAD_MODEL 10k

set_load $MAX_OUTPUT_LOAD [all_outputs]
set_drive 1.5 [all_inputs]

## If real clock, set infinite drive strength
if {[sizeof_collection [get_ports CLK]] > 0} {
   set_drive 0 CLK
}
```

Fig. 7 An example of the constraint file "MyDefaults.con"

3. **Open** "./scripts/FullAdder.tcl" and view it as shown in Fig. 8. The file contains the additional constraints, if different from the default values, and the synthesis commands.

```
1  # The script is designed for teaching purpose
2  # ------------------------------------------------------------
3  # Version | Programmer                | Date       | Remark
4  # ------------------------------------------------------------
5  # V1      | Dr Kwen-Siong Chong       | 31/07/2013 | Initial version
6  # ------------------------------------------------------------
7
8  # Script file for constraining the design
9  set rpt_file "report.rpt"
10 set design "FullAdder"
11
12 current_design FullAdder
13 source "${script_path}/MyDefaults.con"
14
15 # Define design environment
16 set_load $MAX_OUTPUT_LOAD [all_outputs]
17 set_driving_cell -lib_cell $DRIVE_CELL [all_inputs]
18
19 # Define design constraints
20 #set_input_delay  $INPUT_DELAY -clock $CLK_PORT [all_inputs]
21 #set_output_delay $OUTPUT_DELAY -clock $CLK_PORT [all_outputs]
22 set_max_area 0
23
24 # Ungroup DesignWare parts
25 set designware_cells [get_cells -filter "@is_oper==true" *]
26 if {[sizeof_collection $designware_cells] > 0} {
27    set_ungroup $designware_cells true
28 }
29
30 #set_fix_hold [all_clocks]
31 compile
32 write -format verilog -output ${netlist_path}/Syn_$design.v -hier
33 write -format ddc -hierarchy -output "${ddc_path}/${design}.ddc"
34 write_sdf ${netlist_path}/Syn_$design.sdf
35 write_sdc ${netlist_path}/Syn_$design.sdc
36 source "${script_path}/report.tcl"
```

Fig. 8 An example of the constraint file "FullAdder.tcl"

The following are some remarks.

(i) In line 10, please make use the name "FullAdder" is the **SAME** as the name of the top module in your Behavioral Verilog file. **The same principle shall apply to other designs in Modules 4 and 5.**

(ii) In line 12, please make use the name "FullAdder" is the **SAME** as the name of the top module in your Behavioral Verilog file. **The same principle shall apply to other designs in Modules 4 and 5.**

(iii) In lines 30 to 35, the scripts are used to compile the design and generate the respective output files.

Close the file.

4. **Open** "./scripts/ReadVerilogFiles.tcl". There is only one command, i.e. to link the Verilog file, as show below. **Close** the file.

   read_verilog ./sources/FullAdder.v

   [Note: For other projects which may have many Verilog files, please link all Verilog files in "ReadVerilog.tcl".]

5. **Open** "./scripts/run_topdown.tcl". There are only two commands as shown below**.** The commands link all the required scripts together. **Close** the file.

   source "${script_path}/ReadVerilogFiles.tcl"
   source "${script_path}/FullAdder.tcl"

   [Note: For other projects, please make the **NAMES** of the file/folder/module consistent to avoid any possible errors due to missing files/modules]

C. Using the Synopsys Tool – Design Vision

1. **Open** a Terminal. **Type in** the following commands in the Terminal to gain access to the Synopsys Design Vision synthesis tool and GF 22nm process library.

   >> cd /home/msc_tum_grp1/NM6008/Module3/Synthesis
   >> csh
   >> source synopsys_linux_vR_2020_cshrc
   >> design_vision &

   The GUI is shown as below in Fig. 9.

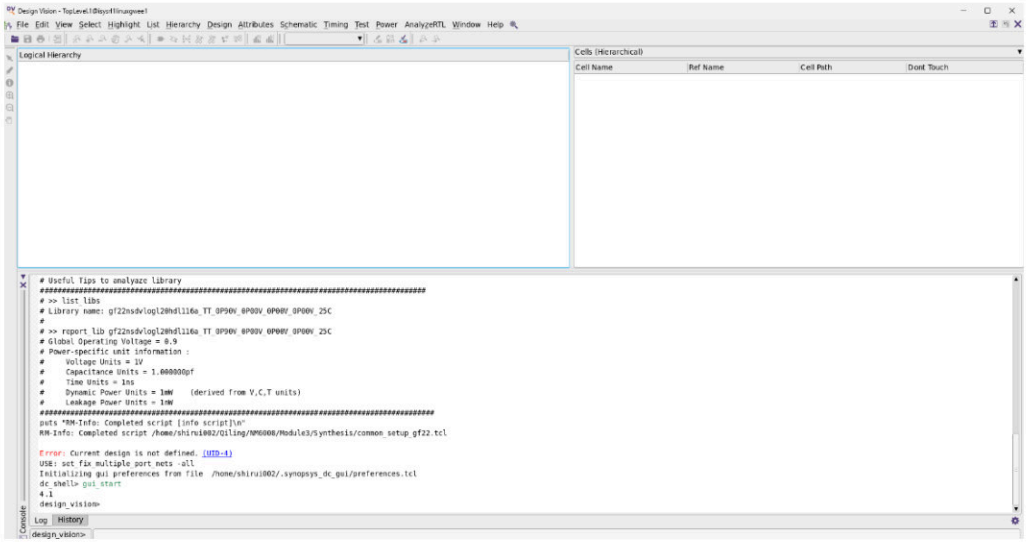


Fig. 9 the GUI for Design Vision

2. **Select "File"** on the toolbar and then **choose "Execute Script".**

3. **Choose** the folder **"scripts"** and then **select** the file **"run_topdown.tcl"**

   [Note 1: For combinational logic synthesis, you will see many error messages. It is probably OK if these error messages relate to a clock signal. If not, please check your setup files and/or Verilog files to make sure that they are error-free.]

   [Note 2: Instead of writing scripts, you could also use the GUI to insert all the commands in order to synthesize your design. However, it is quite common that you might need to re-synthesize your design many times (until it is error-free or you are happy with the result). Hence, writing scripts is still preferred.].

4. **Right-click** on the icon named FullAdder shown under the "**Logical Hierarchy**" tab and then **selects "Schematic View"**. The gate level netlist of the full-adder is obtained and thus you can visualise the structural design of the developed full-adder as shown in Fig. 10.

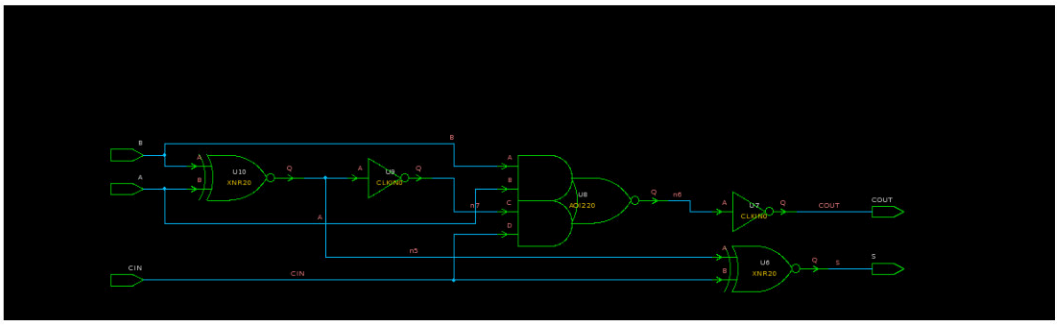


Fig. 10 A viewer to display the full-adder

5. After the synthesis, you should be able to view two generated files in the "**netlist**" folder.

./netlist/Syn_FullAdder.v
./netlist/Syn_FullAdder.sdf
./netlist/Syn_FullAdder.sdc

6. After the synthesis, you should be able to view a generated report in the "**logs**" folder. Take a look what the report tells you about your design.

   ./logs/report.rpt

7. **Answer** the question in Section 3.A.2 in page 21.
8. Type "**Exit**" when you finish the whole process (Important!)

At this stage if you have successfully gone through all the steps, you would have completed the **Synthesis**. Congratulations!

2.3 Post-Synthesis Simulations

The synthesized Verilog code has to be re-simulated to account for the delay introduced by the gates based on the GF 22nm CMOS cell library.

A. <u>Working Environment:</u>

1. **Go** to your home directory on the Desktop, and **choose** the folder "**Module3**".

2. In the folder "**Module3**", **choose** the folder "**RTL**".

3. In the folder "**RTL**", you should be able to see the following files.

   ./synopsys_linux_vP_2019_cshrc
   ./source files
   ./library.f
   ./run

4. **Copy** the synthesized netlist (i.e. Syn_FullAdder.v) , the standard delay format file (i.e. Syn_FullAdder.sdf) and Synopsys Design Constraints (Syn_FullAdder.sdc) in the folder "**RTL**". Refer back to Section 2.2.

5. **Copy** the test-bench file (i.e. FullAdder_tb.v) and the input vector file (i.e. InputVector,txt) into to "**RTL**" folder. Refer back to Section 2.1.

   Now, you are ready to start your hands-on.

[Note: Save all your files in the current folder, i.e. "**RTL**"]

B. Modifying a Behavioural Verilog Code for Creating a Test-bench

1. **Open** "FullAdder_tb.v" using Text Editor. **Insert** the following commands as follows:

   As shown in line 9, note the "'timescale 1ns / 1ps" in order to make time format compatible with the GF 22nm CMOS cell library.

```
 8
 9  `timescale 1ns / 1ps
10
11  module FullAdder_tb;
12
```

   As shown in line 30, "initial $sdf_annotate("Syn_FullAdder.sdf", U1);" to annotate the sdf file.

```
27  integer i;
28
29  //Backannotation //Added
30    initial $sdf_annotate("Syn_FullAdder.sdf", U1);
31
32  FullAdder U1 (//inputs
```

   Take note of line 53, "$vcdpluson(U1);" and in line 67, "$vcdplusoff(U1);" to enable the features of recording all the switching profile.

```
52    B = 0;
53    $vcdpluson(U1);
54
55    $fdisplay(file1,"No\tB\tA\tCIN\tS\tCOUT");
56
57    for (i = 0; i < N; i = i + 1)
58      begin
59        {B, A, CIN}  = REG_DATA[i];
60
61        #(DELAY);
62        $fdisplay(file1,"%d\t%d\t%d\t%d\t%d\t%d", i, CIN, A, B, S, COUT);
63
64      end
65
66    $fclose(file1);
67    $vcdplusoff(U1);
68    $finish;
```

2. **Save** the file.

C. Running Verilog Simulations

1. **Right-click** on Desktop and **open** a Terminal.

2. In the Terminal, **type in** the following commands:

   >> cd /home/msc_tum_grp1/NM6008/Module3/rtl
   >> csh
   >> source synopsys_linux_vP_2019_cshrc

3. In the Terminal, **type in** the following command:

   >> vcs -debug +v2k +neg_tchk ./FullAdder_tb.v ./Syn_FullAdder.v –f ./library.f

4. **Check** the Terminal to see if any error message appears. If yes, PLEASE **re-check** your Verilog codes again and repeat the step 3. You must **fix all** errors indicated by the error messages! If no error message appears, **type in** the following command to invoke the GUI.

   >> simv -gui

   [Tip: as mentioned earlier, you can copy and save the command in the "run" file so that you can re-run the command as many times as possible]

5. In the generated DVE GUI, **select** the relevant the variables present under the hierarchy of FullAdder_tb.

6. **Right-click** on the selected variables and **choose "Add To Waves"** and then **select "New Wave View".** A new DVE waveform GUI will pop up.

7. In the new popped-up DVE waveform window, **choose "simulator"** icon on the toolbar and then **select "Start/Continue".**

8. You can now visualize the input and output waveforms.

9. If you **zoom into** the waveforms you can clearly identify the delay in the generation of output(s) after every input transition. An example is shown in Fig. 11.

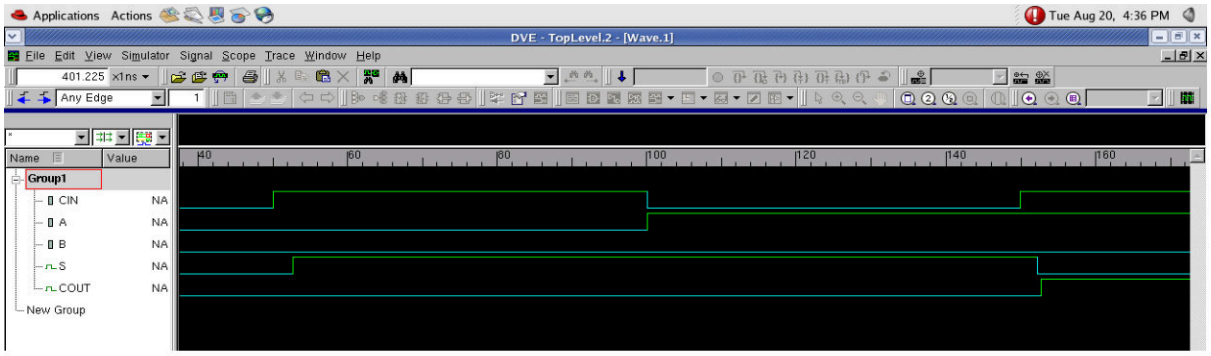


Fig. 11 Waveforms with timing information

10. **Answer** the question in Section 3.A.3 in page 21.

At this stage if you have successfully gone through all the steps, you would have completed the **RTL** simulation. Congratulations!

2.4 Power Simulations

The power simulations are based on the switching profile generated in Section 2.3.

A. <u>Working Environment:</u>

1. **Go** to your home directory on the Desktop, and **choose** the folder "**Module3**".

2. In the folder "**Module3**", **choose** "**RTL**".

3. In the folder "**RTL**", **choose** the folder "**PowerAnalysis**".

4. In the folder "**PowerAnalysis**", you should be able to see the following files:

   ./.synopsys_dc.setup
   ./run

Now, you are ready to start your hands-on.
[Note: Save all your files in the current directory, i.e. "**PowerAnalysis**"]

B. <u>Simulating the Power Dissipation:</u>

1. **Right-click** on Desktop and **open** a Terminal.

2. In the folder /Module3/rtl/PowerAnalysis/, modify common_setup_gf22.tcl

In the line of set ADDITIONAL_SEARCH_PATH1 to "home/msc_tum_grp1/NM6008/Module3/Library"
(Modify this path as your own case)

3. T**ype in** the following commands:

   >> cd /home/[your home directory name]/Module3/rtl/PowerAnalysis
   >> source synosys_linux_vR_2020_cshrc
   >> vpd2vcd ../inter.vpd inter.vcd
   >> vcd2saif -i ./inter.vcd -o inter.saif
   >> dc_shell-xg-t

   [Tip: you can save these commands into the "run" file].

4. **Type in** the following commands:

   >> read_verilog ../Syn_FullAdder.v
   >> current_design FullAdder
   >> link
   >> read_saif -input ./inter.saif -instance FullAdder_tb/U1
   >> report_power >> "rpt_power.txt"
   >> report_power -hier >> "rpt_power_hier.txt"
   >> report_power -hier -nosplit >> "rpt_power_hier_nosplit.txt"
   >> report_power -cell >> "rpt_power_cell.txt"

   [Tip: you can save these commands into the "run" file. For other project, please take note the file/module **NAMES**].

5. Four different reports namely "rpt_power.txt", "rpt_power_cell.txt", "rpt_power_hier.txt", "rpt_power_hier_nosplit.txt" will be generated upon executing the power simulation. Examine these reports, and answer the question in Section 3.A.4 in page 21.

At this stage if you have successfully follow through all the steps, you would have completed the **Power** simulation. Congratulations!

Section 3: Assignment Questions

A. Basic Questions

1. In Section 2.1, fill in the truth table based on your simulated waveforms.

Time	Inputs			Output	
	CIN	A	B	COUT	S

2. In Section 2.2, refer to the file "report.rpt" and fill in the following:

   What are the gates used for realizing the full-adder?

   _____
   _____
   _____

   The COUT delay of the full-adder

   _____

   The S delay of the full-adder

   _____

3. In Section 2.3, based on the given simulated waveform, what are the worst-case delays for the COUT and S signals?

   _____
   _____

4. Compare the power dissipation generated by the "report.rpt" in Section 2.2 and by the reports in Section 2.4. Comment on these results.

   _____
   _____
   _____
   _____
   _____

B. Optional Questions

1. Write a Test-bench Verilog Code to generate all the different possible input pattern transitions?

   [Hint: There are 56 patterns in total]

2. Carrying on from B1, find out the critical delay(s) for the full-adder. What are the possible value(s) of inputs (both the present inputs to the next inputs) that cause the critical delay(s)?

Nanyang Technological University
School of Electrical and Electronic Engineering

NTU-TUM MSc (IC Design) Programme
Digital IC Design Course: NM6008

Laboratory Manual

Module 4

Verilog State Machine

Course Instructors: A/Prof Lim Meng Hiot
A/Prof Gwee Bah Hwee
Dr Cheng Deruo
Dr Chen Juncheng
Sheng Shirui
Yu Guanhua
Lee Yujin
Liu Shukun

12nd Oct – 29st Oct 2026

Section 1: Introduction

This tutorial style laboratory manual document is meant for NTU-TUM Master of Science (MSc) course – Integrated Circuit (IC) Design, NM6008. It is a step-by-step hands-on exercise to familiarize students with HDL state machine implementation.

The following sub-sections state the objectives and the experimental design information of this module – *Verilog State Machine*.

[Important Note: It is a pre-requisite that you have gone through the tutorial in Module 3. You are expected to refer to the Module 3 laboratory manual document for relevant information, if necessary.]

1.1 Objectives:

Module 4 mainly pertains to design and implementation of a state machine. The specific objectives include:

(i) To reinforce students' understanding and familiarization with IC design methodologies by repeating all the design flows/steps in Module 3;

(ii) To understand the sequential logic and the synchronous-logic design methodologies;

(iii) To construct state machine.

1.2 Design Information

1.2.1 General Background

Combinational logic, e.g. the full-adder, which is mentioned in the Module 3, is a type of logic circuit whose output is solely a function of its inputs. Conversely, sequential logic is a type of logic circuit whose output depends not only on its present inputs but also on the past history of its input(s)/output(s). Put simply, sequential logic has a 'memory' function embedded in it. In reality, almost all circuits and systems in practical digital devices are a mixture of combinational logic and sequential logic.

The most basic sequential logic components are latches and flip-flops. Fig. 1 (a) shows a latch example with its corresponding waveforms. Particularly, the latch is a 'logic l' sensitive latch such that when the *CLK* signal is '1' (see the shaded parts), the input *D* is transferred over the output *Q*. When the *CLK* signal is '0', the output *Q* is latched, and hence remains unchanged. Fig. 1 (b) shows a flip-flop example with its

corresponding waveforms. Particularly, the flip-flop is a 'positive-edge' sensitive flip-flop where when the *CLK* signal is triggered from logic '0' to '1' (see the short shaded transition parts), the input *D* is transferred over the output *Q*. Otherwise, the output *Q* is latched and hence remains unchanged. In practice, flip-flops are preferable to latches due to their edge-sensitive attribute so that the former will have lesser timing issues. Hence, it is not surprising that some reference books even recommend **NOT** adopting latches in your design(s) if possible.

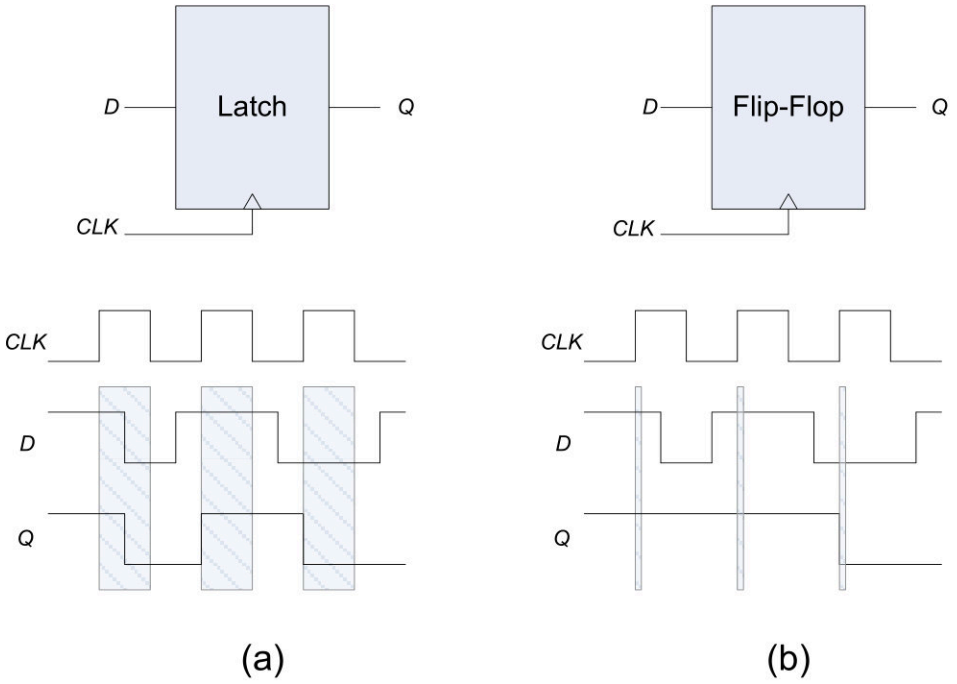


Fig. 1 Example: (a) a Latch with its waveforms, (b) a flip-flop with its waveforms.

The *CLK* signal in the Fig. 1 is essentially a (global) clock signal to synchronize digital circuits, i.e. the transfer of data is done within a well-synchronized timing. Such design methodology is called the synchronous-logic methodology, and is the most prevalent methodology adopted by the circuits and systems community. The main advantage of adopting the synchronous-logic methodology is to mitigate timing issues (and hence the associated design complexity). A good reference book to understand the synchronous-logic methodology is "Digital Integrated Circuits, A Design Perspective" authored by Jan. M. Rabaey *et al.* Design manuals of the Electronic Design Automation (EDA) vendors (e.g. Synopsys, Cadence, Mentor Graphics, Xilinx, Altera, etc.) are also good reading materials if students wish to learn more in the methodology.

In this module, students will learn to design a simple sequential logic design – a state machine.

1.2.2 Design Example – A State Machine

Fig. 2 shows the primary inputs and outputs of the State Machine; for consistency, please use "StateMachine" as the top module name. Consider first the 7 inputs, namely *CLK*, *NRST*, *start*, *rst*, *CIN*, *A*, and *B*, and their associated function(s). The *CLK* signal triggers the state machine to operate state by state, and the *NRST* signal is an asynchronous signal to reset all the flip-flops embodied in the State Machine. The *CIN*, *A*, and *B* signals are the input signals of a full-adder embedded in it. The *start* and *rst* signals are two further control signals for the State Machine. Fig. 3 further shows the state transition diagram. There are four states: S0, S1 S2 and S3. S0 is the default state. When the *start* signal is enabled in S0, the State Machine will jump to S1, or otherwise remains in the same state. In S1, when the *rst* signal is not enabled, the State Machine will jump to S2, or otherwise will jump to S0. In S2, when the *rst* signal is not enabled, the State Machine will jump to S3, or otherwise will jump to S0. In S3, when the *rst* signal is not enabled, the State Machine will jump to S1, or otherwise will jump to S0.

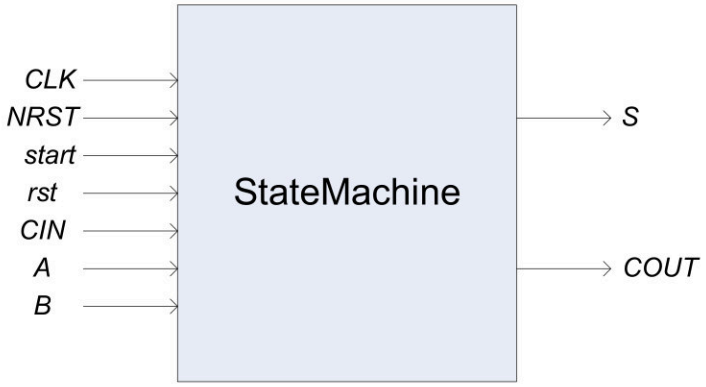


Fig. 2 The primary inputs and outputs of the State Machine.

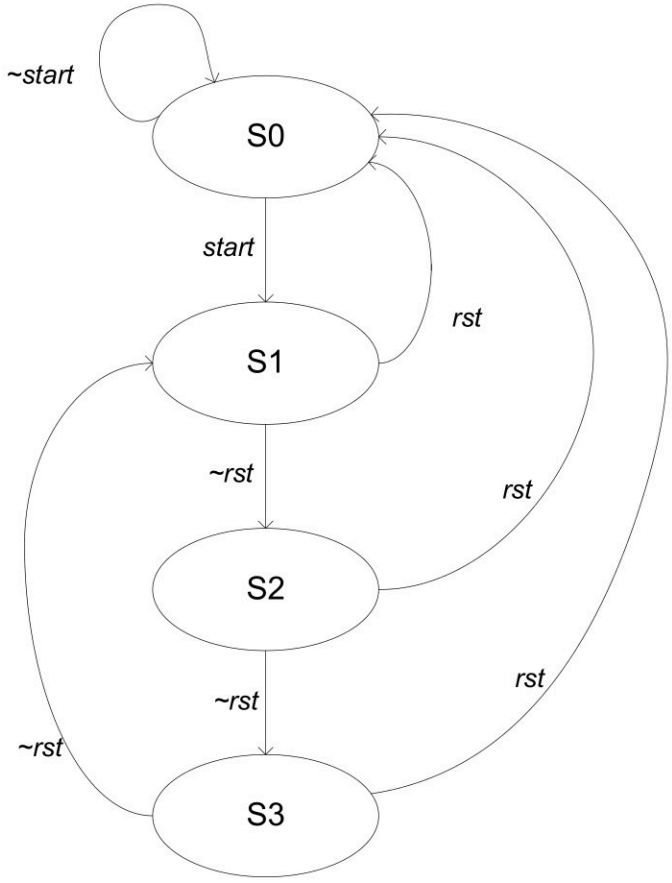


Fig. 3 The state transition diagram

Consider now the 2 outputs, namely *S* and *COUT*. In this exercise, the outputs are required to be realized as combinational logic. In both S1 and S3 stage, the *S* output will function as a Sum signal according to the three input signals, *A*, *B*, and *CIN*. In both S2 and S3 stage, the *COUT* signal will function as a Carry signal according to the three input signals, *A*, *B*, and *CIN*. Otherwise, both the *S* and *COUT* signals are '0'. Refer to the Module 3 on how the *S* and *COUT* signals can be generated.

Section 2: Procedures/Tasks

2.1 Behaviour Modelling and Simulations

A. Working Environment:

1. Create your working environment; please refer to the Module 3.

B. Creating a Behavioural Verilog Code for Implementation of State Machine

1. Create a behaviour code to design the State Machine mentioned in Section 1.2.2. You can follow the tips presented in steps 2 to 5.

2. Tip 1: Define your state parameters

```
module StateMachine (//define your inputs and outputs);

parameter S0 = 2'b00;
parameter S1 = 2'b01;
parameter S2 = 2'b10;
parameter S3 = 2'b11;

//define your inputs and outputs

reg [1:0] state, next_state:
```

3. Tip 2: Synchronous-logic CLK and asynchronous-logic NRST

```
// Sequential Logic for the current state logic
always @ (posedge CLK or negedge NRST)
  if (!NRST)
     begin
       state <= #1 //What is your default state?
     end
     else
     begin
       state <= #1 next_state;
     end
```

4. Tip 3: Combinational Outputs

```
assign S    = //Construct this output by yourself;
assign COUT = //Construct this output by yourself;
```

5. Tip 4: State Machine Coding

```
always @ (state or start or rst)
  case (state)
    S0 : begin
          if (start)
            next_state = S1;
          else
            next_state = S0;
         end
    S1 : begin
          if (rst)
             next_state = S0;
          else
             next_state = S2;
//continue your coding …
```

C. Creating a Behavioural Verilog Code for Creating a Test-bench

1. Create a test-bench to test the State Machine. You can follow the tips below in order to generate a periodical clock signal.

```
parameter CLK_T = 50;

…

 //Clock Generator
initial
  begin
    CLK = 0;
    forever #CLK_T CLK = !CLK;
  end
```

D. Running Verilog Simulations

1. Run your simulations -refer to Module 3.

2.2 Synthesis

A. Working Environment:

1. Create your working environment - refer to Module 3.

B. Modify the Basic Setups/Files

1. Modify the names of the basic setups/files - refer to Module 3.

2. Make sure the clock frequency is 100ns.

C. Using the Synopsys Tool – Design Vision

1. Perform synthesis - refer to Module 3**.**

2. Answer the questions in Section 3.1.

2.3 Post-Synthesis Simulations

A. Working Environment:

1. Create your working environment refer to Module 3.

B. Modifying a Behavioural Verilog Code for Creating a Test-bench to Test the State Machine

1. Modify your test-bench - refer to Module 3.

C. Running Verilog Simulations

1. Run your simulations - refer to Module 3.

2.4 Power Simulations

A. Working Environment:

1. Create your working environment - refer to Module 3.

B. Simulating the Power Dissipation:

1. Run your simulations - refer to Module 3.

Section 3: Assignment Questions

1. Basic Questions

1. Refer to the file "report.rpt" generated in Section 2.2 and answer the following questions:

   What are the gates used for realizing the State Machine?

   _____
   _____
   _____
   _____
   _____

   How many registers were used?

   _____
   _____
   _____
   _____

   Check the timing information, and explain what "Slack" means.

   _____
   _____
   _____
   _____

2. Optional Questions

Since the *S* and *COUT* signals are outputs of a combinational logic circuit, you may observe glitches in these signals. Suppose you are asked to use flip-flops to eliminate these glitches. Design your new circuits, and verify them. What are the additional hardware and power costs compared to the original design?

Nanyang Technological University
School of Electrical and Electronic Engineering

NTU-TUM MSc (IC Design) Programme
Digital IC Design Course: NM6008

Laboratory Manual

Module 5

Verilog Matrix Multiplier

Course Instructors: **A/Prof Lim Meng Hiot**
A/Prof Gwee Bah Hwee
Dr Cheng Deruo
Dr Chen Juncheng
Sheng Shirui
Yu Guanhua
Lee Yujin
Liu Shukun

12nd Oct – 29st Oct 2026

Section 1: Introduction

This tutorial style laboratory manual document is meant for NTU-TUM Master of Science (MSc) course – Integrated Circuit (IC) Design, NM6008. It builds upon the foundation and understanding acquired from going through the hands-on of earlier modules (Modules 3 and 4) by engaging students in the design and implementation of a practical digital system. The preparation of the contents of this manual benefits from the contributions of the following individuals:

Assoc Prof Lim Meng Hiot
Assoc Prof Gwee Bah Hwee
Dr Chong Kwen Siong
Mr Farid Ahamed
Mr Ne Kyaw Zwa Lwin
Ms Sheng Shirui

The following sub-sections state the objectives, the design tools/process used and the experimental design information of this module – *Verilog Matrix Multiplier*.

[Important Note: It is a pre-requisite that you have gone through the tutorials in Module 3 and Module 4. You are expected to refer to the laboratory manual documents of the Modules 3 and 4 for relevant information, if necessary.]

1.1 Objectives:

Module 5 pertains to design and implementation of a small system consisting of a state machine and two functional blocks. The specific objectives include:

(i) To reinforce students' understanding and familiarization with IC design methodologies by repeating all the design flows/steps emphasized in the Modules 3 and 4;

(ii) To apply their knowledge in the implementation of a digital system – hence any system (small or large) could be built in a similar manner.

1.2 Design Information

1.2.1 General Information – Matrix Multiplication

One of the most common mathematical operations in digital signal processing algorithms is matrix multiplication. Typical applications requiring matrix multiplication include Fast-Fourier Transform (FFT), image processing, etc.

Consider a simple matrix multiplication as follows. A 1×2 vector is multiplied by a 2×1 vector, the result of the multiplication is achieved as below:

$$[A1 \quad A0] \times \begin{bmatrix} B1 \\ B0 \end{bmatrix} = A1 \cdot B1 + A0 \cdot B0$$

Students are expected to realize a hardware circuit to perform such matrix multiplication.

1.2.2 Design Example – A Matrix Multiplier

Fig. 2 shows the primary inputs and outputs of the Matrix Multiplier; please use "MatrixMultiplier" as he top module name. Consider first the 5 inputs, namely *CLK*, *NRST*, *START*, *A*, and *B*, and their associated function(s). The *CLK* signal triggers the Matrix Multiplier to perform its operation (see more details later), and the *NRST* signal is an asynchronous input signal to reset the flip-flops. The *START* signal enables the matrix multiplication operation. Once the *START* signal is asserted, the Matrix Multiplier will perform the multiplication operation continuously, and such operation can only be interrupted by resetting the Matrix Multiplier. The *A* and *B* signals are the signed 8-bit input signals. The vectors A1 and A0 share the same A[7:0] signal bus, and hence the A1 and A0 signals are transferred sequentially. The same principle applies to the B[7:0] signal bus.

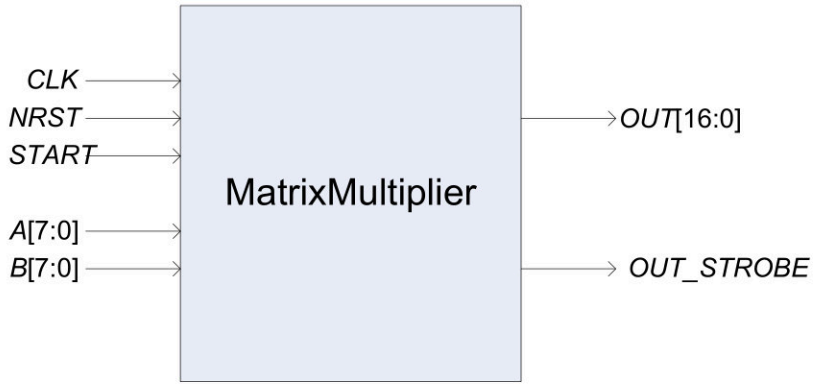


Fig. 2 The primary inputs and outputs of the Matrix Multiplier

Consider now the 2 outputs, namely *OUT* and *OUT_STROBE*. The *OUT* signal bus has 17 bits, generating the outcome of the matrix multiplication. The *OUT_STROBE* signal will be asserted '1' to indicate that the *OUT* signal is now valid/ready. Conversely, the *OUT_STROBE* signal remains '0', indicating that the *OUT* signal is not valid/ready.

Fig. 3 further shows a possible architecture of the Matrix Multiplier. There are three building blocks, the Multiplier, the Accumulator and the State Machine. The State Machine generates the three control signals (*EN*1, *EN*2 and *EN*3) to enable the data transfer between/within the Multiplier and the Accumulator.

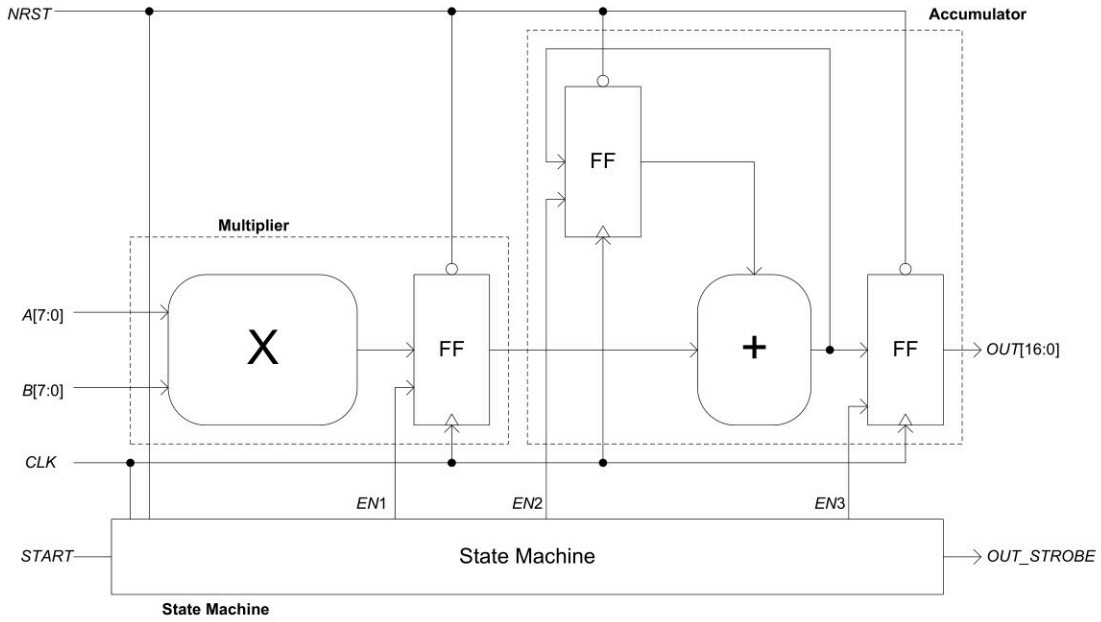


Fig. 3 A possible architecture of the Matrix Multiplier.

Students are expected to follow the architecture shown in Fig. 3 to realize the Matrix Multiplier.

1.2.3 Tips

(i) The signed multiplication can be modelled using the following code (where the *P* is output, and *A* and *B* are inputs)

```
assign P = $signed(A) * $signed(B);
```

(ii) For simplicity, all interfaced signals between the three building blocks can be registered (i.e. generated as sequential logic as opposed to combinational logic).

(iii) Signed extension is required for the Accumulator. For example, if the signal *D* is 17 bits and the signal *C* is only 16 bits, an extra bit can be added to *C* to make the final signal to be 17 bits.

```
assign Q = $signed({C[16], C}) + $signed(D);
```

Section 2: Hands-on

Based on the information in Section 1.2, you are required to carry out the following tasks.

1. Create a behavioural Verilog code for a matrix multiplier design and subsequently develop a test bench Verilog code to simulate the design. Ensure that you provide the input test patterns that you wish to be simulated in the file "inputvector.txt".
2. Simulate and observe the output waveforms obtained.
3. Synthesize the behavioural code to obtain a structural design of the matrix multiplier circuit based on the GF 22nm technology libraries.
4. Perform post synthesis simulation(s) to visualize the behaviour of the circuit, taking into consideration the effect of the various gate delays based on the GF 22nm technology libraries.
5. Perform a power simulation to estimate the overall dynamic as well as leakage power consumed by the overall circuit realized.
6. Submit a very (brief) report to the Instructor.

附录：历史完整 Tcl / csh run / 排错脚本

以下是此前 1,817 行原详细版，作为代码查阅存档。脚本尚未在学校所有 Synopsys 环境中完整实测；如与手册原文或本 PDF 前段存在冲突，以手册和前段核查说明为准。

NM6008 Module 3/4/5 → 陌生数字 IC Lab：四阶段超详细手把手 SOP

> **适用场景**：你已经做过 2026 版 NM6008 Module 3 Full-Adder、Module 4 State Machine、Module 5 Matrix Multiplier；现在拿到一个陌生但流程相同的题目，希望在 NTU 实验室 CentOS/Linux 上，从空白题目一步步完成 **RTL → Behavioral VCS → GF22 Synthesis → SDF Gate-Level VCS → VPD/VCD/SAIF → Power Compiler → 报告**。
>
> **阅读方法**：第一次做按 **A（人工安全流程）** 从头逐条执行；在机房完整验证脚本后，才可使用 **B（四个 `csh run` 半自动流程）**。遇到报错转到 **C（故障定位）**。不得把自动化“命令执行成功”等同于电路或测试已签核。
>
> **来源界线**：标注“**手册**”的步骤取自已上传的 2026 版《NM6008 Module 3 - Full-Adder(3).pdf》《NM6008 Module 4 - Verilog State Machine(5).pdf》《NM6008 Module 5 - Verilog Matrix Multiplier(3).pdf》。标注“**增强建议**”“**教学模板**”的代码是为了陌生题目可复用而扩展，**不是老师手册的逐字原文，也未在你学校 Synopsys 机房实际执行过**。遇到差异，以课程手册、老师当天要求以及机器实际存在的文件为准。

---

0. 一页读懂：你的任务到底是什么

```
题目/Spec
   ↓ 画接口、真值表/状态图/运算公式，确认时钟、复位、位宽、输出有效条件
RTL + TB + InputVector.txt
   ↓ T1: Behavioral Simulation (VCS, DVE)
功能 PASS / sim_beh.log / behavioral.vpd
   ↓ T2: Synthesis (DC / Design Vision + GF22 .db)
Syn_TOP.v / Syn_TOP.sdf / report.rpt / （有的脚本还有 .sdc/.ddc）
   ↓ T3: Gate-Level Simulation (VCS + 标准单元 Verilog library.f + SDF)
门级功能 PASS / SDF 注释成功 / inter.vpd
   ↓ T4: Power (vpd2vcd → vcd2saif → DC Power Compiler)
inter.vcd / inter.saif / 4 种功耗报告 / SAIF 覆盖率
   ↓
整理证据、回答 Assignment、提交简洁报告
```

Module	Top（手册指定）	理解与考试重点	手册页面
3	`FullAdder`	1-bit 全加器：组合逻辑、8 个输入组合、传播延迟；基本操作全部在本手册	M3 P4–20，题目 P21
4	`StateMachine`	4 态 FSM + Full-Adder，NRST 异步复位、`rst` 同步状态返回；100 ns Clock Period	M4 P4–8，题目 P9
5	`MatrixMultiplier`	8-bit Signed A/B、Multiplier + Accumulator + FSM，17-bit OUT、OUT_STROBE	M5 P3–5；流程明确回指 M3/M4

成功标准是四次检查，而不是四条命令： T1 自检真的 PASS；T2 Top、标准单元、约束和网表正确；T3 SDF 真实注释、同一功能仍 PASS；T4 SAIF 匹配且功耗报告实际生成。

0.1 明确三个终端提示符（万不可混淆）

```
[msc_tum_grp2@icd3pc19 Synthesis]$   ← Linux shell (bash/csh 等)
design_vision>                       ← Design Vision 内置 Tcl/DC 命令窗口
dc_shell>                           ← Design Compiler / Power Compiler 命令窗口
```

- `csh` / `source synopsys_linux_..._cshrc` 是 **C Shell 命令**，不要在 `dc_shell>` 执行。
- `read_verilog`、`create_clock`、`compile`、`read_saif`、`report_power` 是 **DC Tcl 命令**，不要在普通 `[user@pc]$` 执行。
- `common_setup_gf22.tcl` 是 **Tcl，不要**在 C Shell `source ./common_setup_gf22.tcl`；否则出现 `puts: Command not found`、`set: Variable name must begin with a letter`。
- 手册不同页写 `RTL`、`rtl` 大小写可能不一致；Linux **大小写敏感**，以 `pwd`、`ls` 的真实结果为准。

0.2 Module 3 的三套已验证环境入口（按手册）

```
# T1：Module3/Behavioral
cd ~/NM6008/Module3/Behavioral
csh
source synopsys_linux_vP_2019_cshrc
which vcs
```

```
# T2：另开终端，Module3/Synthesis
cd ~/NM6008/Module3/Synthesis
csh
source synopsys_linux_vR_2020_cshrc
which design_vision

# T3：另开终端，实际目录是 Module3/rtl（优先检查）
cd ~/NM6008/Module3/rtl
csh
source synopsys_linux_vP_2019_cshrc
which vcs

# T4：另开终端，到 PowerAnalysis
cd ~/NM6008/Module3/rtl/PowerAnalysis
csh
source ../../Synthesis/synopsys_linux_vR_2020_cshrc
which dc_shell-xg-t
```

> `which` 返回 `Command not found` 通常表示**环境尚未加载**，不能据此直接认定服务器没有工具/没有 License。**先加载路径，运行软件后才能判断许可证是否可用**。手册 P8、P14、P18、P20 分别介绍这四阶段。

A. 最稳的人工流程（考试现场优先）

A1. 接到新题目：先解析 Spec，不要立刻写代码

纸上或注释中回答下面 12 个问题：

1. 模块 Top 名称是什么？是否有规定（M3 `FullAdder`、M4 `StateMachine`、M5 `MatrixMultiplier`）？
2. 每个 `input/output` 的方向、位宽、signed/unsigned 是什么？
3. 这是纯组合逻辑、时序逻辑、FSM、Datapath，还是它们组合？
4. 有没有 `CLK`？上升沿还是下降沿更新？周期多少？
5. 有没有异步 `NRST`？高/低有效？复位后状态和输出是什么？
6. `start/enable/valid/strobe` 具体控制什么？何时采样？
7. 输入如何排序？同一个总线是否分拍传送（Module 5 是这种情况）？
8. 输出在什么时候有效？固定延迟还是由 valid/strobe 指示？
9. 运算位宽、符号扩展、溢出/截断/饱和规则是什么？
10. 所有非法状态、同时到来的复位/开始、背靠背事务怎么处理？
11. Testbench 的 Expected Result 如何**独立**于 DUT 计算？
12. 实验要求哪些图、哪几种功耗、怎样比较？

Spec 表模板（拿到新题目先填）：

项目	填写内容
TOP	_____
TB module / DUT instance	_____ / _____
Inputs	_____
Outputs	_____
CLK Period	_____ ns / 无时钟
Reset	_____
功能公式 / FSM 转移表	_____
期望输出何时有效	_____
测试点	正常、边界、复位、连续事务、异常
需要的报告图	功能、Cell、Timing、GLS、Power

A2. 原始项目先复制一份，不破坏已完成模块

文件管理器方式（最适合不熟 Linux 的人）：

1. 打开 Home → `NM6008`。
2. 找到完整 `Module5` 或已成功的 `Module3` 文件夹。
3. `Ctrl+C` → 在上一级 `Ctrl+V` → 把复制的目录重命名为 `Test2_Exam`。
4. **保留** `Library/`、`Behavioral/`、`Synthesis/`、`rtl/` 原目录结构和工艺模型路径。
5. 不要将学校 GF22 PDK、`.db/.sdb`、专有 EDA 库上传到公开仓库。

命令行等价方式（`cp -a` 会复制目录、尽量保留时间戳/权限/符号链接等元数据）：

```
cd ~/NM6008
cp -a Module5 Test2_Exam
cd Test2_Exam
pwd
ls -la
```

不要随意使用 `cp -rf` **或** `rm -rf`：`-f` 的强制覆盖/删除机制更容易破坏唯一的工作副本。`cp -a` 不保证绝对安全，因此目标路径存在时也先查看内容。

A3. 用 `pwd` / `ls` / `find` 找准实际文件

```
pwd                             # 当前完整路径
ls                              # 普通文件名
ls -la                          # 包括隐藏文件和权限/所有者/时间戳
ls -lh                          # 人类可读文件大小；默认不显示隐藏文件
ls -lah                         # 最常用：隐藏文件 + 详细信息 + 易读大小
ls -lh --full-time netlist/     # 确认新生成文件的精确时间
```

```
find . -name 'library.f' -print
find . -name '*cshrc' -print
cat rtl/library.f             # 整个文件内容
less Synthesis/logs/report.rpt # 分页阅读，q 退出
```

grep 常用：

```
grep -n 'create_clock' Synthesis/scripts/*.tcl             # 显示行号
grep -RniE 'create_clock|CLOCK_PERIOD|compile' Synthesis/scripts/ # 递归+忽略大小写+正则
grep -inE 'Error:|Warning:|VIOLATED' Synthesis/logs/report.rpt
```

看见什么： grep 匹配的行会带行号/文件名；什么都没打印可能是没匹配到，不一定是命令失败；less 内按 /slack 搜索，n 跳下一个，q 退出。

A4. 固定你的工作目录结构

```
Test2_Exam/
├── Behavioral/
│   ├── <Top>.v                # 你的原始设计
│   ├── <TB>.v 或 .sv          # 原始 TB
│   ├── InputVector.txt        # 若题目用文件驱动
│   ├── synopsys_linux_vP_2019_cshrc
│   └── run
├── Synthesis/
│   ├── .synopsys_dc.setup
│   ├── common_setup_gf22.tcl
│   ├── synopsys_linux_vR_2020_cshrc
│   ├── scripts/{ReadVerilogFiles.tcl,run_topdown.tcl,<Top>.tcl,MyDefaults.con,report.tcl}
│   ├── sources/               # 拷贝 RTL，不放 TB
│   ├── netlist/               # Syn_<Top>.v / Syn_<Top>.sdf
│   ├── logs/                  # report.rpt
│   └── ddc/                   # 如果脚本导出 DDC
├── Library/                   # 学校提供的 GF22 标准单元文件
└── rtl/                       # P16-18 实际目录可能是小写 rtl
    ├── Syn_<Top>.v
    ├── Syn_<Top>.sdf
    ├── <TB>_gate.sv 或手册要求的 TB
    ├── library.f
    ├── synopsys_linux_vP_2019_cshrc
    ├── inter.vpd
    ├── run
    └── PowerAnalysis/
        ├── .synopsys_dc.setup
        ├── common_setup_gf22.tcl
        ├── inter.vcd
        ├── inter.saif
        ├── rpt_power*.txt
        └── run
```

不要机械复制具体文件名：如果已有的 Library 只能留在旧目录，务必检查 library.f、common_setup_gf22.tcl 中的路径是否仍然指向正确工艺文件。

A5. T1 — Behavioral：写 RTL

模块名必须一致。下面三个案例只用于展示“题目怎么变，模板怎么变”，不是让你在新题中照抄原功能。

案例 M3：FullAdder（完整组合逻辑示例）

Behavioral/FullAdder.v：

```
module FullAdder (
    input  wire A, B, CIN,
    output wire S, COUT
);
    assign S    = A ^ B ^ CIN;
    assign COUT = (A & B) | (A & CIN) | (B & CIN);
endmodule
```

要检查：8 种输入组合；S=A xor B xor CIN；COUT=AB+ACIN+BCIN。

案例 M4：StateMachine（参考手册状态图）

Behavioral/StateMachine.v：

```
module StateMachine (
    input  wire CLK, NRST, start, rst, CIN, A, B,
    output wire S, COUT
);
    localparam [1:0] S0=2'b00, S1=2'b01, S2=2'b10, S3=2'b11;
    reg [1:0] state, next_state;
    wire sum   = A ^ B ^ CIN;
    wire carry = (A & B) | (A & CIN) | (B & CIN);

    // 只有 NRST 可以不等时钟边沿立即使 state 复位
    always @(posedge CLK or negedge NRST) begin
        if (!NRST) state <= S0;
        else       state <= next_state;
    end

    // 组合 next_state：输入/状态变化后重新计算，不直接改变 state
    always @(*) begin
        next_state = S0; // 默认分支防 latch
        case (state)
            S0: next_state = start ? S1 : S0;
            S1: next_state = rst   ? S0 : S2;
            S2: next_state = rst   ? S0 : S3;
            S3: next_state = rst   ? S0 : S1;
            default: next_state = S0;
        endcase
    end

    assign S    = ((state == S1) || (state == S3)) && sum;
    assign COUT = ((state == S2) || (state == S3)) && carry;
endmodule
```

关键辨析：

- NRST 低有效**异步**，rst 是在 FSM 转移逻辑中使用的**同步控制输入**，不是同一个 Reset。
- next_state 是组合电路持续求值；state 只有有效时钟边沿更新（异步 NRST 除外）。
- case 并非只能在 always @(*) 中使用，也能用于 always_ff 或函数；**本例用 always @(*) 是因为要计算组合的 next_state**。
- 手册 M4 P6 的 state <= #1 next_state 里 #1 是**仿真时间延迟**，不应理解为会综合成 1 ns 物理延迟。可综合 RTL 推荐不带它；遵循老师原始演示时需记录仿真行为差异。

案例 M5：MatrixMultiplier（功能分解模板，不臆造老师具体 FSM 时序）

老师给定的结构是 Multiplier + Accumulator + State Machine，A/B 都是 signed 8-bit，同一对总线分拍输入，OUT 是 17-bit，OUT_STROBE 指示有效结果。

先建立数学 Reference：

```
logic signed [7:0] a0, a1, b0, b1;
logic signed [15:0] p0, p1;
logic signed [16:0] expected;

p0 = $signed(a0) * $signed(b0);
p1 = $signed(a1) * $signed(b1);
expected = $signed({p0[15], p0}) + $signed({p1[15], p1});
```

不得凭空规定固定“第几拍 OUT_STROBE 有效”，而是先由你实现的 FSM 时序、老师 Fig.3 的寄存器级结构和输出协议决定，再写 Checker。具体 RTL 的乘法/累加寄存器使能必须与手册结构一致。

陌生题目 RTL 改名模板

```
module NEW_TOP (
    input wire CLK, NRST,
    // TODO: 在这里按试卷定义完整输入
    // TODO: 在这里按试卷定义完整输出
);
    // TODO: 组合数据通路：assign 或 always @(*)
    // TODO: 时序寄存器：always @(posedge CLK or negedge NRST)
    // TODO: FSM：状态转移、控制输出、非法状态缺省处理
endmodule
```

A6. T1 — Testbench：编写、编译、运行、保存证据

TB 的 6 个必备部件

1. module 声明和输入激励寄存器（reg/logic）；输出信号用 wire/logic。
2. DUT 实例化并按端口名字连接；记住实例名（例如 U1）。

3. 若 DUT 有时钟，产生对应周期的 CLK。
4. 初始化/复位，随后施加测试向量。
5. 对照 Expected 与 Actual 做判定，出现错误时 $fatal 或累加失败。
6. 输出 TEST PASS 或 TEST FAIL，并生成可查看的波形。

M3 可直接用于练习的自检 TB

Behavioral/FullAdder_tb.sv：

```
`timescale 1ns/1ps
module FullAdder_tb;
    reg A, B, CIN;
    wire S, COUT;
    integer i, checks;
    reg [1:0] expected;

    FullAdder U1 (.A(A), .B(B), .CIN(CIN), .S(S), .COUT(COUT));

    initial begin
        A=0; B=0; CIN=0; checks=0;
        for (i=0; i<8; i=i+1) begin
            {A,B,CIN} = i[2:0];
            #10;
            expected = {1'b0,A} + {1'b0,B} + {1'b0,CIN};
            if ({COUT,S} !== expected)
                $fatal(1,"FAIL inputs=%b actual=%b expected=%b",
                       {A,B,CIN},{COUT,S},expected);
            checks = checks + 1;
        end
        $display("TEST PASS: %0d checks",checks);
        $finish;
    end
endmodule
```

- !== 是 SystemVerilog 四态比较，X/Z 不会意外地被当成 PASS。
- 此 TB 是**比手册更严格的增强建议**：手册原 TB 主要使用 InputVector.txt、$readmemh、$fdisplay 写出测试记录；不要误以为手册要求使用 $fatal。
- 若考试必须使用 InputVector.txt，按题目的向量格式改成 $readmemh，保证读取文件与当前目录一致。

M4 TB 的关键测试清单（不可只看“波形像样”）

```
[ ] NRST=0：不用等时钟边沿，state 进入 S0
[ ] S0 + start=0：保持 S0
[ ] S0 + start=1：下一时钟沿进入 S1
[ ] S1 + rst=0 → S2，S2 + rst=0 → S3，S3 + rst=0 → S1
[ ] S1/S2/S3 + rst=1：下一时钟沿回 S0
[ ] S0: S=0 COUT=0
[ ] S1: S=sum COUT=0
[ ] S2: S=0 COUT=carry
[ ] S3: S=sum COUT=carry
[ ] 同一状态下 A/B/CIN 改变时，组合输出实时更新
```

TB timing 模板：

```
`timescale 1ns/1ps
reg CLK=0;
always #50 CLK=~CLK;  // 一个周期 100 ns

initial begin
    NRST=0; start=0; rst=0;
    A=0; B=0; CIN=0;
    #12 NRST=1;
    @(negedge CLK); start=1;  // 在下降沿施加刺激，避开上升沿竞争
    @(posedge CLK); #2;       // 允许 NBA 和组合输出稳定后检查
    // TODO: 检查 S1 以及由 A/B/CIN 算出的 S/COUT
    @(negedge CLK); start=0;
    // TODO: 继续状态路径、同步 rst、异步 NRST 的检查
end
```

注意：仅靠 dut.state 层次访问内部寄存器，综合后同名信号可能不存在；要做可复用的门级 TB，应主要根据外部端口行为判定，内部信号只作为 Debug 辅助。

M5 TB 的关键测试点

```
[ ] 正/负 Signed 8-bit（-128、-1、0、1、127）
[ ] A0*B0+A1*B1，与 17-bit sign extension 对应
[ ] 同一 A/B 端口分拍传输的次序正确
```

```
[ ] START 启动后 FSM/Datapath 协同工作
[ ] OUT_STROBE=1 时 OUT 必须是期望值
[ ] NRST 中断/清零后的行为
[ ] 连续输入流以及结果更新次序
```

T1 手册操作（不要跳过）

在 Behavioral/：

```
csh
source synopsys_linux_vP_2019_cshrc
```

纯 .v 手册命令（M3 P8）：

```
vcs -debug +v2k ./FullAdder_tb.v ./FullAdder.v
./simv -gui
```

使用上面的 .sv 自检 TB 时，建议：

```
vcs -full64 -sverilog -debug_access+all \
  ./FullAdder_tb.sv ./FullAdder.v -o simv -l compile_beh.log
./simv -l sim_beh.log
```

> 注意：编译生成可执行文件 simv；./simv -l sim_beh.log 执行并记录日志；./simv -gui 执行并启动图形交互。两种执行方式**不是**功能模型或延迟模型的区别，它们的模型由编译时装入的文件/SDF 决定。

如果想让 DVE 读取已保存波形，在 TB 中增加：

```
initial begin
    $vcdplusfile("behavioral.vpd");
    $vcdpluson(0, FullAdder_tb);
end
```

运行后：

```
./simv -l sim_beh.log
dve -vpd behavioral.vpd &
```

T1 截图证据：DVE 同屏保留输入/输出信号（FSM 另加 state），以及包含 TEST PASS 的终端/日志。通过 $display 打印并不等于有自动 Checker；必须阅读 TB 的 if/Reference 代码。

A7. T2 — GF22 Logic Synthesis：逐个文件改什么（最常考）

Step 0：从 T1 拷贝 RTL 至 sources/（不拷 TB）

```
cd ~/NM6008/Test2_Exam/Synthesis
cp ../Behavioral/Adder4.v ./sources/
# 多文件工程例：cp ../Behavioral/FullAdder.v ./sources/
ls -lah ./sources/
```

务必检查拷贝后的文件内容/时间戳，否则会出现 T1 修改了 RTL、T2 却综合旧版本的问题。

Step 1：检查 GF22 common_setup_gf22.tcl

```
cd ~/NM6008/Test2_Exam/Synthesis
less common_setup_gf22.tcl
ls -lah ../Library/db ../Library/sdb
```

关键概念：

- target_library：本次 Technology Mapping 使用的标准单元 .db。
- link_library：解析实例引用时搜索的库，常用 * $TARGET_LIBRARY_FILES $synthetic_library。
- search_path：工具搜索库/设计文件的目录。
- symbol_library：Schematic 符号 .sdb，主要用于 GUI 展示，**不是**功能仿真模型。
- .db：综合/时序/功耗的逻辑库；门级 VCS **不能**直接把 .db 当作 Verilog 模块。

示意（**取自你 M4 GF22 环境的变量命名和结构，文件名必须以实验室 ls 为准**）：

```
set ADDITIONAL_SEARCH_PATH1 "../Library/db"
set ADDITIONAL_SEARCH_PATH2 "../Library/sdb"
set TARGET_LIBRARY_FILES "gf22nsdvlogl20hdl116a_TT_0P90V_0P00V_0P00V_0P00V_25C.db"
set TARGET_LIBRARY_SYMBOL "gf22nsdvlogl20hdl116a.sdb"
```

```
set search_path "$ADDITIONAL_SEARCH_PATH1 $ADDITIONAL_SEARCH_PATH2"
set target_library "$TARGET_LIBRARY_FILES"
set synthetic_library "dw_foundation.sldb"
set link_library "* $TARGET_LIBRARY_FILES $synthetic_library"
```

只做路径修复，不要擅自将 TT 改成 FF/SS Corner。如果复制工程后 `Library` 的位置发生变化，先确定 `pwd`，再计算相对路径。

Step 2：检查 `MyDefaults.con`

手册 M3 P12 指出该文件放默认约束变量；不同模板变量名可能不同，不要凭空假设都有 `CLOCK_PERIOD`。

```
cat scripts/MyDefaults.con
# 观察 CLK_PERIOD/CLOCK_PERIOD、INPUT_DELAY、OUTPUT_DELAY、OUTPUT_LOAD、DRIVE_CELL 等
```

对于 M4：最后必须保证 **实际生效** 的 `create_clock` 周期是 100 ns。M3 是纯组合 Full-Adder，不能强行添加不存在的 CLK。M5 的 Clock Period 按题目要求和老师模板设置，**不默认和 Module 4 相同**。

Step 3：逐行改 `ReadVerilogFiles.tcl`

Module 3 示例：

```
read_verilog ./sources/FullAdder.v
```

Module 4 示例：

```
read_verilog ./sources/StateMachine.v
```

Module 5 示例（文件名为示意；先 `find` 核实）：

```
read_verilog ./sources/Multiplier.v
read_verilog ./sources/Accumulator.v
read_verilog ./sources/StateMachine.v
read_verilog ./sources/MatrixMultiplier.v
```

陌生题目 `Adder4`（层次化、FullAdder 子模块）：

```
read_verilog ./sources/FullAdder.v
read_verilog ./sources/Adder4.v
```

如果使用 SystemVerilog `.sv`，可能需要在当前 DC 版本改用 `analyze -format sverilog` + `elaborate` 或 `read_file -format sverilog`；**先以 Module 3 手册已验证的 `.v` 读法为基准**。`-sverilog` 是 VCS 开关，不是 `read_verilog` 的通用开关。

Step 4：改 `scripts/<Top>.tcl`（务必至少核对 8 项）

用老师已有脚本改，优于从零写一个无法覆盖库初始化的脚本。

示例核心框架（`# TODO` 代表必须人工检查）：

```
# scripts/Adder4.tcl —— 教学示意；建议复制手册现有设计 Tcl 再逐项修改
set design "Adder4"                    ;# [1] 对应 module Adder4
set rpt_file "report.rpt"                ;# [2] 结果名
current_design Adder4                  ;# [3] 对应 Top，必须先 read_verilog
source "${script_path}/MyDefaults.con" ;# [4] 默认约束

# [5] 若有 CLK，按 Spec 创建。M3 无时钟时不能盲目添加。
# create_clock -name CLK -period 100 [get_ports CLK]

# [6] 输入驱动、输出负载、I/O Delay、面积要求，根据题目/模板指定
# set_max_area 0  ;# 老师教学脚本中可能存在，并非合理的现实面积上限

check_design                           ;# 可选但推荐
check_timing                           ;# 检查约束是否合理
compile                                ;# [7] 技术映射

# [8] 写出网表、SDF、报告
# 层次模块建议明确 -hierarchy，否则可能只导出 Top
write -format verilog -hierarchy -output ${netlist_path}/Syn_$design.v
write_sdf ${netlist_path}/Syn_$design.sdf
source "${script_path}/report.tcl"
```

与 Module 3 手册的区别说明：P13 示意原脚本也可能只使用 `write -format verilog ...`。`-hierarchy` 是我们因 Module 4 Registered Wrapper 模块层次而增加的稳健做法，并非所有手册脚本原样具有该选项。

M3 / M4 / M5 三组 TOP 替换清单：

文件	M3	M4	M5
顶层模块名	`FullAdder`	`StateMachine`	`MatrixMultiplier`

文件	M3	M4	M5
设计 Tcl 文件	FullAdder.tcl	StateMachine.tcl	MatrixMultiplier.tcl（若另有名称，以实际文件为准）
set design	FullAdder	StateMachine	MatrixMultiplier
current_design	FullAdder	StateMachine	MatrixMultiplier
内部 Clock	无	有，100 ns 周期	有；按实际约束

Step 5：改 scripts/run_topdown.tcl

M3 手册 P14 原样结构：

```
source "${script_path}/ReadVerilogFiles.tcl"
source "${script_path}/FullAdder.tcl"
```

M4 改为：

```
source "${script_path}/ReadVerilogFiles.tcl"
source "${script_path}/StateMachine.tcl"
```

M5 改为：

```
source "${script_path}/ReadVerilogFiles.tcl"
source "${script_path}/MatrixMultiplier.tcl"
```

新题 Adder4 改为：

```
source "${script_path}/ReadVerilogFiles.tcl"
source "${script_path}/Adder4.tcl"
```

script_path 是 Tcl 变量，通常由老师工程的 .synopsys_dc.setup 初始化。**如果** design_vision> **中** info exists script_path **返回 0，可先执行** set script_path [file normalize ./scripts]。**但不能用 C Shell 去** source run_topdown.tcl。

Step 6：检查 scripts/report.tcl

M3 原模板具体命令以机器实际脚本为准，建议至少保证：

```
check_design > "${log_path}/${rpt_file}"
report_area >> "${log_path}/${rpt_file}"
report_reference >> "${log_path}/${rpt_file}"
report_timing -delay_type max >> "${log_path}/${rpt_file}"
# 若老师脚本有其他报告，请保留：report_clock、report_constraint、report_power 等
```

这里 > 覆盖创建，>> 追加；不要把多条 > 用在同一文件上导致前面的报告内容被覆盖。report_power 在没有真实 SAIF 时的结果**不是 T4 的最终 Activity-Based Power**。

Step 7：打开 Design Vision 并运行老师脚本

```
cd ~/NM6008/Test2_Exam/Synthesis
csh
source synopsys_linux_vR_2020_cshrc
design_vision &
```

在 GUI：**File → Execute Script →** scripts/run_topdown.tcl（M3 P14–16）。

或 design_vision>：

```
pwd
info exists script_path
set script_path [file normalize ./scripts] ;# 仅在尚未定义时执行
source ./scripts/run_topdown.tcl
current_design
report_area
report_reference
report_timing -delay_type max -max_paths 5
```

若启动时遇到 Error: Current design is not defined (UID-4)，核查 .synopsys_dc.setup 是否过早调用 set_fix_multiple_port_nets -all；**不要以启动信息代替本次 compile 的错误判断**。若运行综合本身失败，应定位 compile/reader 的第一条 Error。

Step 8：确认 T2 产物是新生成的

```
cd ~/NM6008/Test2_Exam/Synthesis
ls -lah --full-time netlist/
```

```
ls -lah --full-time logs/
grep -nE '^module |^endmodule' netlist/Syn_Adder4.v
grep -inE 'Error:|Warning:|VIOLATED|unresolved' logs/report.rpt
```

预期：`Syn_Adder4.v`、`Syn_Adder4.sdf` 和 `logs/report.rpt`。若设计有多个模块，应查看网表是否导出子模块；如果 `write` 没有 `-hierarchy`，极可能漏掉所需定义。

Step 9：认真读 Timing Report 和面积约束

你 Module 4 的真实案例：

```
set_max_area 0
Registered Cell Area = 9.354240
Area max_area slack = -9.35 (VIOLATED)
单独的 reg-to-reg setup path slack ≈ +99.94 ns (MET)
```

结论：**我们当时没有改掉 `set_max_area 0`，只是辨认了 VIOLATED 的类型**。要判断题目是否合格，查看 `report_constraint` 中违例的对象、`report_timing` 的 Startpoint/Endpoint/Data Required/Arrival/Slack，以及 I/O 是否完整设置了 Delay。未完整约束不能声称 Timing Sign-off。

如果题目给真实面积限制，将 `set_max_area 0` 改成合理的正数要求，并对所有比较版本应用**相同**的约束；不得一边设 0，一边不设 0 后称两组结果完全同条件。

A8. T3 — Post-Synthesis Gate-Level Simulation（Module 3 P16–18）

Step 1：复制 Netlist、SDF、TB、向量

```
cd ~/NM6008/Test2_Exam/rtl
cp ../Synthesis/netlist/Syn_Adder4.v .
cp ../Synthesis/netlist/Syn_Adder4.sdf .
cp ../Behavioral/Adder4_tb.sv .
# 如采用外部文件向量：
cp ../Behavioral/InputVector.txt .
ls -lah
```

先检查 `library.f`：

```
cat library.f
```

M3 的 `library.f` 应列出 GF22 单元的 Verilog 功能/时序模型（在你真实机房曾见 `gf22nsdvlogl20hdl116a.v` 等多个 `-v` 入口）。不要把 `.db/.sdb` 写进 `library.f` 冒充门级 Verilog；也不要在没核对路径时复制别的工艺库 filelist。

Step 2：对照你发的 Module 3 P17 截图修改 Testbench

图一的三处修改，是门级仿真必修：

修改 1：加时间单位

在 TB 文件最前面（必须位于 module 声明之前）：

```
`timescale 1ns/1ps
```

解释：1 ns time unit、1 ps precision。`#1` 代表 1 ns，`#0.001` 代表 1 ps。**不要把 `#1` 理解为综合出的固定延迟门。**

如果标准单元文件带有自己的 `timescale` 或当前 VCS 报 `Illegal timescale for module`，先检查文件编译顺序与编译选项；可在当次编译使用适当的 `-timescale=1ns/1ps`，不要直接改写学校提供的 GF22 仿真库。

修改 2：SDF 反标

```
initial $sdf_annotate("Syn_Adder4.sdf", U1);
```

- `Syn_Adder4.sdf` 是 T2 新生成并复制到 T3 目录的文件。
- `U1` 是 TB 里面例化 DUT 的**实例名**，不是 DUT `module` 名。
- 如果 TB 写 `Adder4 dut (...);`，那就写 `$sdf_annotate("Syn_Adder4.sdf", dut);`。
- **同一份 TB 若还要拿去做 T1 RTL 仿真，应仅在门级模式启用 SDF：**

```
`ifdef GATE_SIM
    initial $sdf_annotate("Syn_Adder4.sdf", U1);
`endif
```

T3 编译命令增加 `+define+GATE_SIM`，T1 不加。

修改 3：记录 VPD 切换活动

手册 M3 P17 原样方法是把 `$vcdpluson(U1);` 放在实际激励开始前，把 `$vcdplusoff(U1);` 放在激励结束后，默认生成 `inter.vpd`（具体路径/默认名以实际运行结果为准）。

更清晰的增强建议：

```
initial begin
    $vcdplusfile("inter.vpd");
    $vcdpluson(0, Adder4_tb);  // 全部 TB 层次：包括 U1
end
```

主 Testbench 最后：

```
$vcdplusoff;
$finish;
```

二选一：若采用 `$vcdpluson(U1)` 与 `$vcdplusoff(U1)`，不要同时再插入完全重复的全层次 Dump 而不清楚记录范围。SAIF 的 instance path 必须匹配所用层次。

Step 3：使用 T3 的 VCS 环境 + library.f

```
cd ~/NM6008/Test2_Exam/rtl
csh
source synopsys_linux_vP_2019_cshrc
which vcs
```

老师手册 M3 P18：

```
vcs -debug +v2k +neg_tchk ./FullAdder_tb.v ./Syn_FullAdder.v -f ./library.f
./simv -gui
```

陌生 Adder4（SV 增强版）：

```
vcs -full64 -sverilog -debug_access+all +neg_tchk \
  -timescale=1ns/1ps +define+GATE_SIM \
  ./Adder4_tb.sv ./Syn_Adder4.v -f ./library.f \
  -o simv_gate -l compile_gate.log
./simv_gate -l sim_gate.log
```

必须主动检查：

```
grep -inE 'Error|Warning|SDF|PASS|FAIL' compile_gate.log sim_gate.log
ls -lh inter.vpd
```

- `Doing SDF annotation ... Done` 是有用证据，但仍需留意实例/弧匹配告警。
- `+neg_tchk` 与某些 Timing Check Warning 的处理有关，**不是一条命令就能保证时序满足要求**。
- `sim_gate.log` 的 PASS 不等于全部可能路径已验证；依然要判断检查覆盖了哪些 testpoint。

Step 4：在 DVE 中看完整波形

```
dve -vpd inter.vpd &
```

或手册交互式 `./simv -gui`，在 DVE：选 TB scope → 选输入/输出信号 → Add to Waves → New Wave View → Start/Continue。

要截图的三类行为：

- M3：8 个输入组合、S/COUT 真实变化与传播延迟。
- M4：CLK、NRST、start、rst、state、A/B/CIN、S/COUT；放大 SDF Glitch/TcQ。
- M5：输入样本顺序、运算过程、OUT、OUT_STROBE 有效周期。

A9. T4 — Power：完整 VPD → VCD → SAIF → Power Compiler（M3 P19–20）

Step 1：修改 GF22 搜索路径（你图二遗漏的问题）

进入目录并检查实际文件位置：

```
cd ~/NM6008/Test2_Exam/rtl/PowerAnalysis
pwd
```

```
ls -lah ../../Library/db ../../Library/sdb
ls -lah common_setup_gf22.tcl .synopsys_dc.setup
```

为什么路径不是照抄老师的 `/home/msc_tum_grp1/...`**？** 因为老师账号、你的账号、Module 序号可能都不同。M3 P20 要求把 `ADDITIONAL_SEARCH_PATH1` 修改为本机真实 `Library`。你曾使用的路径结构：

```
# 当前工作目录为 Test2_Exam/rtl/PowerAnalysis 时：
set ADDITIONAL_SEARCH_PATH1 "../../Library/db"
set ADDITIONAL_SEARCH_PATH2 "../../Library/sdb"
set ADDITIONAL_SEARCH_PATH3 "../../Library"
```

相当于：

```
PowerAnalysis → ../rtl → ../../Test2_Exam → ../../Library/db
```

绝对路径写法（可以用，但要改成自己的真实路径）：

```
set ADDITIONAL_SEARCH_PATH1 "/home/msc_tum_grp2/NM6008/Test2_Exam/Library/db"
```

如果你的 `common_setup_gf22.tcl` 已经定义 `target_library`、`link_library`、`search_path`，只改错的查找路径，不用整份覆盖。打开 `common_setup_gf22.tcl` 检查 `TARGET_LIBRARY_FILES` 的文件名**真实存在**；路径/库不对，DC 会报 `Cannot find target library` 或 unresolved references。

与图一关联： 没有 T3 中 `$vcdpluson` 生成的 `inter.vpd`，T4 就无法用实测激励做活动功耗估算。

Step 2：准备 VPD，并转换成 SAIF

```
cd ~/NM6008/Test2_Exam/rtl/PowerAnalysis
ls -lh ../inter.vpd
```

如果文件不存在：回到 `../`，检查 TB 是否打开 VPD dump、是否运行了门级仿真、输出路径是否与预期一致。

按照 M3 P20：

```
vpd2vcd ../inter.vpd ./inter.vcd
vcd2saif -i ./inter.vcd -o ./inter.saif
ls -lh inter.vcd inter.saif
```

这三个文件的意义：

文件	生产者	内容	消费者
`inter.vpd`	VCS `$vcdpluson`	仿真波形/切换活动	DVE、`vpd2vcd`
`inter.vcd`	`vpd2vcd`	文本格式 Value Change Dump	`vcd2saif`
`inter.saif`	`vcd2saif`	层次化活动统计	Power Compiler 的 `read_saif`

DVE 看波形只需要 VPD，Power 流程需 SAIF。

Step 3：启动 R-2020 Design Compiler / Power Compiler

```
cd ~/NM6008/Test2_Exam/rtl/PowerAnalysis
csh
source ../../Synthesis/synopsys_linux_vR_2020_cshrc
which dc_shell-xg-t
dc_shell-xg-t
```

确保提示符变成：

```
dc_shell>
```

Step 4：在 `dc_shell>` 内执行 Tcl（不是 Linux C Shell）

M3 原始手册 P20：

```
read_verilog ../Syn_FullAdder.v
current_design FullAdder
link
read_saif -input ./inter.saif -instance FullAdder_tb/U1
report_power >> "rpt_power.txt"
report_power -hier >> "rpt_power_hier.txt"
report_power -hier -nosplit >> "rpt_power_hier_nosplit.txt"
report_power -cell >> "rpt_power_cell.txt"
```

工程增强建议（时序设计）： 在 `link` 完成后加上与综合一致的时钟定义，例如 Module 4：

```
create_clock -name CLK -period 100 [get_ports CLK]
```

然后：

```
report_saif -hierarchy
report_saif -missing
report_power
```

确认 `report_saif` 中的 annotated Nets/Ports/Pins 对应正确层次且不是全部 0%，并且 `report_saif -missing` 中没有需要分析的未覆盖对象。使用 `report_power > rpt_power.txt` 保存一次新运行的报告；`>>` 会累积多个时间点的数据，容易误读旧结果。

Step 5：对照 M3/M4/M5 修改两个一定要修改的字符串

场景	current_design	read_saif -instance（示例，真实 TB 以源码为准）
M3	FullAdder	FullAdder_tb/U1
M4	StateMachine	tb_StateMachine_post/dut
M5	MatrixMultiplier	tb_MatrixMultiplier/dut（并非手册指定名称）
陌生 Adder4	Adder4	Adder4_tb/U1

注意：T3 新建 TB `tb_Adder4_gate` 与 T1 的 `Adder4_tb` 若名字不同，应填真实门级 TB 名。Power 映射失败时，先打开 SAIF 文件看层次，不要重复试错猜路径。

Step 6：看懂 Power Report

```
Cell Internal Power  = XXX nW
Net Switching Power  = YYY nW
Total Dynamic Power  = XXX+YYY nW
Cell Leakage Power   = ZZZ nW
Total Estimated Power = Dynamic + Leakage（统一单位后自行相加）
```

M4 Optional 的真实练习结果（不同电路的对比，不是陌生题默认目标）：

项	Baseline	Registered
State/Output FF 总数	2	4
Leaf Cells	12	14
Area	6.514560	9.354240
Internal	153.7659 nW	255.6843 nW
Net Switching	32.8061 nW	40.4426 nW
Dynamic	186.5719 nW	296.1269 nW
Leakage	676.2555 nW	945.6757 nW
Total（Dynamic+Leakage）	862.8274 nW	1241.8026 nW

额外 FF 的面积开销：`(9.354240 / 6.514560 - 1)*100% ≈ 43.59%`；总功耗开销：`(1241.8026 / 862.8274 - 1)*100% ≈ 43.92%`。这种对比的**前提**是相同激励、Corner、Power Compiler 版本及分析条件，不能把无 SAIF 注释的默认估算与正常 SAIF 功耗拿来直接相减。

签核限制：本实验是 GF22 库 + ZeroWLM 等条件下的 Pre-layout Power Estimate，不是含真实布局布线寄生参数的硅片 Sign-off Power。

A10. T1/T2/T3/T4 每次结束必须执行的人工签核

T1：Behavioral（无此项不进 T2）

- ☐ `compile_beh.log` 没有 blocking Error。
- ☐ TB `TEST PASS` 真正来自 assert/checker，不是无条件 `$display("PASS")`。
- ☐ 输入场景覆盖了题面真值表/状态转移/边界。
- ☐ DVE 波形能解释期望结果，重置、有效周期正确。

T2：Synthesis（无此项不进 T3）

-

- [ ] `current_design` 是本次题的 TOP。
- [ ] `report_reference/report_cell` 使用 GF22 标准单元，无不该存在的黑盒。
- [ ] 没有 unintended latches；所有子模块有定义。
- [ ] Clock/IO/Area 约束符合题目，所有违例有原因和说明。
- [ ] `Syn_TOP.v/Syn_TOP.sdf` 为本次新产物，不是复制模板残留。

T3：Post-Synthesis（无此项不进 T4）

- [ ] 编译时 `-f library.f` 正确定位标准单元 Verilog 模型。
- [ ] SDF annotation 完成，没有影响有效性的层次匹配错误。
- [ ] 门级 TB 真的 PASS；观察 X/Glitch/输出延迟是否合理。
- [ ] 新的 `inter.vpd` 存在，时间戳对应本次仿真。

T4：Power（无此项不能宣称完成功耗分析）

- [ ] `inter.vcd`、`inter.saif` 都存在且非空。
- [ ] Power DC `link` 没有 unresolved cells。
- [ ] `read_saif` 层次匹配，`report_saif` 显示有效注释。
- [ ] `rpt_power.txt`、`rpt_power_hier.txt`、`rpt_power_hier_nosplit.txt`、`rpt_power_cell.txt` 存在并来自本次运行。
- [ ] 正确区分 Internal / Switching / Dynamic / Leakage；写明电压、Corner、周期、活动窗口、工具版本。

B. 四阶段 csh run 参考模板（半自动；必须先在机房验证）

> **风险声明**：下述四个 run 和两个 Tcl 辅助脚本是可阅读、可改造的**教学参考实现**。它们在本对话环境中**不能访问学校 Synopsys 许可证/PDK，也没有在你的 CentOS + VCS/DC 上实跑**；不能保证与每种课程模板兼容。尤其 T2 的老师 Tcl 有自己的 MyDefaults.con、变量和 .synopsys_dc.setup。**第一次陌生题建议 A 部分手工流程，已跑通后才用自动化。**
>
> **别误解“一条 csh run”**：它替你调用工具、管理输入/日志，**不会**理解题目、重写 RTL、自动填完 TB、决定波形是否正确。若生成了输出但没有 Checker PASS，仍然失败。

B0. 目标目录与配置文件

```
Test2_Exam/
├── exam_config.csh       ← 统一设置名字（你必须填写）
├── Behavioral/run         ← T1
├── Synthesis/run          ← T2
│   └── scripts/exam_bootstrap.tcl  ← T2 辅助启动 Tcl
└── rtl/
    ├── run                ← T3
    └── PowerAnalysis/
        ├── run            ← T4
        └── exam_power.tcl ← T4 功耗 Tcl
```

每次新题，先修改 exam_config.csh：

```
# exam_config.csh —— 必须在 C Shell 中 source
set TOP          = "Adder4"
set TB_FILE      = "Adder4_tb.sv"
set TB_MODULE    = "Adder4_tb"
set DUT_INSTANCE = "U1"
set RTL_FILES    = ( "Adder4.v" "FullAdder.v" )

# 组合逻辑：HAS_CLOCK=0；时序题 HAS_CLOCK=1
set HAS_CLOCK    = 0
set CLK_PORT     = "CLK"
set CLK_PERIOD_NS = 100
```

字段一一对应：

配置	必须对应哪里	常见错法
TOP	设计 Verilog module ...、设计 Tcl 中 Top、综合网表名	把 TB 名写成 TOP
TB_FILE	Behavioral 文件名	大小写不一致；文件在 tb/ 而非 Behavioral/
TB_MODULE	TB 的 module 名、VPD hierarchy	把 DUT 的 module 名填进去
DUT_INSTANCE	TB 中 Adder4 U1 (...) 的 U1	写成 TOP 名称
RTL_FILES	全部真实设计 .v 文件；**不包含 TB**	缺少一个子模块
HAS_CLOCK	0 组合题、1 时序题	FullAdder 平白加 CLK
CLK_PORT	时序设计的实际时钟端口	CLK 与 clk 混淆
CLK_PERIOD_NS	题目要求的 ns 周期，T2/T4 与 TB 一致	把频率当周期

三个已学 Module 的 CONFIG 实例：

```
# M3：FullAdder，组合
set TOP="FullAdder"
set TB_FILE="FullAdder_tb.v"
set TB_MODULE="FullAdder_tb"
set DUT_INSTANCE="U1"
set RTL_FILES=( "FullAdder.v" )
set HAS_CLOCK=0
```

```
# M4：StateMachine，时序（TB 具体文件名/实例名以真实代码为准）
set TOP="StateMachine"
set TB_FILE="tb_StateMachine.sv"
set TB_MODULE="tb_StateMachine"
set DUT_INSTANCE="dut"
set RTL_FILES=( "StateMachine.v" )
set HAS_CLOCK=1
```

```
set CLK_PORT="CLK"
set CLK_PERIOD_NS=100
```

```
# M5：MatrixMultiplier，时序（子模块/文件名用 `ls` 核实）
set TOP="MatrixMultiplier"
set TB_FILE="tb_MatrixMultiplier.sv"
set TB_MODULE="tb_MatrixMultiplier"
set DUT_INSTANCE="dut"
set RTL_FILES=( "MatrixMultiplier.v" "Multiplier.v" "Accumulator.v" "StateMachine.v" )
set HAS_CLOCK=1
set CLK_PORT="CLK"
set CLK_PERIOD_NS=100  # 这里只是例子，不是 M5 手册硬性规定
```

把 GUI 里实际的实例名抄给配置，不要只因为模块相似就猜。

B1. T1 — `Behavioral/run` 完整参考脚本

```
#!/bin/csh -f
# 使用方式：cd Test2_Exam/Behavioral; csh run
if (! -f ../exam_config.csh) then
    echo "ERROR: ../exam_config.csh 不存在"
    exit 2
endif
source ../exam_config.csh
if (! -f "$TB_FILE") then
    echo "ERROR: TB 文件不存在：$TB_FILE"
    exit 2
endif
foreach f ( $RTL_FILES )
    if (! -f "$f") then
        echo "ERROR: RTL 文件不存在：$f"
        exit 2
    endif
end
if (! -f ./synopsys_linux_vP_2019_cshrc) then
    echo "ERROR: 缺少 VCS 环境脚本"
    exit 2
endif
source ./synopsys_linux_vP_2019_cshrc
which vcs
if ($status != 0) exit 2

echo "=== T1: compiling $TOP ==="
vcs -full64 -sverilog -debug_access+all \
    -o simv -l compile_beh.log ./$TB_FILE $RTL_FILES
if ($status != 0) then
    echo "FAIL T1 compile：查看 compile_beh.log"
    exit 3
endif

echo "=== T1: simulating ==="
./simv -l sim_beh.log
if ($status != 0) then
    echo "FAIL T1 sim：查看 sim_beh.log"
    exit 4
endif

echo "T1 完成运行；必须手动检查 Checker PASS、波形和覆盖点"
ls -lh compile_beh.log sim_beh.log
```

执行：

```
cd ~/NM6008/Test2_Exam/Behavioral
csh run
```

人工检查：打开 `sim_beh.log` 看 PASS，查看 TB 是否真的有比较语句；若 TB 没有自动退出（例如 `$finish`），脚本会一直运行，不能说“自动完成”。

B2. T2 — `Synthesis/run` + `scripts/exam_bootstrap.tcl`

这一阶段非常容易因 `script_path`/原始老师 Tcl 环境而失败。**先准备**：

1. 把老师旧 `<旧Top>.tcl` 复制为 `scripts/Adder4.tcl`。
2. 改里面 `set design`、`current_design`、时钟周期、输出文件名等（见 A7）。
3. 把 `scripts/run_topdown.tcl` 的第二条 `source` 改为 `Adder4.tcl`。
4. 确保 `Synthesis/sources/` 会收到所有 RTL。
5. 原始 `.synopsys_dc.setup` 和 GF22 Library 路径仍然有效。

scripts/exam_bootstrap.tcl：

```
# 由 DC / Design Vision 的 Tcl 解释器执行；不是 C Shell
set script_path [file normalize ./scripts]
set netlist_path [file normalize ./netlist]
set log_path [file normalize ./logs]
set ddc_path [file normalize ./ddc]
file mkdir $netlist_path $log_path $ddc_path
source ./scripts/run_topdown.tcl
# 这里没有擅自覆盖老师的报告配置
# 批处理结束后退出 DC；GUI 手工方案不用执行本文件
exit
```

Synthesis/run：

```
#!/bin/csh -f
# 使用方式：cd Test2_Exam/Synthesis; csh run
if (! -f ../exam_config.csh) then
    echo "ERROR: 缺少 ../exam_config.csh"
    exit 2
endif
source ../exam_config.csh
if (! -f ./scripts/${TOP}.tcl) then
    echo "ERROR: 缺少 scripts/${TOP}.tcl；先按 A7 修改老师设计 Tcl"
    exit 2
endif
if (! -f ./scripts/run_topdown.tcl) then
    echo "ERROR: 缺少 run_topdown.tcl"
    exit 2
endif
if (! -f ./scripts/exam_bootstrap.tcl) then
    echo "ERROR: 缺少 scripts/exam_bootstrap.tcl"
    exit 2
endif

# 把当前 RTL 拷入 sources（老师原 sources 旧文件不直接删除）
mkdir -p sources netlist logs ddc
foreach f ( $RTL_FILES )
    if (! -f "../Behavioral/$f") then
        echo "ERROR: 找不到 Behavioral/$f"
        exit 2
    endif
    cp -p "../Behavioral/$f" ./sources/
end

# 自动重写 ReadVerilogFiles.tcl；先备份原文件
set stamp = `date +%Y%m%d_%H%M%S`
if ( -f scripts/ReadVerilogFiles.tcl ) then
    cp -p scripts/ReadVerilogFiles.tcl scripts/ReadVerilogFiles.tcl.backup_$stamp
endif
echo "# Auto-generated by exam T2 run; verify names" >! scripts/ReadVerilogFiles.tcl
foreach f ( $RTL_FILES )
    echo "read_verilog ./sources/$f" >> scripts/ReadVerilogFiles.tcl
end

# 旧 netlist 报告移入备份目录，避免误把旧产物当新结果
set backupdir = "old_output_${stamp}_$$"
mkdir -p "$backupdir"
if ( -f "netlist/Syn_${TOP}.v" ) mv "netlist/Syn_${TOP}.v" "$backupdir/"
if ( -f "netlist/Syn_${TOP}.sdf" ) mv "netlist/Syn_${TOP}.sdf" "$backupdir/"
if ( -f "logs/report.rpt" ) mv "logs/report.rpt" "$backupdir/"

if (! -f ./synopsys_linux_vR_2020_cshrc) then
    echo "ERROR: 缺少 DC 环境脚本"
    exit 2
endif
source ./synopsys_linux_vR_2020_cshrc
which dc_shell-xg-t
if ($status != 0) exit 2

echo "=== T2: batch synthesis $TOP ==="
dc_shell-xg-t -f ./scripts/exam_bootstrap.tcl >&! run_exam_syn.log
if ($status != 0) then
    echo "FAIL T2：查看 run_exam_syn.log；可退回老师 Design Vision GUI 手工执行"
    exit 3
endif

if (! -s "netlist/Syn_${TOP}.v" || ! -s "netlist/Syn_${TOP}.sdf") then
    echo "FAIL T2：未生成非空的网表/SDF，检查命令、script_path、Top、GF22 库"
    exit 4
endif
```

```
echo "T2 已生成输出，但请检查 report.rpt、真实的 compile Error、Timing 与路径"
ls -lh "netlist/Syn_${TOP}.v" "netlist/Syn_${TOP}.sdf"
```

> **重点**：这段批处理保留了老师的 `run_topdown.tcl`。如果该 Tcl 中引用的 `script_path`、`netlist_path`、`ddc_path`、`log_path` 与你的原始配置不兼容，**不要硬改库，退回手册的 Design Vision GUI 执行**。`-f` 的批处理适用于 DC，但不同版本退出码和警告处理不保证一致；看日志+产物时间戳是必需的。

执行：

```
cd ~/NM6008/Test2_Exam/Synthesis
csh run
```

B3. T3 — `rtl/run` 完整参考脚本

先保证 TB 中已经按 A8 写好（不能仅改 config）：

```
`timescale 1ns/1ps
module Adder4_tb;
    // ... IO、DUT U1、Stimulus、Checker ...
`ifdef GATE_SIM
    initial $sdf_annotate("Syn_Adder4.sdf", U1);
`endif
`ifdef POWER_DUMP
    initial begin
        $vcdplusfile("inter.vpd");
        $vcdpluson(0, Adder4_tb);
    end
`endif
    // 在激励结束后 $finish; 用于终止运行
endmodule
```

`rtl/run`：

```
#!/bin/csh -f
# 使用方式：cd Test2_Exam/rtl; csh run
if (! -f ../exam_config.csh) then
    echo "ERROR: 缺少 exam_config.csh"
    exit 2
endif
source ../exam_config.csh
if (! -f ./library.f) then
    echo "ERROR: library.f 不存在，不能门级编译"
    exit 2
endif
if (! -s "../Synthesis/netlist/Syn_${TOP}.v") then
    echo "ERROR: 综合网表不存在/为空"
    exit 2
endif
if (! -s "../Synthesis/netlist/Syn_${TOP}.sdf") then
    echo "ERROR: SDF 不存在/为空"
    exit 2
endif
if (! -f "../Behavioral/$TB_FILE") then
    echo "ERROR: TB 不存在"
    exit 2
endif

cp -p "../Synthesis/netlist/Syn_${TOP}.v" .
cp -p "../Synthesis/netlist/Syn_${TOP}.sdf" .
cp -p "../Behavioral/$TB_FILE" .
if ( -f ../Behavioral/InputVector.txt ) cp -p ../Behavioral/InputVector.txt .

# 保存上一次产生的 VPD 避免误认
if ( -f inter.vpd ) then
    set stamp = `date +%Y%m%d_%H%M%S`
    mv inter.vpd inter_old_${stamp}_$$.vpd
endif

if (! -f ./synopsys_linux_vP_2019_cshrc) then
    echo "ERROR: 缺少 VCS 环境脚本"
    exit 2
endif
source ./synopsys_linux_vP_2019_cshrc
which vcs
if ($status != 0) exit 2

echo "=== T3: gate compile $TOP ==="
vcs -full64 -sverilog -debug_access+all +neg_tchk \
```

```
    -timescale=1ns/1ps +define+GATE_SIM +define+POWER_DUMP \
    -f ./library.f ./$TB_FILE ./Syn_${TOP}.v \
    -o simv_gate -l compile_gate.log
if ($status != 0) then
    echo "FAIL T3 compile：查看 compile_gate.log"
    exit 3
endif

./simv_gate -l sim_gate.log
if ($status != 0) then
    echo "FAIL T3 sim：查看 sim_gate.log"
    exit 4
endif
if (! -s inter.vpd) then
    echo "FAIL T3：没有 inter.vpd。检查 TB 的 POWER_DUMP、vcdplusfile/on"
    exit 5
endif

echo "T3 已生成 VPD；仍需检查 SDF annotation、TB PASS、Timing Check Warning"
ls -lh inter.vpd sim_gate.log
```

执行：

```
cd ~/NM6008/Test2_Exam/rtl
csh run
```

额外提醒：如果 `RTL_FILES` 中多个模块定义被综合为包含 `module` 的单个 `Syn_TOP.v`，T3 **只编译完整综合网表**，不要再同时编译所有原始 RTL，避免重复定义或混用 RTL/门级实现。

B4. T4 — `rtl/PowerAnalysis/run` + `exam_power.tcl`

`rtl/PowerAnalysis/exam_power.tcl`：

```
# 使用前确认：当前工作目录在 rtl/PowerAnalysis
# 工艺库读取：以原工程 .synopsys_dc.setup 为主；额外 source 可确保配置一致
if {[file exists "./common_setup_gf22.tcl"]} {
    source ./common_setup_gf22.tcl
}

set design $env(NM_TOP)
read_verilog "../Syn_${design}.v"
current_design $design
link

if {$env(NM_HAS_CLOCK) == "1"} {
    create_clock -name $env(NM_CLK_PORT) \
        -period $env(NM_CLK_PERIOD_NS) \
        [get_ports $env(NM_CLK_PORT)]
}

# 这是 Testbench/DUT 的层次路径，不是电路层次文件路径
set instance "$env(NM_TB_MODULE)/$env(NM_DUT_INSTANCE)"
read_saif -input ./inter.saif -instance $instance

report_saif -hierarchy > ./rpt_saif.txt
report_saif -missing >> ./rpt_saif.txt
report_power > ./rpt_power.txt
report_power -hier > ./rpt_power_hier.txt
report_power -hier -nosplit > ./rpt_power_hier_nosplit.txt
report_power -cell > ./rpt_power_cell.txt
exit
```

`rtl/PowerAnalysis/run`：

```
#!/bin/csh -f
# 使用方式：cd Test2_Exam/rtl/PowerAnalysis; csh run
if (! -f ../../exam_config.csh) then
    echo "ERROR: 缺少 exam_config.csh"
    exit 2
endif
source ../../exam_config.csh
if (! -s ../inter.vpd) then
    echo "ERROR: T3 的 ../inter.vpd 不存在/为空"
    exit 2
endif
if (! -f ./exam_power.tcl) then
    echo "ERROR: 缺少 exam_power.tcl"
    exit 2
endif
if (! -f ../../Synthesis/synopsys_linux_vR_2020_cshrc) then
    echo "ERROR: 缺少 R-2020 Synopsys 环境脚本"
```

```
    exit 2
endif

source ../../Synthesis/synopsys_linux_vR_2020_cshrc
which vpd2vcd
if ($status != 0) exit 2
which vcd2saif
if ($status != 0) exit 2
which dc_shell-xg-t
if ($status != 0) exit 2

# 旧的 power/SAIF/VCD 备份，防止误判复用旧文件
set stamp = `date +%Y%m%d_%H%M%S`
set backupdir = "old_power_${stamp}_$$"
mkdir -p "$backupdir"
foreach f ( inter.vcd inter.saif rpt_power.txt rpt_power_hier.txt rpt_power_hier_nosplit.txt
rpt_power_cell.txt rpt_saif.txt )
    if ( -f "$f" ) mv "$f" "$backupdir/"
end

vpd2vcd ../inter.vpd ./inter.vcd
if ($status != 0 || ! -s ./inter.vcd) then
    echo "FAIL vpd2vcd"
    exit 3
endif
vcd2saif -i ./inter.vcd -o ./inter.saif
if ($status != 0 || ! -s ./inter.saif) then
    echo "FAIL vcd2saif"
    exit 4
endif

setenv NM_TOP "$TOP"
setenv NM_TB_MODULE "$TB_MODULE"
setenv NM_DUT_INSTANCE "$DUT_INSTANCE"
setenv NM_HAS_CLOCK "$HAS_CLOCK"
setenv NM_CLK_PORT "$CLK_PORT"
setenv NM_CLK_PERIOD_NS "$CLK_PERIOD_NS"

dc_shell-xg-t -f ./exam_power.tcl >&! run_exam_power.log
if ($status != 0 || ! -s ./rpt_power.txt) then
    echo "FAIL T4：检查 DC 报错和 SAIF/Library 路径"
    exit 5
endif

echo "T4 已生成报告；必须检查 rpt_saif.txt 的 annotation 百分比！"
ls -lh rpt_power*.txt rpt_saif.txt
```

执行：

```
cd ~/NM6008/Test2_Exam/rtl/PowerAnalysis
csh run
```

自动化仍然会遗漏的事：`read_saif` 可能返回成功，但 Activity 映射不到预期 DUT；`report_power` 可能仍有数字，却来自默认活动假设。只有 `rpt_saif.txt` 和真实功耗数据都检查通过，才算 T4 有效。

B5. 四个自动脚本怎样测：不要上考场直接第一次运行

推荐在已通过手动流程的 **Module 3 复制品** 做验证，不要拿唯一原始目录冒险。

```
cd ~/NM6008/Test2_Exam/Behavioral
csh run
# 检查 sim_beh.log

cd ../Synthesis
csh run
# 核对两个综合网表/SDF 文件和 report.rpt

cd ../rtl
csh run
# 核对 SDF 与 inter.vpd

cd PowerAnalysis
csh run
# 核对 rpt_saif.txt 和四种 Power Report
```

某个脚本跑不通的安全回退：保留该阶段的原始输入文件，按 A 部分逐条手动执行；不要为让 `run` 绿灯而删掉 `-f library.f`、`$sdf_annotate`、TB checker、或者把 `read_saif` 变成可选。

C. 常见故障：按照 现象 → 根因 → 去哪里改 → 修复命令 排查

现象（真实/常见）	首先怀疑	具体动作
`[user@pc]$ vcs: Command not found`	没加载 P-2019	`csh` → `source ...vP_2019_cshrc` → `which vcs`
`[user@pc]$ dc_shell-xg-t: Command not found`	没加载 R-2020	`csh` → `source ...vR_2020_cshrc` → `which dc_shell-xg-t`
`puts: Command not found`	把 Tcl 当 C Shell 脚本执行	先启动 `dc_shell-xg-t`，只在 `dc_shell>` 中 `source common_setup_gf22.tcl`
`script_path: Undefined variable`	在 Linux Shell source 了 run_topdown.tcl；或 DC 未初始化 `script_path`	`design_vision>` 下 `info exists script_path`，必要时 `set script_path [file normalize ./scripts]`
`Current design is not defined (UID-4)`	`.synopsys_dc.setup` 在 `read_verilog` 前调用了需要 Current Design 的命令	识别它发生的阶段；启动后读设计/设 Top/Link；必要时把 `set_fix_multiple_port_nets` 移到 Current Design 有效之后
`Unresolved reference` / unknown module	少编译了子模块/标准单元库未加载	T2 检查 ReadVerilogFiles；T3 检查 `-f library.f` 的路径和模块定义；T4 检查 .db/link_library
`Unknown type 'wire'` / port syntax	Verilog 文件某行拼写有误或文件未保存	打开报错中的具体文件/行；看 `wire` 之前是不是漏了逗号、分号、关键字
`syntax error near always`	上一个 `always`/`case` 缺 `end`/`endcase`，或多余字符	优先查报错**前**的 5–10 行，不只盯着报错行
`Syntax error token ';'`	`assign` 的括号未匹配、表达式不完整	缩小表达式，逐个检查 `( )`、`&` `` ` ``
VCS `Illegal timescale for module`	TB 有 timescale，之前的模块没有；编译次序/全局 timescale 冲突	统一当次编译的 time unit，审查 TB 顶部声明和库文件编译方式；不要改 GF22 库
VCS SDB database generated by different version	旧 `simv.daidir`/仿真数据库版本冲突	使用新的输出名/新构建目录，或备份旧数据库后重新编译；别先删唯一证据
`wave.vpd doesn't exist`	TB 没真正调用 `$vcdpluson`；文件另在其他目录；仿真未执行	在仿真 cwd `find . -name '*.vpd' -print`；查看 TB 和 sim log
DVE 波形一条直线或全是 0	时间尺度缩太远、没真正运行、信号/区间选错	Zoom Fit/局部放大，检查信号时钟和 TB run 时间；确认不是历史 VPD
`library.f: No such file`	目录不对或大小写 RTL/rtl	`pwd; find . -name library.f -print; cat <path>`；确保 `-f` 路径基于实际仿真 cwd
SDF annotation warning / mismatch	`$sdf_annotate` 的实例名不等于 DUT，网表层次不符	检查 `module TB` 内实例名、SDF CELLPATH、综合网表实例名
Gate 输出 X 或门级检查失败	Reset/时序窗口/SDF 延迟、功能 bug 或 Timing Check	放大 DVE，先排查 Reset 是否已释放、TB 是否过早在输入变化同刻采样；不要直接屏蔽所有 warning
`Clock not found` / `No clocks in this design`	无时钟设计，或约束 Top/端口名错	M3 FullAdder 无内部 CLK 可以没有 Clock；M4 必须检查 `get_ports CLK` / `report_clock`
`Slack -9.35 (VIOLATED)` 与 `+99.94 (MET)` 并存	**不同类型的违例**	看 `report_constraint` 行附近；Module 4 `set_max_area 0` 导致面积 Slack -9.35，不是 setup -9.35
`vpd2vcd: registered.vpd doesn't exist`	Registered 仿真没跑/没打开 Dump	先 `./simv_registered -l sim_registered.log` 再 `ls -lh registered.vpd`，无文件不能继续转换
`vcd2saif can't open .vcd`	上一步 VPD→VCD 失败	不要无视上一个错误继续执行下一步；确认 `.vcd` 非空
`Cannot find SAIF file (PWR-201)`	路径错/没转换出文件/在错误 cwd 启动 DC	`pwd; ls -lh inter.saif`；PowerAnalysis 的 cwd 是否正确
`report_saif` Nets/Ports/Pins 都 0%	SAIF 内 Testbench/DUT 层次写错	改 `read_saif -instance TB_NAME/U1`，检查 SAIF scope，别拿默认活动估算做结论
`report_switching_activity: unknown command`	本 DC 版本不支持该命令	用可运行的 `report_saif -hierarchy` / `report_saif -missing`；先看 `help`
`report_power > file` 之后文件没找到	在错误的 DC cwd 写到别处	DC 输入 `pwd`；Linux 用 `find . -name 'rpt_power*'` 查找
Power 数值与先前不同	工具版本、库、激励、窗口、SAIF 覆盖率不同	同版本/同 corner/同 activity 条件重跑两组，再计算 overhead
`quit: Command not found`	已经回到普通 Shell，`quit` 不是 Shell 指令	终端退出当前 shell 用 `exit` 或 Ctrl+D；要关闭 GUI 用 GUI 的 File→Exit

现象（真实/常见）	首先怀疑	具体动作
^C 仍然存在提示符	Ctrl+C 只中断前台程序，不是关终端	看提示符；如果已是 [user@pc]$ 就在 Shell 中，继续或 exit；套了几层 csh 需要逐层退出

故障处理黄金顺序：

1. 读第一条 Error（不要只看最后一行）
2. 定位是 Shell / VCS Compiler / VCS Runtime / DC Tcl / SDF / Power 哪一层
3. 确认 pwd 和真实文件路径（尤其大小写）
4. 看“本次”日志和文件时间戳
5. 只改最小必要部分，重新执行失败阶段及所有下游依赖阶段
6. 保留错误记录，确认原因得到解释

D. 脚本/代码逐个怎么查：考试前必背的 20 个工具与命令

命令/语句	在哪一层使用	作用	例子
pwd	Linux / Tcl 环境各自	查看当前目录	pwd
ls -lah	Linux	文件、隐藏文件、大小、时间	ls -lah netlist/
cat file	Linux	输出文件全文	cat library.f
less file	Linux	分页查看	less logs/report.rpt; q 退出
find ... -name ...	Linux	找文件	find ~/NM6008 -name library.f -print
grep -RniE	Linux	递归忽略大小写的正则搜索 + 行号	`grep -RniE 'create_clock
cp -a	Linux	保留属性/目录结构复制	cp -a Module5 Test2_Exam
csh	Linux Shell	进入 C Shell	csh
source ...cshrc	**C Shell**	加载 Synopsys 环境	source synopsys_linux_vP_2019_cshrc
which vcs	Linux Shell	检查可执行程序路径	which vcs
vcs	Linux Shell	编译/链接仿真程序	vcs ... -o simv -l compile.log
./simv -gui	Linux Shell	仿真 + DVE GUI	./simv -gui
./simv -l sim.log	Linux Shell	仿真 + 日志	./simv -l sim.log
read_verilog	DC Tcl	读取设计 Verilog	read_verilog ./sources/Top.v
current_design	DC Tcl	选当前 Top	current_design Top
create_clock	DC Tcl	施加时钟约束	create_clock -period 100 [get_ports CLK]
compile	DC Tcl	工艺映射综合	compile
write_sdf	DC Tcl	导出延迟文件	write_sdf ./netlist/Syn_Top.sdf
$sdf_annotate	**SV/Verilog TB**	门级 SDF 反标	$sdf_annotate("Syn_Top.sdf", dut)
$vcdpluson	**SV/Verilog TB**	记录波形活动	$vcdpluson(0, tb_Top)
vpd2vcd	Linux Shell	VPD → VCD	vpd2vcd inter.vpd inter.vcd
vcd2saif	Linux Shell	VCD → SAIF	vcd2saif -i inter.vcd -o inter.saif
read_saif	DC Tcl	映射活动	read_saif -input inter.saif -instance TB/dut
report_saif	DC Tcl	检查 Annotation	report_saif -hierarchy
report_power	DC Tcl	估算动态/静态功耗	report_power > rpt_power.txt

D1. 你之前特别问过的 `cat / ls / grep`：亲自看一次就理解

例如：

```
cat library.f
```

可能打印：

```
-v ../Library/gf22nsdvlogl20hdl116a.v
-v ../Library/gf22nsdvlogl20hdl116a_updated.v
-v ../Library/atpg_primitives.v
...
```

这表示 VCS 按需从这些 **Verilog 库**中查找单元定义。

```
ls -la
```

会看到 `.synopsys_dc.setup` 这种隐藏文件；

```
ls -lh
```

默认**看不到隐藏文件**，但字节大小会显示成 K/M；

```
grep -RniE 'create_clock|CLOCK_PERIOD|compile' scripts/
```

会列出 `scripts/StateMachine.tcl:31:create_clock ...` 之类文件、行号和匹配内容，可以迅速判断 Clock 约束在哪一份脚本里生效。

E. 面向考试的最终行动清单

E1. 考试前：必须在实验室准备而不是临场猜

1. 至少用 M3 已跑通的复制品完整跑一次 T1/T2/T3/T4，并留存四阶段原始日志。
2. 试运行一次 B 部分的四个 `csh run`；**任一阶段失败就回退 A 手动流程，不要贸然依赖脚本**。
3. 确认 `csh`、`vcs`、`design_vision`、`dc_shell-xg-t`、`vpd2vcd`、`vcd2saif` 都可正常定位。
4. 熟悉 `ls -lah`、`find`、`grep`、`less`、`pwd`，确保找不到文件时能独立排查。
5. 逐份手动检查 `.synopsys_dc.setup`、`common_setup_gf22.tcl`、`library.f` 的本机相对路径。
6. 将此 `.md` 与合法共享的自写 TB/Tcl 模板保存在个人目录；禁止外传工艺库和学校限制访问的 IP/脚本。

E2. 考试当天：拿到一份完全陌生的题目

- ☐ **第一轮读题**：填 A1 的 Spec 表格、画 Block Diagram/State Diagram，明确输出有效周期。
- ☐ **写设计**：按 Top/IO/位宽和同步复位规则写 RTL，不急着启动工具。
- ☐ **写自检**：TB 应主动计算 Expected，有 0、最大、最小、复位/连续输入等边界。
- ☐ **先跑 T1**：不通过就只改 RTL/TB，别开始综合。
- ☐ **整理 T2 五文件**：`common_setup_gf22.tcl`、`MyDefaults.con`、`ReadVerilogFiles.tcl`、`<Top>.tcl`、`run_topdown.tcl`（外加 `report.tcl`）。
- ☐ **跑 T2**：Cell、Area、Clock、Timing、Warnings 与 netlist/SDF 均核对。
- ☐ **按截图改门级 TB**：`` `timescale ``、`$sdf_annotate`、VPD Dump；`library.f` 一定包含标准单元模型。
- ☐ **跑 T3**：确认 SDF 注释且门级功能 PASS，保存 DVE 证据。
- ☐ **跑 T4**：检查 GF22 搜索路径、VPD→VCD→SAIF、`read_saif` 层次和 Power Report。
- ☐ **写 Report**：有图有数据，能解释结果和约束限制；禁止“TEST PASS”但无真实 Checker。

E3. 如果时间不够，按依赖关系重新安排（不要伪造结果）

```
高优先级且是下游前提：Spec → RTL/TB → T1 → T2
若题目明确要求：T3 → T4 不能跳过
报告部分：用已有真实证据先写，未运行阶段明确标为未完成
```

哪些变化要重跑哪些阶段？

改动了什么	最少需要重做
RTL 内容	T1、T2、T3、T4（全部下游）
只改 TB 刺激/判定	T1、T3、T4；T2 的网表若未改则可复用
`create_clock` 或综合约束	T2、T3（如果网表/SDF 改变）、T4
`library.f` 门级模型路径	T3、T4
T3 的 `$vcdpluson` / VPD 窗口	T3、T4
Power 的 SAIF `-instance` 或工艺库路径	T4；前面正确时不用重跑 T1–T3
Report 语言/布局	仅 Report，不需要重新跑工具

F. 参考手册索引与真实性说明

文档	本 SOP 参考的原始位置
2026 Module 3 Full-Adder	P4–9 行为建模、TB、VCS/DVE；P10–16 GF22 Synthesis/五类脚本；P16–18 Gate-Level SDF 与 `library.f`；P19–20 Power；P21 Assignment
2026 Module 4 State Machine	P4–5 IO/四状态 FSM；P6–7 RTL/CLK/TB；P8 综合、SDF、功耗沿用 M3，100 ns 时钟周期；P9 Assignment + Optional Glitch
2026 Module 5 Matrix Multiplier	P3–4 Signed 8-bit Matrix Multiply、Multiplier/Accumulator/FSM、`OUT_STROBE`；P5 要求重复 M3/M4 全流程并提交简短报告

关于数据：正文的 M4 `2 FF → 4 FF`、Cell Area `6.514560 → 9.354240`、总功耗 `862.8274 → 1241.8026 nW` 来自你在先前实验对话中提供的真实 Report 摘要，**只是学习案例，不属于 M3/M5 的标准结果**。

关于脚本：B 部分自动化是为了备考写出的示例；**未进行机房实测，也不能代替 PDF 手册要求的实际验证**。如果 2026 课堂补充指示与它冲突，以老师实际 instruction 为准。

最后的核心理解：

```
RTL 是“描述功能”
Behavioral Simulation 是“检查功能”
Synthesis 是“选择和连接 GF22 门/FF”
SDF Gate Simulation 是“在门级延迟下再检查行为”
SAIF 是“把真实测试活动交给功耗工具”
Power Compiler 是“结合工艺库和活动信息估算功耗”
```

你应能随时回答：**当前执行哪种工具、读入什么文件、输出什么文件、成功凭什么证据、失败去查哪里。** 如果能做到这一点，拿到陌生的数字 IC 前端 Lab，也有能力沿相同流程独立完成。

G. 额外补充：易漏掉的“每个文件应该长什么样”

G1. .synopsys_dc.setup：DC 启动后读到什么

作用：DC/Design Vision 启动时读取的 Tcl 配置（通常自动读取），可用来设置脚本目录、库路径。你们的原工程已有这个文件，应优先保留。下面只是**最小框架说明**：

```
# .synopsys_dc.setup —— 示意，不要直接覆盖原工程
set script_path  [file normalize ./scripts]
set netlist_path [file normalize ./netlist]
set log_path     [file normalize ./logs]
set ddc_path     [file normalize ./ddc]
source ./common_setup_gf22.tcl

# 错误示范：尚未 read_verilog 就执行需要 Current Design 的命令
# set_fix_multiple_port_nets -all
```

> 如果老师的 .synopsys_dc.setup 还定义了其他 Technology / Mapping 设置，不能为了消除某个启动 UID-4 而整份替换。可以在 current_design 已建立后，按需求调用 set_fix_multiple_port_nets -all。

G2. MyDefaults.con：如何修改约束而不是乱改库

先阅读老师真实的 scripts/MyDefaults.con：

```
cd ~/NM6008/Test2_Exam/Synthesis
less scripts/MyDefaults.con
```

例如一个教学示例可能声明：

```
set CLK_PERIOD 100
set CLK_PORT "CLK"
set INPUT_DELAY  2
set OUTPUT_DELAY 2
set MAX_OUTPUT_LOAD 0.02
```

这 2 ns、0.02 是解释语法的虚构示意值，不是 Module 3/4/5 手册的默认参数，不能未经 Spec 授权用于你的实验。

接着在设计 Tcl 中真正施加约束，例如：

```
# 仅针对确实有 CLK 的题目：
create_clock -name CLK -period $CLK_PERIOD [get_ports $CLK_PORT]

# 当试卷明确给出 IO budget 时，按真实接口指定：
# set_input_delay $INPUT_DELAY -clock CLK [remove_from_collection [all_inputs] [get_ports CLK]]
# set_output_delay $OUTPUT_DELAY -clock CLK [all_outputs]
# set_load $MAX_OUTPUT_LOAD [all_outputs]
```

只写 set CLK_PERIOD 100 **并不会自动生成时钟**，还必须确认某处 create_clock 或等价命令真的使用它。report_clock 是检查实际生效约束的步骤。

G3. InputVector.txt：手册用文件驱动的版本

Module 3 手册 P6–7 用 $readmemh("./InputVector.txt", REG_DATA)。对于 3-bit 输入 (A,B,CIN)，一个可用的 InputVector.txt 内容是：

```
0
1
2
3
4
5
6
7
```

对应 TB：

```
reg [2:0] REG_DATA [0:7];
integer i;
initial begin
    $readmemh("./InputVector.txt", REG_DATA);
```

```
    for (i=0; i<8; i=i+1) begin
        {A,B,CIN}=REG_DATA[i];
        #10;
        // 这里添加 Expected-Actual Checker
    end
    $finish;
end
```

重点：`$readmemh` 的 `h` 代表把文件中的 token 按十六进制解析，而不是文件名必须以 `.hex` 结束。对于 3-bit 范围 0–7，单个字符 0–7 与二进制三位的数值对应，所以这 8 行成立；如果写 `000, 001, 010, 011` 这种二进制字符串，应改用 `$readmemb`，否则含 `010` 的 token 在 `$readmemh` 中表示十六进制 0x10，不是二进制 2。

G4. `library.f`：一行一个门级仿真库模型

结构示意（以 Module 4 机房实际出现的相对路径样式为例）：

```
-v ../Library/gf22nsdvlogl20hdl116a.v
-v ../Library/gf22nsdvlogl20hdl116a_updated.v
-v ../Library/atpg_primitives.v
-v ../Library/rtl_primitives.v
-v ../Library/std_primitives.v
+sdf_nocheck_celltype
```

不能假设这些 `.v` 在每台机器都存在。在 `rtl/` 做：

```
cat library.f
ls -lah ../Library/
```

若当前 filelist 的真实文件与模板不一致，就以工程实际 `library.f` 为准，不要复制一个自己不知道来源的 GF22 filelist。“`-v`”让 VCS 把该文件按库模块方式处理（需要时解析定义）。`+sdf_nocheck_celltype` 是特定仿真 SDF 选项，不代表关闭所有 Timing Check。

G5. `report.tcl` 最常见的“看起来生成了但数据不对”

- `report_area` 统计 Leaf Area 时，可能存在一个层次化 `u_core` 实例；**不能将层次统计中的所有行都视为独立标准单元重复计算**。
- `report_timing` 的 Setup Slack ≠ `report_constraint` 中 `max_area` 的违例 Slack。
- `report_power` 在综合阶段通常缺少 Gate-Level SAIF，不能替代最终 Power Compiler 活动驱动估算。
- 对于纯组合电路，Clock/Reg-to-Reg 报告为空是可能的，不要强行假造一个寄存器。
- `.sdc` 是约束，不是 SDF；`.ddc` 是 DC 数据库，不是 DVE 波形。

G6. 正式 Report 最简单的模板（每个数据都有对应来源）

```
# NM6008 - <Module / Design Name> Report

## 1. Objective and Design Specification
- IO table, functional equation / FSM chart, clock/reset description

## 2. RTL Implementation
- architecture and key RTL fragments, bit-width/signedness explanation

## 3. Behavioral Verification
- Testbench methodology, vectors, edge cases
- PASS log excerpt and DVE waveform (Fig. 1)

## 4. Synthesis and Timing Analysis
- GF22 library/corner, Top name, constraints
- cell references/count, FF count, cell area (Table 1)
- critical path, arrival/required/slack, constraints coverage (Fig. 2)

## 5. Post-Synthesis Simulation
- SDF annotation status, gate waveform and timing impact (Fig. 3)
- real checker results and warnings explained

## 6. Power Analysis
- VPD/VCD/SAIF preparation, activity annotation result
- internal, switching, dynamic, leakage and total power (Table 2)

## 7. Assignment Answers and Discussion
- answer exactly what this module's manual asks
- interpret trade-offs, differences, limits; do not overclaim
```

```
## 8. Conclusion
- short summary with actual quantitative evidence
```

最后一次签核：报告中不能出现类似 `user-supplied screenshot、should be checked before claiming sign-off` 这种像审查记录的文字；应该站在实验者角度客观写“测了什么、得到了什么、结论能到什么程度”。